

**ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING
DI KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD**

SKRIPSI



oleh

Muhammad Ilham
NIM. E41220495

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
2026**

**ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING
DI KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD**

SKRIPSI



sebagai salah satu syarat untuk memperoleh gelar Sarjana Sains Terapan Komputer (S.Tr.Kom)
di Program Studi Teknik Informatika
Jurusan Teknologi Informasi

oleh

Muhammad Ilham
NIM. E41220495

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
2026**

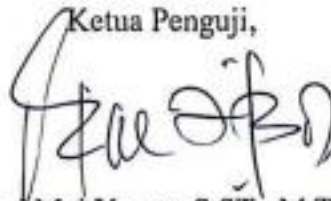
**KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
POLITEKNIK NEGERI JEMBER
JURUSAN TEKNOLOGI INFORMASI**

**ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING DI
KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD**

Muhammad Ilham (E41220495)

Telah Diuji pada Tanggal 10 Juni 2026
dan Dinyatakan Memenuhi Syarat

Ketua Penguji,



Dia Bitari Mei Yuana, S.ST., M.Tr.Kom.
NIP. 199305082 022032 013

Sekretaris Penguji,



Ery Setiawan Jullev Atmadji, S.Kom, M.Cs
NIP. 19890710 201903 1 010

Anggota Penguji,



Arvita Agus Kumiasari, S.ST., M.Tr.Kom
NIP. 199308312 024062 001

Dosen Pembimbing,



Ery Setiawan Jullev Atmadji, S.Kom, M.Cs
NIP. 19890710 201903 1 010

Mengesahkan,
Kepala Jurusan Teknologi Informasi



Riskiawan, S.Kom, M.Cs
NIP. 19830203 200604 1 003

SURAT PERNYATAAN

Saya yang bertanda tangan di bawah ini:

Nama : Muhammad Ilham

NIM : E41220495

Menyatakan dengan sebenar-benarnya bahwa segala pernyataan dalam Laporan Skripsi saya yang berjudul **“ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING DI KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD”** merupakan gagasan dan hasil karya saya sendiri dengan arahan komisi pembimbing, dan belum pernah diajukan dalam bentuk apa pun pada perguruan tinggi mana pun.

Semua data dan informasi yang digunakan telah dinyatakan secara jelas dan dapat diperiksa kebenarannya. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan oleh penulis lain telah disebutkan secara tertulis di dalam naskah dan dicantumkan ke dalam Daftar Pustaka di bagian akhir Laporan Skripsi ini.

Demikian surat pernyataan ini saya buat dengan sebenarnya. Apabila di kemudian hari terbukti terdapat penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik sesuai dengan peraturan yang berlaku di Politeknik Negeri Jember.

Jember, 10 Juni 2026



Muhammad Ilham
NIM. E41220495



**PERNYATAAN
PERSETUJUAN PUBLIKASI
KARYA ILMIAH UNTUK KEPENTINGAN
AKADEMIS**

Yang bertandatangan di bawah ini, saya:

Nama : Muhammad Ilham
NIM : E41220495
Program Studi : Teknik Informatika
Jurusan : Teknologi Informasi

Demi pengembangan Ilmu Pengetahuan, saya menyetujui untuk memberikan kepada UPT. Perpustakaan Politeknik Negeri Jember, Hak Bebas Royalti NonEksklusif (Non-Exclusive Royalty Free Right) atas Karya Ilmiah berupa **Laporan Skripsi saya yang berjudul:**

**ANALISIS IMPLEMENTASI MODSECURITY DAN
NGINX CACHING DI KUBERNETES UNTUK MITIGASI
HTTP(S) FLOOD**

Dengan Hak Bebas Royalti Non-Eksklusif ini UPT. Perpustakaan Politeknik Negeri Jember berhak menyimpan, mengalih media atau format, mengelola dalam bentuk Pangkalan Data (Database), mendistribusikan karya dan menampilkan atau mempublikasikannya di Internet atau media lain untuk kepentingan akademis tanpa perlu meminta izin dari saya selama tetap mencantumkan nama saya sebagai penulis atau pencipta.

Saya bersedia untuk menanggung secara pribadi tanpa melibatkan pihak Politeknik Negeri Jember, Segala bentuk tuntutan hukum yang timbul atas Pelanggaran Hak Cipta dalam Karya ilmiah ini.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Jember
Pada Tanggal : 10 Juni 2026
Yang menyatakan,

Nama : Muhammad Ilham
NIM : E41220495

MOTTO

“The struggle itself towards the heights is enough to fill a man’s heart.

One must imagine Sisyphus happy.”

(Albert Camus)

“Amicus Plato amicus Aristoteles magis amica veritas.”

(Sir Isaac Newton)

“Ingatlah bahwa kechilafan satu orang sahaja tjukup sudah
menjebabkan keruntuhan negara.”

(dr. Roebiono Kertopati)

“Tanpa cinta, kecerdasan itu berbahaya.
Dan tanpa kecerdasan, cinta itu tidak cukup.”

(B.J. Habibie)

“We walk the talk, not only talk the talk.
Everything will be fine, no need to worry.”

(Joko Widodo)

PERSEMBAHAN

Dengan rasa syukur yang teramat dalam, karya tulis ilmiah (skripsi) ini dipersembahkan kepada:

1. Diri sendiri, atas segala kerja keras, ketahanan mental, dan komitmen untuk pantang menyerah hingga tugas akhir ini dapat diselesaikan dengan sebaik-baiknya.
2. Orang tua tercinta, atas segala curahan doa yang tiada henti, kasih sayang, serta dukungan tulus hingga skripsi ini dapat diselesaikan tepat pada waktunya.
3. Bapak Ery Setyawan Jullev Atmadji, S.Kom., M.Cs., selaku Dosen Pembimbing, atas keikhlasan, kesabaran, serta ilmu yang telah diberikan dalam mengarahkan setiap langkah penyusunan skripsi ini.
4. Segenap dosen dan staf pengajar Politeknik Negeri Jember, khususnya pada Program Studi Teknik Informatika, atas segala dedikasi dalam membagikan ilmu pengetahuan yang sangat berharga.
5. Sahabat-sahabat terdekat, teristimewa rekan-rekan satu tempat tinggal, atas segala motivasi, kepedulian, dan kebersamaan yang menjadi penyemangat di setiap proses pengerjaan skripsi ini.
6. Rekan-rekan seperjuangan mahasiswa Teknik Informatika angkatan 2022, atas segala solidaritas, bantuan, dan kenangan berharga yang telah diukir bersama selama menempuh perjalanan akademik ini.

ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING DI KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD

Pembimbing Ery Setyawan Jullev Atmadji, S.Kom., M.Cs.

Muhammad Ilham
Program Studi Teknik Informatika
Jurusan Teknologi Informasi

ABSTRAK

Serangan *Distributed Denial of Service* (DDoS), khususnya jenis *HTTP(S) Flood*, menjadi ancaman serius bagi ketersediaan layanan web karena kemampuannya dalam menguras sumber daya server. Penelitian ini menganalisis implementasi arsitektur pertahanan ganda (*dual-layer defense*) yang mengombinasikan ModSecurity sebagai *Web Application Firewall* (WAF) dan *Nginx Caching* pada kluster Kubernetes untuk memitigasi serangan tersebut. Pengujian dilakukan menggunakan metode *incremental load testing* dan *stress testing* hingga beban 1.000 *concurrent users* melalui perangkat Apache JMeter. Hasil eksperimen menunjukkan bahwa penerapan mekanisme mitigasi tunggal menyisakan celah kerentanan kritis, seperti manipulasi *cache bypass* atau terjadinya efek *bottleneck* komputasi CPU yang parah pada gerbang *Ingress*. Sebaliknya, penggabungan kedua metode tersebut terbukti secara efektif mampu mengisolasi *Application Pods* dari ancaman kelumpuhan layanan (*downtime*), menjaga konsumsi CPU internal tetap stabil di bawah 25 *millicores*, serta mencapai tingkat pemblokiran serangan yang konsisten sebesar 95,00%. Kendati demikian, ketahanan infrastruktur kluster pada akhirnya dibatasi oleh spesifikasi perangkat keras fisik (2 *core* CPU), yang memicu lonjakan anomali *Context Switches* serta fenomena *CPU throttling* saat menerima tekanan beban ekstrem.

Kata kunci: Kubernetes, ModSecurity, Nginx, Caching, HTTP(S) Flood, DDoS, *Web Application Firewall*, *Nested Virtualization*, Proxmox, Apache JMeter

ANALYSIS OF MODSECURITY AND NGINX CACHING IMPLEMENTATION IN KUBERNETES FOR HTTP(S) FLOOD MITIGATION

Supervisor Ery Setyawan Jullev Atmadji, S.Kom., M.Cs.

Muhammad Ilham
Study Program of Informatics Engineering
Majoring of Information Technology

ABSTRACT

Distributed Denial of Service (DDoS) attacks, particularly HTTP(S) Floods, pose a severe threat to web service availability as they rapidly exhaust server resources. This study analyzes the implementation of a dual-layer defense architecture that combines ModSecurity as a Web Application Firewall (WAF) and Nginx Caching within a Kubernetes cluster to mitigate these attacks. Testing was executed using incremental load and stress testing scenarios with up to 1,000 concurrent users conducted via Apache JMeter. The experimental results demonstrate that deploying a single mitigation mechanism leaves critical vulnerabilities open to cache bypass manipulations and causes severe CPU utilization bottlenecks at the Ingress gateway. In contrast, combining both methods effectively isolates Application Pods against service downtime, maintaining internal CPU consumption consistently below 25 millicores and achieving a stable attack blocking rate of 95.00%. However, the overall cluster resilience remains constrained by the physical hardware limitations (2 CPU cores), triggering elevated context switches and CPU throttling under extreme load pressures.

Keywords: Kubernetes, ModSecurity, Nginx, Caching, HTTP(S) Flood, DDoS, Web Application Firewall, Nested Virtualization, Proxmox, Apache JMeter

RINGKASAN

ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING DI KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD, Muhammad Ilham, NIM. E41220495, Tahun 2026, 160 halaman., Teknologi Informasi, Politeknik Negeri Jember, Ery Setyawan Jullev Atmadji, S.Kom., M.Cs. (Pembimbing).

Serangan *Distributed Denial of Service* (DDoS) jenis *HTTP(S) Flood* menjadi ancaman serius yang menguras sumber daya server dan melumpuhkan ketersediaan layanan. Karakteristik serangan ini mengeksploitasi layer aplikasi dengan mengirimkan volume permintaan dalam jumlah masif. Tanpa perlindungan yang mumpuni, arsitektur *backend* tradisional akan mengalami saturasi proses pekerja (*worker saturation*) hingga memicu kelumpuhan total layanan (*downtime*).

Penelitian ini bertujuan menganalisis implementasi arsitektur pertahanan ganda (*dual-layer defense*) yang mengombinasikan *Web Application Firewall* (WAF) ModSecurity dan *Nginx Caching*. Kedua komponen tersebut diintegrasikan secara simultan pada gerbang *NGINX Ingress Controller* di dalam kluster Kubernetes multi-node. Pengujian dilakukan menggunakan Apache JMeter melalui skenario *incremental load testing* dan *stress testing* dengan simulasi beban bertahap hingga 1.000 pengguna virtual (*concurrent users*) secara simultan.

Hasil pengujian membuktikan bahwa penerapan mekanisme mitigasi tunggal menyisakan celah kerentanan kritis, seperti manipulasi *cache bypass* atau efek *bottleneck* CPU pada gerbang *Ingress*. Sebaliknya, kombinasi kedua metode sukses mengisolasi *Application Pods* dari ancaman *downtime*, menjaga konsumsi komputasi internal tetap stabil di bawah 25 *millicores*, dan mencapai tingkat pemblokiran serangan di atas 95,00%. Kendati demikian, ketahanan infrastruktur pada akhirnya dibatasi oleh spesifikasi perangkat keras (2 *core* CPU *host*), yang memicu lonjakan anomali *Context Switches* serta fenomena *CPU throttling* saat menerima tekanan beban ekstrem.

PRAKATA

Puji syukur senantiasa penulis panjatkan kehadiran Allah SWT, karena berkat rahmat, taufik, dan hidayah-Nya, penyusunan laporan skripsi yang berjudul **“ANALISIS IMPLEMENTASI MODSECURITY DAN NGINX CACHING DI KUBERNETES UNTUK MITIGASI HTTP(S) FLOOD”** dapat diselesaikan dengan baik.

Laporan skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Sains Terapan Komputer (S.Tr.Kom) pada Program Studi Teknik Informatika, Jurusan Teknologi Informasi, Politeknik Negeri Jember.

Pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Direktur Politeknik Negeri Jember.
2. Ketua Jurusan Teknologi Informasi.
3. Ketua Program Studi Teknik Informatika.
4. Bapak Ery Setyawan Jullev Atmadji, S.Kom., M.Cs., selaku Dosen Pembimbing yang telah memberikan bimbingan serta arahan.
5. Orang tua dan keluarga tercinta yang senantiasa memberikan doa, motivasi, serta dukungan baik secara moral maupun material.
6. Rekan-rekan seperjuangan dan seluruh pihak yang tidak dapat disebutkan satu per satu.

Laporan skripsi ini masih jauh dari kata sempurna karena keterbatasan pengetahuan dan pengalaman. Akhir kata, semoga laporan skripsi ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan.

Jember, 10 Juni 2026



Penulis

DAFTAR ISI

	Halaman
HALAMAN SAMPUL.....	i
HALAMAN JUDUL	ii
HALAMAN PENGESAHAN.....	iii
SURAT PERNYATAAN	iv
PERSETUJUAN PUBLIKASI.....	v
MOTTO	vi
PERSEMBAHAN.....	vii
ABSTRAK	viii
RINGKASAN	x
PRAKATA.....	xi
DAFTAR ISI.....	xii
DAFTAR GAMBAR.....	xvi
DAFTAR TABEL.....	xix
DAFTAR LAMPIRAN	xxii
BAB 1. PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah	3
1.3 Tujuan.....	3
1.4 Manfaat.....	4
1.5 Batasan Masalah	4
BAB 2. TINJAUAN PUSTAKA.....	5
2.1 Virtualisasi dan Container	5
2.2 Hypervisor	5
2.3 Proxmox Virtual Environment.....	6
2.4 Kernel.....	7
2.5 Model OSI Layer.....	8
2.5.1 <i>Application Layer</i> (Lapisan 7)	9
2.5.2 <i>Presentation Layer</i> (Lapisan 6)	9

2.5.3 <i>Session Layer</i> (Lapisan 5).....	10
2.5.4 <i>Transport Layer</i> (Lapisan 4).....	10
2.5.5 <i>Network Layer</i> (Lapisan 3)	11
2.5.6 <i>Data Link Layer</i> (Lapisan 2).....	11
2.5.7 <i>Physical Layer</i> (Lapisan 1)	12
2.6 Kubernetes	12
2.6.1 Arsitektur Jaringan Kubernetes	13
2.6.2 Komponen Jaringan dalam Kubernetes	14
2.7 Ingress	15
2.7.1 <i>Ingress Controller</i>	15
2.7.2 Fungsi dan Manfaat Ingress	16
2.7.3 Kerentanan dan Perlindungan	16
2.8 Nginx	16
2.9 Caching	17
2.10 DDoS (<i>Distributed Denial of Service</i>)	18
2.11 HTTP(S) Flood	20
2.12 WAF (<i>Web Application Firewall</i>)	21
2.13 ModSecurity	22
2.14 Apache JMeter	24
2.15 <i>State of the Art</i>	25
BAB 3. METODE PENELITIAN	29
3.1 Waktu dan Tempat	29
3.2 Tahapan Penelitian	29
3.2.1 Fase Inisiasi dan Studi Literatur	30
3.2.2 Fase Perancangan dan Pembuatan Sistem	30
3.2.3 Fase Pengujian dan Pengumpulan Data.....	30
3.2.4 Fase Analisis, Validasi, dan Penarikan Kesimpulan.....	31
3.3 Perancangan Sistem	31
3.3.1 Perancangan Lingkungan Virtual (<i>Before Environment</i>)	31
3.3.2 Perancangan Topologi Kluster (<i>Before Topology</i>)	33
3.3.3 Integrasi ModSecurity dan NGINX Caching pada Ingress	35

3.3.4 Alur Proses Kerja Sistem Mitigasi.....	38
3.4 Pengujian Inkremental.....	38
3.4.1 Klasifikasi Metode Pengujian.....	39
3.4.2 Penentuan Skenario Pengujian.....	40
3.4.3 Prosedur Pengujian	40
3.4.4 Metrik Pengukuran	41
3.4.5 Hasil yang Diharapkan.....	42
3.5 Analisis Hasil Pengujian.....	42
3.5.1 Pemrosesan Data.....	42
3.5.2 Perbandingan Skenario	43
3.5.3 Interpretasi	43
3.5.4 Kesimpulan Awal	43
3.6 Validasi dan Verifikasi Sistem.....	44
3.6.1 Validasi Sistem	44
3.6.2 Verifikasi Sistem.....	44
BAB 4. HASIL DAN PEMBAHASAN	46
4.1 Hasil Studi Literatur dan Spesifikasi Infrastruktur	46
4.1.1 <i>Software</i>	47
4.1.2 <i>Hardware</i>	48
4.2 Implementasi dan Konfigurasi Sistem.....	50
4.2.1 Pemasang Proxmox.....	50
4.2.2 Konfigurasi Antarmuka Jaringan.....	52
4.2.3 Persiapan Kontainer untuk Kubernetes.....	55
4.2.4 Deployment Aplikasi pada Kubernetes	59
4.3 Skenario Pengujian.....	62
4.3.1 Model Serangan HTTP(S) Flood.....	62
4.3.2 Parameter Beban dan Intensitas Serangan	63
4.3.3 Konfigurasi Pengujian pada Apache JMeter	63
4.3.4 Pengambilan dan Ekstraksi Data	67
4.3.5 Validasi Perhitungan Metrik Pengujian.....	68
4.4 Hasil Pengujian Idle Sistem	70

4.4.1 Performa Idle WordPress.....	71
4.4.2 Performa Idle Ghost.....	72
4.4.3 Performa Idle Drupal	73
4.4.4 Hasil Komparasi Idle Sistem	73
4.4.5 Hasil Komparasi Alokasi Resource Idle Sistem	74
4.5 Hasil Pengujian Baseline Sistem.....	76
4.5.1 Performa Baseline WordPress	76
4.5.2 Performa Baseline Ghost	79
4.5.3 Performa Baseline Drupal.....	81
4.5.4 Hasil Komparasi Baseline Sistem.....	83
4.5.5 Hasil Komparasi Alokasi Resource Baseline Sistem ...	84
4.5.6 Hasil Komparasi Utilisasi Resource Baseline Sistem...	85
4.6 Hasil Pengujian Incremental Load Sistem	87
4.6.1 Pengujian Incremental Cache Only	87
4.6.2 Pengujian Incremental ModSecurity Only	94
4.6.3 Pengujian Incremental Kombinasi	100
4.7 Hasil Pengujian Stress Testing Sistem	108
4.7.1 Performa Stress Testing WordPress	108
4.7.2 Performa Stress Testing Ghost	109
4.7.3 Performa Stress Testing Drupal	110
4.7.4 Hasil Komparasi Stress Testing Sistem	111
4.7.5 Hasil Komparasi Alokasi Resource Stress Test Sistem	113
4.7.6 Hasil Komparasi Utilisasi Resource Stress Test Sistem	114
4.8 Perbandingan Skenario Pengujian.....	115
4.9 Analisis Hasil Pengujian Secara Keseluruhan	118
BAB 5. KESIMPULAN DAN SARAN.....	121
5.1 Kesimpulan.....	121
5.2 Saran	122
DAFTAR PUSTAKA.....	124
LAMPIRAN.....	133

DAFTAR GAMBAR

	Halaman
Gambar 2. 1 Proxmox (Sumber: Proxmox)	7
Gambar 2. 2 Model OSI 7 Layer (Sumber: Cloudflare)	8
Gambar 2. 3 <i>Application Layer</i> (Sumber: Cloudflare).....	9
Gambar 2. 4 <i>Presentation Layer</i> (Sumber: Cloudflare).....	10
Gambar 2. 5 <i>Session Layer</i> (Sumber: Cloudflare)	10
Gambar 2. 6 <i>Transport Layer</i> (Sumber: Cloudflare)	11
Gambar 2. 7 <i>Network Layer</i> (Sumber: Cloudflare).....	11
Gambar 2. 8 <i>Data Link Layer</i> (Sumber: Cloudflare)	11
Gambar 2. 9 <i>Physical Layer</i> (Sumber: Cloudflare)	12
Gambar 2. 10 Arsitektur Kubernetes	13
Gambar 2. 11 NGINX (Sumber: NGINX).....	17
Gambar 2. 12 <i>Caching Process</i>	18
Gambar 2. 13 <i>DDoS Attack</i>	19
Gambar 2. 14 <i>HTTP(S) Flood Attack</i>	20
Gambar 2. 15 Mekanisme <i>Web Application Firewall</i>	22
Gambar 2. 16 ModSecurity (Sumber: ModSecurity).....	23
Gambar 2. 17 Apache JMeter (Sumber: Apache JMeter)	24
Gambar 3. 1 Diagram Alir Tahapan Penelitian.....	30
Gambar 3. 2 Arsitektur Proxmox sebagai Hypervisor (<i>Before Environment</i>)	32
Gambar 3. 3 Kluster Kubernetes Multi-Node (<i>Before Topology</i>).....	34
Gambar 3. 4 Flowchart Sistem.....	35
Gambar 3. 5 Arsitektur Alur Data Sistem Mitigasi.....	36
Gambar 3. 6 Bagan Klasifikasi Metode Pengujian	39
Gambar 4. 1 Spesifikasi Sistem	49
Gambar 4. 2 Antarmuka Manajemen pada Microsoft Azure	49
Gambar 4. 3 Antarmuka Manajemen pada Amazon Web Services.....	50
Gambar 4. 4 File <code>/etc/hosts</code>	51
Gambar 4. 5 Konfigurasi Modules.....	51

Gambar 4. 6 Status pveversion.....	51
Gambar 4. 7 Antarmuka Manajemen Proxmox	52
Gambar 4. 8 Konfigurasi <i>Virtual Bridge (vmbf0)</i>	53
Gambar 4. 9 Aktivasi <i>IP Forwarding</i> pada Konfigurasi Kernel Sistem.....	53
Gambar 4. 10 Konfigurasi <i>Postrouting</i> NAT.....	53
Gambar 4. 11 Konfigurasi NGINX Stream Reverse Proxy	54
Gambar 4. 12 Status Service containerd	55
Gambar 4. 13 Konfigurasi cgroup.....	55
Gambar 4. 14 Verifikasi Versi Komponen Kubernetes	56
Gambar 4. 15 Daftar LXC pada Dashboard Proxmox	56
Gambar 4. 16 Daftar <i>nodes</i> di Kubernetes	57
Gambar 4. 17 Keepalived <i>Virtual IP</i> Klaster	57
Gambar 4. 18 Status <i>Ready</i> Setelah Instalasi <i>Network Plugin</i>	58
Gambar 4. 19 Direktori Penyimpanan NFS	58
Gambar 4. 20 Antarmuka Manajemen WordPress	59
Gambar 4. 21 Antarmuka Manajemen Ghost	60
Gambar 4. 22 Antarmuka Manajemen Drupal.....	61
Gambar 4. 23 Konfigurasi Thread Group	64
Gambar 4. 24 Konfigurasi HTTP Request.....	64
Gambar 4. 25 Regular Expression Extractor Cache Status.....	65
Gambar 4. 26 Regular Expression Extractor WAF Status.....	65
Gambar 4. 27 Grafik Komparasi Idle Sistem.....	74
Gambar 4. 28 Grafik Komparasi Resource Idle Sistem	75
Gambar 4. 29 Grafik Komparasi Baseline Sistem	83
Gambar 4. 30 Grafik Komparasi Resource Baseline Sistem	85
Gambar 4. 31 Grafik Komparasi Kernel Baseline Sistem	86
Gambar 4. 32 Grafik Hasil Komparasi Base Cache Sistem.....	88
Gambar 4. 33 Grafik Komparasi Resource Base Cache	89
Gambar 4. 34 Grafik Komparasi Kernel Base Cache	90
Gambar 4. 35 Grafik Hasil Komparasi Cache Inject Sistem	91
Gambar 4. 36 Grafik Komparasi Resource Cache Inject.....	92

Gambar 4. 37 Grafik Komparasi Kernel Cache Inject	93
Gambar 4. 38 Grafik Komparasi Bypass ModSecurity Sistem.....	94
Gambar 4. 39 Grafik Komparasi Resource Bypass ModSecurity.....	95
Gambar 4. 40 Grafik Komparasi Kernel Bypass ModSecurity.....	96
Gambar 4. 41 Grafik Komparasi ModSecurity Payload Sistem	97
Gambar 4. 42 Grafik Komparasi Resource ModSecurity Payload	98
Gambar 4. 43 Grafik Komparasi Kernel ModSecurity Payload	99
Gambar 4. 44 Grafik Komparasi Kombinasi Sistem	104
Gambar 4. 45 Grafik Komparasi Resource Kombinasi Sistem.....	105
Gambar 4. 46 Grafik Komparasi Kernel Kombinasi Sistem.....	107
Gambar 4. 47 Grafik Komparasi Stress Testing Sistem	112
Gambar 4. 48 Grafik Komparasi Resource Stress Testing Sistem.....	113
Gambar 4. 49 Grafik Komparasi Kernel Stress Testing Sistem.....	114

DAFTAR TABEL

	Halaman
Tabel 2. 1 <i>State of the Art</i>	25
Tabel 3. 1 Waktu Penelitian	29
Tabel 3. 2 Skenario Pengujian Inkremental	40
Tabel 4. 1 Konfigurasi Payload.....	62
Tabel 4. 2 Intensitas Beban	63
Tabel 4. 3 Sampel Data Pengujian JMeter	68
Tabel 4. 4 Validasi Perhitungan Manual Metrik Pengujian	70
Tabel 4. 5 Performa Idle WordPress	71
Tabel 4. 6 Konsumsi Daya pada Pod WordPress.....	71
Tabel 4. 7 Performa Idle Ghost	72
Tabel 4. 8 Konsumsi Daya pada Pod Ghost.....	72
Tabel 4. 9 Performa Idle Drupal	73
Tabel 4. 10 Konsumsi Daya pada Pod Drupal	73
Tabel 4. 11 Komparasi Idle Sistem	74
Tabel 4. 12 Hasil Komparasi Resource Idle Sistem (Node)	75
Tabel 4. 13 Performa Baseline WordPress	77
Tabel 4. 14 Konsumsi Sumber Daya Pod WordPress.....	77
Tabel 4. 15 Utilisasi Resource Kernel WordPress	78
Tabel 4. 16 Performa Baseline Ghost	79
Tabel 4. 17 Konsumsi Sumber Daya Pod Ghost.....	79
Tabel 4. 18 Utilisasi Resource Kernel Ghost	80
Tabel 4. 19 Performa Baseline Drupal	81
Tabel 4. 20 Konsumsi Sumber Daya Pod Drupal	81
Tabel 4. 21 Utilisasi Resource Kernel Drupal	82
Tabel 4. 22 Hasil Komparasi Baseline Sistem	83
Tabel 4. 23 Komparasi Resource Baseline Sistem.....	84
Tabel 4. 24 Komparasi Utilisasi Resource Baseline Sistem	86
Tabel 4. 25 Performa Aplikasi pada Skenario Base Cache.....	88

Tabel 4. 26 Konsumsi Resource Pod pada Skenario Base Cache.....	88
Tabel 4. 27 Utilisasi Kernel Host Proxmox Base Cache	89
Tabel 4. 28 Performa Aplikasi pada Skenario Cache Inject Query	91
Tabel 4. 29 Resource Pod pada Skenario Cache Inject Query.....	92
Tabel 4. 30 Utilisasi Kernel Host Proxmox Cache Inject Query	92
Tabel 4. 31 Performa Aplikasi pada Skenario Bypass ModSecurity	94
Tabel 4. 32 Resource Pod pada Skenario Bypass ModSecurity	95
Tabel 4. 33 Utilisasi Kernel Host Proxmox Bypass ModSecurity	95
Tabel 4. 34 Performa Aplikasi pada Skenario ModSecurity Payload.....	97
Tabel 4. 35 Resource Pod pada Skenario ModSecurity Payload.....	98
Tabel 4. 36 Utilisasi Kernel Host Proxmox ModSecurity Payload	98
Tabel 4. 37 Performa Kombinasi WordPress.....	100
Tabel 4. 38 Resource Pod pada Skenario Kombinasi WordPress.....	100
Tabel 4. 39 Utilisasi Kernel Host Proxmox Kombinasi WordPress	101
Tabel 4. 40 Performa Kombinasi Ghost.....	101
Tabel 4. 41 Resource Pod pada Skenario Kombinasi Ghost.....	102
Tabel 4. 42 Utilisasi Kernel Host Proxmox Kombinasi Ghost	102
Tabel 4. 43 Performa Kombinasi Drupal	102
Tabel 4. 44 Resource Pod pada Skenario Kombinasi Drupal	103
Tabel 4. 45 Utilisasi Kernel Host Proxmox Kombinasi Drupal.....	103
Tabel 4. 46 Hasil Komparasi Performa Skenario Kombinasi	104
Tabel 4. 47 Hasil Komparasi Resource Pod Skenario Kombinasi.....	105
Tabel 4. 48 Hasil Komparasi Kernel Host Skenario Kombinasi	106
Tabel 4. 49 Performa Stress Testing WordPress.....	108
Tabel 4. 50 Resource Pod pada Skenario Stress Testing WordPress.....	108
Tabel 4. 51 Utilisasi Kernel Host Proxmox Stress Testing WordPress	109
Tabel 4. 52 Performa Stress Testing Ghost.....	109
Tabel 4. 53 Resource Pod pada Skenario Stress Testing Ghost.....	110
Tabel 4. 54 Utilisasi Kernel Host Proxmox Stress Testing Ghost	110
Tabel 4. 55 Performa Stress Testing Drupal	110
Tabel 4. 56 Resource Pod pada Skenario Stress Testing Drupal	111

Tabel 4. 57 Utilisasi Kernel Host Proxmox Stress Testing Drupal.....	111
Tabel 4. 58 Hasil Komparasi Performa Stress Testing Sistem	111
Tabel 4. 59 Hasil Komparasi Resource Pod Stress Testing Sistem	113
Tabel 4. 60 Hasil Komparasi Kernel Host Stress Testing Sistem.....	114
Tabel 4. 61 Perbandingan Pengujian WordPress	116
Tabel 4. 62 Perbandingan Pengujian Ghost	116
Tabel 4. 63 Perbandingan Pengujian Drupal.....	117

DAFTAR LAMPIRAN

	Halaman
Lampiran 1 Set Up Proxmox.....	133
Lampiran 2 Manage Network Interface	135
Lampiran 3 Configuration Lxc (Host Proxmox).....	137
Lampiran 4 Preparing Containers For Kubernetes.....	139
Lampiran 5 Runtime Container.....	140
Lampiran 6 Installing Kubeadm, Kubectl, And Kubelet	142
Lampiran 7 Creating An LXC Template	143
Lampiran 8 Load Balancing On Master Node	144
Lampiran 9 Join And Configure The Network Plugin.....	146
Lampiran 10 NFS Server and Postfix Configuration (Host Proxmox).....	149
Lampiran 11 Nginx Ingress Deployment.....	150
Lampiran 12 Load Testing Automation.....	155

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Pesatnya pertumbuhan dan penggunaan internet di berbagai sektor telah mendorong transformasi digital di Indonesia (Gunawan dkk., 2020). Di era teknologi informasi saat ini, kebutuhan akan ketersediaan tinggi (*high availability*) dan keandalan layanan digital menjadi sangat kritis (Šimon dkk., 2023). Untuk menjawab tantangan tersebut, sistem aplikasi modern semakin mengadopsi arsitektur berbasis kontainer, di mana Kubernetes telah muncul sebagai platform orkestrasi terkemuka yang memungkinkan organisasi mengelola dan menyebarkan aplikasi berskala besar secara efisien (Aqasizade dkk., 2024). Keandalan sistem berbasis web ini sangat penting untuk memastikan kelancaran operasional, terutama saat menghadapi periode dengan aktivitas atau beban trafik yang tinggi (Annisa dkk., 2026). Di sisi lain, peningkatan penggunaan layanan digital berskala besar ini juga diiringi dengan meningkatnya ancaman keamanan siber yang mengintai infrastruktur tersebut.

Berdasarkan laporan APJII (Asosiasi Penyelenggara Jasa Internet Indonesia) tahun 2024, jumlah pengguna internet di Indonesia mencapai 221 juta jiwa (Asosiasi Penyelenggara Jasa Internet Indonesia, 2026). Angka tersebut mencakup 79,5% dari total populasi yang dilaporkan oleh BPS (Badan Pusat Statistik) yaitu sebanyak 281,6 juta jiwa (Badan Pusat Statistik, 2026). Data ini menggambarkan skala besar penetrasi internet di Indonesia, sekaligus memperlihatkan potensi risiko keamanan siber yang ada di Indonesia.

Ancaman keamanan siber di Indonesia tercermin dari laporan BSSN (Badan Siber dan Sandi Negara) tahun 2024, yang mencatat sebanyak 330,5 juta anomali lalu lintas jaringan. Salah satu ancaman utama yang diprediksi terus meningkat pada tahun 2025 adalah serangan DDoS (*Distributed Denial of Service*). Ancaman ini menjadi perhatian karena dapat mengganggu operasional layanan digital secara signifikan (Badan Siber dan Sandi Negara, 2025).

DDoS, khususnya serangan HTTP(S) Flood, merupakan salah satu ancaman yang berdampak signifikan untuk mengganggu ketersediaan layanan website dengan membanjiri server target dengan permintaan HTTP(S) seperti *GET* dan *POST* secara berlebihan (Sarmah dkk., 2025). Serangan HTTP(S) Flood, memanfaatkan permintaan yang tampak sah atau menyerupai trafik pengguna asli dan kompleks atau berat untuk diproses dengan tujuan menghabiskan sumber daya server, sehingga menyebabkan penurunan performa atau kelumpuhan layanan (Fadhilah dkk., 2023). Dalam banyak kasus, sistem tidak mengalami kegagalan total, tetapi mengalami degradasi layanan yang berdampak pada menurunnya kualitas layanan. Untuk itu dibutuhkan solusi dalam mengatasi masalah penurunan performa atau kelumpuhan pada layanan server.

Untuk mengatasi permasalahan tersebut, berbagai pendekatan telah dikembangkan. Dari sisi keamanan, solusi seperti ModSecurity dapat diterapkan. ModSecurity adalah WAF (*Web Application Firewall*) yang merupakan alat keamanan berbasis aturan yang dapat dikonfigurasi untuk mendeteksi dan mencegah serangan siber, termasuk DDoS. Dengan aturan yang telah ditetapkan, ModSecurity mampu mengidentifikasi pola serangan dan memblokirnya (Zain dkk., 2023). Fadhilah dkk. (2023) menunjukkan bahwa serangan DDoS jenis SlowHTTP dapat dimitigasi secara efektif menggunakan ModSecurity dan ModAntiLoris, yang dimana sangat relevan untuk strategi perlindungan terhadap serangan HTTP(S) Flood. Selain itu, Indrajid, dkk. (2023) menganalisis dampak serangan SYN Flood, menegaskan kerentanan infrastruktur web terhadap variasi serangan DDoS.

Melengkapi solusi keamanan seperti ModSecurity, diperlukan pendekatan tambahan dari sisi performa. Pemanfaatan teknologi modern seperti Kubernetes dengan fitur Nginx Caching dapat meningkatkan efisiensi dan ketahanan infrastruktur web terhadap lonjakan lalu lintas jaringan (Gautham S dkk., 2021). Dengan caching, Nginx dapat menyimpan konten statis atau respons yang dihasilkan sementara, sehingga meminimalkan pemrosesan berulang terhadap permintaan HTTP(S) yang berat, seperti yang sering dimanfaatkan dalam serangan HTTP(S) Flood.

Efektivitas penerapan ModSecurity sebagai mekanisme filtering serta Nginx Caching sebagai mekanisme optimasi dalam meningkatkan ketersediaan layanan pada sistem berbasis Kubernetes menjadi aspek penting yang perlu dianalisis. Penelitian ini berfokus pada pendekatan berlapis yang menggabungkan ModSecurity dan Nginx Caching dalam Kubernetes untuk menciptakan strategi yang kuat dalam mitigasi serangan HTTP(S) Flood. ModSecurity berperan dalam mendeteksi dan memblokir permintaan berbahaya melalui aturan keamanan, sementara Nginx Caching mengurangi beban server dengan menyimpan respons yang sering diakses, menjaga ketersediaan layanan bagi pengguna sah.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana kinerja layanan Content Management System pada kluster Kubernetes tanpa mekanisme mitigasi dalam menghadapi serangan HTTP(S) Flood?
2. Bagaimana pengaruh penerapan Nginx Caching terhadap performa layanan Content Management System pada kluster Kubernetes dalam kondisi serangan HTTP(S) Flood?
3. Bagaimana pengaruh penerapan ModSecurity terhadap ketersediaan layanan (*availability*) Content Management System pada sistem berbasis Kubernetes?
4. Bagaimana efektivitas kombinasi ModSecurity dan Nginx Caching dalam meningkatkan ketersediaan layanan Content Management System dan performa sistem pada kluster Kubernetes?

1.3 Tujuan

Tujuan dari penelitian ini adalah:

1. Menganalisis kinerja layanan Content Management System pada kluster Kubernetes tanpa mekanisme mitigasi terhadap serangan HTTP(S) Flood.
2. Menganalisis pengaruh Nginx Caching terhadap performa layanan Content Management System pada kluster Kubernetes.

3. Menganalisis pengaruh ModSecurity terhadap ketersediaan layanan (*availability*) Content Management System pada kluster Kubernetes.
4. Menganalisis efektivitas kombinasi ModSecurity dan Nginx Caching dalam meningkatkan ketersediaan layanan Content Management System dan performa system pada kluster Kubernetes.

1.4 Manfaat

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

1. Memberikan referensi empiris mengenai efektivitas mekanisme mitigasi HTTP(S) Flood pada lingkungan Kubernetes.
2. Menjadi dasar pertimbangan dalam memilih pendekatan keamanan (*filtering*) dan optimasi performa (*caching*) dalam menjaga ketersediaan layanan.
3. Memberikan gambaran mengenai trade-off antara keamanan dan performa dalam sistem berbasis container.

1.5 Batasan Masalah

Untuk menjaga ruang lingkup penelitian tetap terfokus dan dapat diselesaikan sesuai waktu dan sumber daya yang tersedia, penelitian ini dibatasi pada hal-hal berikut:

1. Penelitian difokuskan pada mitigasi serangan HTTP(S) Flood dan tidak mencakup jenis serangan DDoS lainnya.
2. Pengujian dilakukan menggunakan simulasi serangan HTTP(S) Flood dengan Apache JMeter.
3. Implementasi menggunakan ModSecurity dengan aturan standar (OWASP Core Rule Set) dan Nginx Caching berbasis reverse proxy.
4. Sistem diuji dalam lingkungan Kubernetes dengan konfigurasi yang telah ditentukan.
5. Parameter pengujian meliputi latency, throughput, dan request per second dalam kondisi beban terkontrol.

BAB 2. TINJAUAN PUSTAKA

2.1 Virtualisasi dan Container

Virtualisasi adalah teknologi yang memungkinkan abstraksi sumber daya fisik untuk menjalankan beberapa sistem terisolasi di atas satu perangkat keras. Secara umum, terdapat dua pendekatan utama: virtualisasi berbasis *Hypervisor* (Virtual Machines) dan virtualisasi tingkat sistem operasi (*Containers*).

Menurut Silva dkk. (2024), meskipun virtualisasi tradisional berbasis mesin virtual (VM) menawarkan isolasi yang kuat, teknologi ini memiliki *overhead* yang besar karena setiap VM harus menjalankan kernel sistem operasinya sendiri. Sebagai alternatif, teknologi *container* seperti Linux Containers (LXC) menawarkan solusi yang lebih ringan (*lightweight*). LXC bekerja dengan berbagi kernel sistem operasi host dan menggunakan fitur kernel Linux seperti *namespaces* dan *control groups* (cgroups) untuk isolasi.

Wiedner dkk. (2024) menambahkan bahwa kontainer sangat unggul dalam skenario yang membutuhkan latensi rendah dan efisiensi penggunaan sumber daya, karena tidak adanya lapisan emulasi perangkat keras yang biasanya ditemukan pada VM tradisional. Hal ini membuat kontainer menjadi pilihan utama untuk pengemasan aplikasi yang membutuhkan performa mendekati *bare-metal* namun tetap terisolasi secara logis.

2.2 Hypervisor

Hypervisor, yang juga dikenal sebagai *Virtual Machine Manager* (VMM), adalah lapisan perangkat lunak, *firmware*, atau perangkat keras yang berfungsi untuk membuat, mengelola, dan menjalankan *Virtual Machines*. Peran utama hypervisor adalah mengalokasikan sumber daya fisik (seperti CPU, memori, dan penyimpanan) secara dinamis ke setiap VM dan menjaga isolasi antar VM tersebut (Aalam dkk., 2021).

Berdasarkan arsitekturnya, hypervisor secara umum dikategorikan menjadi dua jenis:

1. Type-1 (Bare-metal)

Berjalan langsung di atas perangkat keras fisik tanpa memerlukan sistem operasi host. Jenis ini menawarkan performa dan keamanan yang lebih tinggi karena memiliki *overhead* yang rendah dan akses langsung ke instruksi CPU.

2. Type-2 (Hosted)

Berjalan sebagai aplikasi di atas sistem operasi host (seperti Windows atau Linux). Jenis ini lebih mudah dikonfigurasi namun memiliki latensi yang lebih tinggi karena harus melewati lapisan sistem operasi host untuk mengakses perangkat keras.

Dalam perkembangannya, hypervisor tidak hanya digunakan untuk efisiensi server, tetapi juga sebagai fondasi keamanan sistem. Salah satu contohnya adalah penggunaan *thin hypervisor* atau hypervisor minimalis yang dirancang khusus untuk mengamankan operasi spesifik, seperti perlindungan data input dari *keylogger* atau pengamanan *clipboard* melalui enkripsi, dengan cara melakukan intersepsi pada level instruksi istimewa (*privileged instructions*) sebelum mencapai sistem operasi (Kiperberg & Zaidenberg, 2023). Karena posisinya yang berada di bawah sistem operasi (Ring -1), hypervisor memiliki kontrol penuh dan visibilitas terhadap seluruh aktivitas sistem, menjadikannya komponen krusial dalam infrastruktur IT modern (Aalam dkk., 2021).

2.3 Proxmox Virtual Environment

Proxmox Virtual Environment adalah platform manajemen virtualisasi *open-source* berbasis Debian Linux yang menggabungkan dua teknologi virtualisasi utama, yaitu KVM (*Kernel-based Virtual Machine*) dan LXC (*Linux Containers*). LXC adalah teknologi virtualisasi tingkat sistem operasi yang memungkinkan untuk menjalankan beberapa sistem Linux yang terisolasi (*container*) pada satu host fisik, menggunakan satu kernel Linux yang sama. Sebagai antarmuka *userspace* untuk fitur *containment* kernel (seperti *cgroups* dan

namespaces), LXC sangat ringan, efisien, dan ideal untuk membuat lingkungan sistem utuh (*system container*) yang terisolasi. Menurut analisis Šimon dkk. (2023), Proxmox menonjol karena kemampuannya memberikan ketersediaan tinggi (*High Availability*) untuk infrastruktur kontainer Linux.



Gambar 2. 1 Proxmox (Sumber: Proxmox)

Dalam pengujian yang dilakukan oleh Šimon dkk. (2023), Proxmox terbukti sangat efektif dalam skenario yang memprioritaskan kecepatan transfer data dan kemudahan manajemen melalui antarmuka berbasis web. Proxmox memungkinkan administrator untuk mengelola penyimpanan secara terpusat dan melakukan pemantauan performa secara real-time. Bagi penelitian ini, pemilihan Proxmox Virtual Environment sangat krusial karena platform ini mendukung pengoperasian LXC secara *native* pada arsitektur amd64, sehingga memungkinkan pemanfaatan fitur-fitur kernel host secara maksimal untuk mendukung beban kerja server yang efisien.

2.4 Kernel

Kernel merupakan inti dari sistem operasi yang berperan dalam mengelola sumber daya sistem seperti CPU, memori, dan proses. Performa kernel memiliki pengaruh signifikan terhadap stabilitas dan efisiensi sistem, terutama dalam manajemen penggunaan memori dan penjadwalan CPU pada berbagai versi kernel (Godinho dkk., 2025). Selain itu, modifikasi kernel (seperti penggunaan patch *real-time*) terbukti dapat meningkatkan responsivitas sistem melalui pengurangan *latency* untuk mencapai determinisme yang lebih baik (Adam dkk., 2021). Dalam perkembangannya, kernel modern juga mulai mendukung mekanisme penyesuaian performa secara dinamis (*in-situ tuning*) melalui parameter performa yang dapat

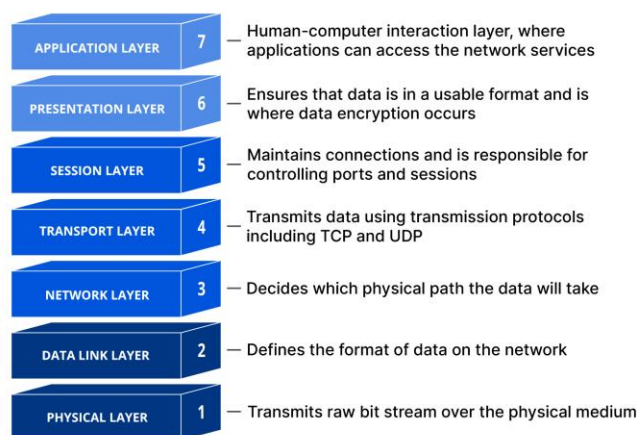
diprogram untuk menyesuaikan dengan perubahan beban kerja secara fleksibel (Chen dkk., 2025).

Meskipun kernel Linux dapat diimplementasikan pada berbagai arsitektur seperti ARM, (Adam dkk., 2021), penelitian ini secara khusus berfokus pada arsitektur amd64 (x86-64). Pemilihan ini didasarkan pada fakta bahwa amd64 merupakan standar utama dalam infrastruktur server dan lingkungan virtualisasi. Hal ini memastikan bahwa fitur-fitur manajemen sumber daya kernel, seperti efisiensi memori dan mekanisme penyesuaian dinamis, dapat berjalan dengan kompatibilitas serta performa yang optimal (Godinho dkk., 2025).

2.5 Model OSI Layer

Model OSI (*Open Systems Interconnection*) adalah kerangka kerja konseptual yang terdiri dari tujuh lapisan yang masing-masing memiliki fungsi spesifik dalam komunikasi data antar sistem. Model ini membagi proses komunikasi menjadi *Physical*, *Data Link*, *Network*, *Transport*, *Session*, *Presentation*, dan *Application Layer*, guna menyederhanakan desain dan interoperabilitas jaringan.

Menurut Permana dkk. (2024), pemahaman terhadap model OSI tetap krusial di era Industri 4.0. Struktur lapisannya memberikan kerangka kerja yang jelas dalam mendesain sistem komunikasi yang efisien, aman, dan interoperabel, khususnya pada domain *Internet of Things (IoT)*, *cloud computing*, dan *big data*.



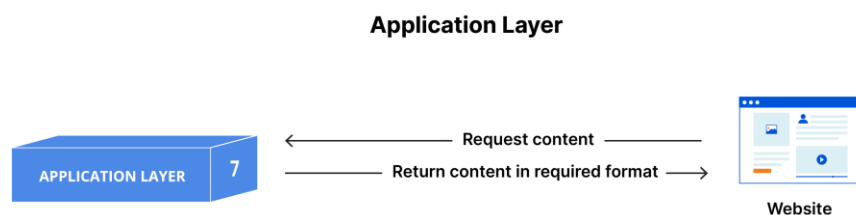
Gambar 2. 2 Model OSI 7 Layer (Sumber: Cloudflare)

Dalam konteks penelitian ini, jenis serangan yang diamati adalah HTTP(S) Flood, yang bekerja pada Layer 7 (*Application Layer*). Untuk itu, pemahaman terhadap keseluruhan model OSI penting agar dapat ditempatkan mitigasi secara tepat, terutama pada lapisan yang menjadi target utama serangan.

2.5.1 *Application Layer* (Lapisan 7)

Berfungsi sebagai antarmuka antara pengguna/aplikasi dan jaringan. Protokol seperti HTTP, FTP, DNS, dan SMTP bekerja di lapisan ini, memungkinkan aplikasi membaca dan menulis data melalui jaringan.

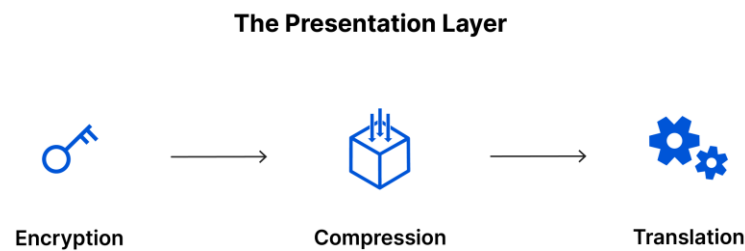
Serangan HTTP(S) Flood mengeksploitasi lapisan ini dengan cara mengirim permintaan HTTP(S) (GET/POST) yang tampak sah dan menyerupai trafik pengguna asli guna menguras sumber daya server (Fadhilah dkk., 2023). Hal ini menyebabkan metode pertahanan tradisional di lapisan bawah (seperti *packet filtering* pada Layer 3/4) kurang efektif, sehingga diperlukan mitigasi spesifik seperti *Web Application Firewall* (WAF) dan mekanisme *rate limiting* yang mampu melakukan inspeksi paket secara mendalam (*Deep Packet Inspection*).



Gambar 2. 3 *Application Layer* (Sumber: Cloudflare)

2.5.2 *Presentation Layer* (Lapisan 6)

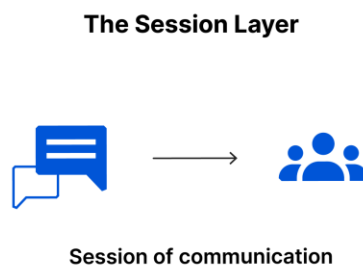
Bertanggung jawab untuk translasi data, enkripsi/dekripsi, dan kompresi, menjamin konsistensi format dan keamanan data agar aplikasi di kedua ujung dapat memahami dan memprosesnya.



Gambar 2. 4 *Presentation Layer* (Sumber: Cloudflare)

2.5.3 *Session Layer* (Lapisan 5)

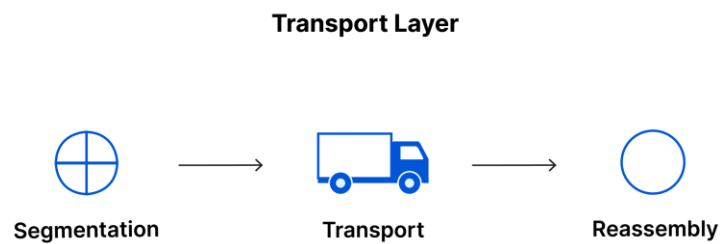
Mengelola sesi koneksi termasuk pembukaan, pemeliharaan, dan penutupan. Fitur seperti *checkpointing* memungkinkan pemulihan *transfer* data dari titik tertentu jika koneksi terganggu.



Gambar 2. 5 *Session Layer* (Sumber: Cloudflare)

2.5.4 *Transport Layer* (Lapisan 4)

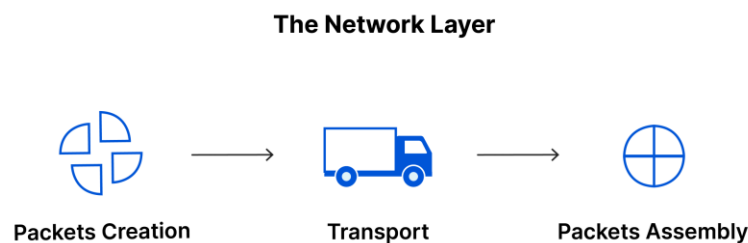
Memastikan pengiriman data *end-to-end* yang andal. Protokol seperti TCP (untuk koneksi *handshaking* & urutan data) dan UDP (tanpa jaminan urutan) bekerja di sini, lengkap dengan mekanisme flow control dan error control.



Gambar 2. 6 *Transport Layer* (Sumber: Cloudflare)

2.5.5 *Network Layer* (Lapisan 3)

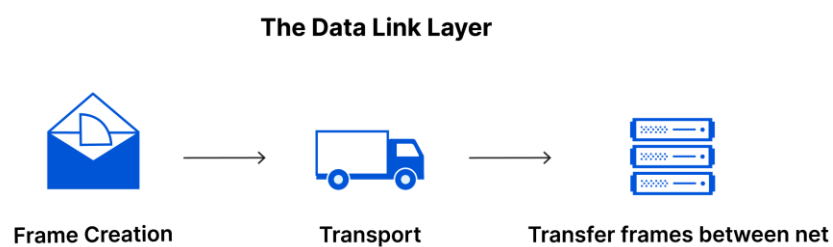
Memfasilitasi routing dan pengalamatan logis (digital), seperti penggunaan alamat IP untuk membantu paket melalui jalur optimal antar jaringan.



Gambar 2. 7 *Network Layer* (Sumber: Cloudflare)

2.5.6 *Data Link Layer* (Lapisan 2)

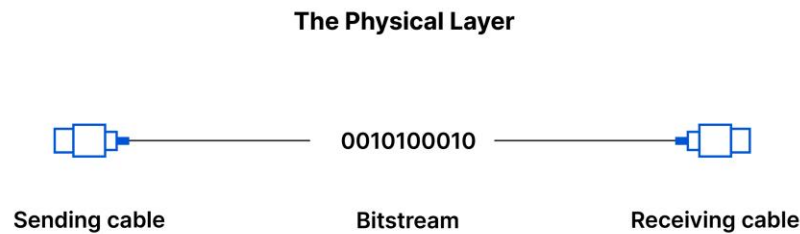
Mengatur pengiriman *frame* antar perangkat dalam satu jaringan lokal, termasuk manajemen alamat fisik (MAC), *error detection*, dan *flow control* tingkat link.



Gambar 2. 8 *Data Link Layer* (Sumber: Cloudflare)

2.5.7 Physical Layer (Lapisan 1)

Bertanggung jawab atas transmisi fisik bit 0 dan 1 melalui media seperti kabel, gelombang listrik, atau gelombang radio. Spesifikasi sinyal, voltase, dan pengkodean berada di lapisan ini.



Gambar 2. 9 *Physical Layer* (Sumber: Cloudflare)

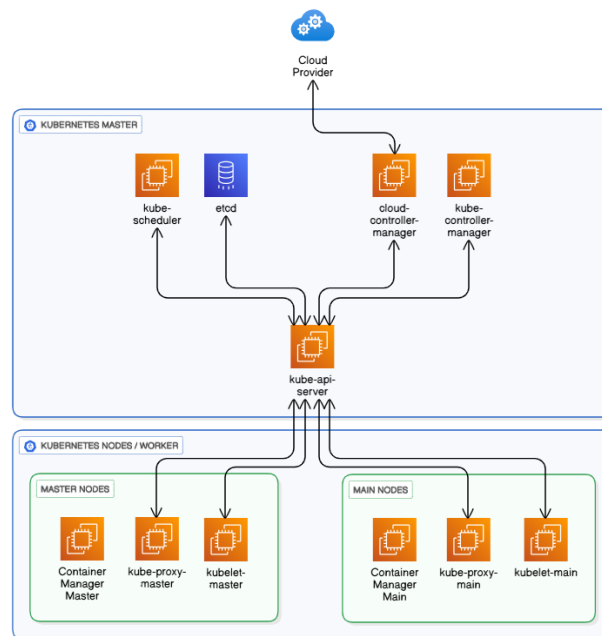
Model OSI tidak hanya menjadi panduan teknis, tapi juga alat analisis dalam keamanan siber, karena memungkinkan klasifikasi jenis serangan berdasarkan lapisan target. Dalam konteks ini, HTTP(S) Flood pada Layer 7 membutuhkan solusi di lapisan aplikasi, yang berbeda dari serangan seperti UDP Flood yang terjadi pada Layer 3 atau SYN Flood yang terjadi pada Layer 4.

2.6 Kubernetes

Kubernetes merupakan platform orkestrasi kontainer *open-source* yang dirancang untuk mengelola aplikasi terdistribusi dalam skala besar secara otomatis. Melalui *Pod*, *Deployment*, *Service*, dan *Ingress*, Kubernetes menyederhanakan proses penyebaran, pemeliharaan, dan penskalaan aplikasi kontainer di berbagai lingkungan. Platform ini secara *native* mendukung *auto-scaling*, *load balancing*, serta pemulihan otomatis (*self-healing*) terhadap *pod* yang gagal (Gautham S dkk., 2021)

Dukungan terhadap skalabilitas dan ketahanan terhadap lonjakan lalu lintas membuat Kubernetes cocok digunakan untuk sistem yang memerlukan ketersediaan tinggi (Ramadhani & Ridha, 2022). Arsitektur publik seperti *Ingress* yang mengekspos layanan internal tetap memiliki kerentanan terhadap serangan

Distributed Denial of Service (DDoS) jika tidak dikonfigurasi dengan benar. Oleh karena itu, keamanan pada lapisan aplikasi seperti *rate limiting* dan *Web Application Firewall* (WAF) menjadi penting untuk mencegah eksploitasi dan menjaga kestabilan sistem.



Gambar 2. 10 Arsitektur Kubernetes

2.6.1 Arsitektur Jaringan Kubernetes

Arsitektur jaringan dalam Kubernetes dirancang untuk mendukung komunikasi yang andal dan fleksibel, baik antar komponen internal (*pod*, *service*) maupun antara pengguna eksternal dan sistem di dalam *cluster*. Berdasarkan gambar 2.10 Arsitektur Kubernetes di atas, jaringan Kubernetes terdiri atas dua bagian utama yaitu *Control Plane* (*Master*) dan *Node* (*Worker*), dengan komponen-komponen khusus yang saling terhubung melalui API.

a. *Control Plane* (*Master*)

1. *kube-apiserver*

Titik utama komunikasi antar komponen Kubernetes. Semua *request*, baik dari pengguna maupun komponen internal, masuk melalui API Server.

2. *etcd*

Penyimpanan konfigurasi *cluster* dalam format *key-value*. Menyimpan status semua *resource* dalam *cluster*.

3. *kube-scheduler*

Menentukan *node* terbaik untuk menempatkan *pod* baru berdasarkan *resource*, kebijakan, dan *constraints*.

4. *controller-manager*

Menjalankan berbagai *controller* yang memantau status *resource* dan memastikan kesesuaian dengan *desired state* (misalnya, *deployment controller*, *replication controller*).

b. *Node (Worker)*

1. *kubelet*

Agen utama pada *node* yang mengatur dan memantau *pod*.

2. *kube-proxy*

Mengelola *load balancing* dan jaringan layanan. Meneruskan permintaan dari *service* ke *pod* yang sesuai berdasarkan IP dan port.

3. *Container Manager*

Komponen ini menjalankan kontainer melalui *container runtime* yang kompatibel dengan *Container Runtime Interface* (CRI), seperti *containerd*.

2.6.2 Komponen Jaringan dalam Kubernetes

a. *Pod Networking*

Setiap *pod* dalam Kubernetes memiliki alamat IP unik yang membuatnya dapat berkomunikasi langsung dengan *pod* lain, bahkan lintas *node*. Untuk mendukung hal ini, Kubernetes memanfaatkan *Container Network Interface* (CNI) *plugin* seperti Calico, Flannel, atau Weave untuk mengatur alokasi IP, *routing*, dan kebijakan jaringan antar *pod*. Jaringan ini bersifat *flat*, artinya tidak menggunakan NAT, sehingga koneksi antar *pod* berjalan secara langsung.

b. *Service Networking*

Karena IP *pod* bersifat tidak tetap (*ephemeral*), Kubernetes menyediakan objek *service* sebagai cara stabil untuk mengakses sekumpulan *pod*. *Service* menyediakan *load balancing internal* dan dapat diakses melalui beberapa tipe:

1. *ClusterIP, default* hanya dapat diakses dari dalam *cluster*.
2. *NodePort*, mengekspos port layanan di setiap *node* agar bisa diakses dari luar *cluster*.
3. *LoadBalancer*, menggunakan *load balancer* eksternal (biasanya dari *cloud provider*) untuk menyediakan *endpoint* publik.

Komponen *kube-proxy* di setiap *node* akan meneruskan trafik dari *client* ke *pod* berdasarkan pengaturan *service* tersebut.

2.7 Ingress

Ingress adalah objek API Kubernetes yang mengelola akses eksternal ke layanan di dalam klaster, terutama lalu lintas HTTP(S). Ingress bekerja pada lapisan aplikasi (Layer 7), sehingga mendukung pengaturan trafik berbasis konten (*content-based routing*) melalui aturan rute berbasis *hostname* atau *path*. Hal ini memberikan fleksibilitas lebih tinggi dibandingkan dengan *service* tipe *NodePort* atau *LoadBalancer* yang bekerja di lapisan *transport* (Layer 4) dan hanya melakukan *routing* berdasarkan port atau IP (Ramadhani & Ridha, 2022).

Ingress Controller, seperti NGINX, bertugas mengimplementasikan aturan tersebut serta mendukung fitur tambahan seperti terminasi SSL/TLS, *load balancing*, dan integrasi dengan *Web Application Firewall* (WAF) untuk mitigasi serangan (Gautham S dkk., 2021).

2.7.1 Ingress Controller

Agar objek Ingress dapat berfungsi, diperlukan Ingress Controller sebagai komponen aktif yang membaca dan mengeksekusi aturan (*rules*) yang didefinisikan. Ingress Controller berjalan sebagai *pod* di dalam klaster dan dapat berbasis berbagai *engine*.

Dalam penelitian ini, NGINX Ingress Controller digunakan karena popularitasnya dalam mendukung konfigurasi granular, performa tinggi, serta kemudahan integrasi dengan modul keamanan seperti ModSecurity.

2.7.2 Fungsi dan Manfaat Ingress

1. Routing Berdasarkan *Path* dan *Hostname*

Mengarahkan permintaan ke berbagai layanan melalui satu alamat IP publik tunggal.

2. *Load Balancing*

Mendistribusikan trafik secara merata ke sejumlah *pod backend* untuk menjaga stabilitas.

3. *TLS Termination*

Menangani proses enkripsi/dekripsi HTTP(S) di level Ingress, sehingga menyederhanakan manajemen sertifikat pada sisi aplikasi.

4. Pengendalian Akses dan Keamanan

Bertindak sebagai gerbang utama (*gateway*) yang memungkinkan penerapan kebijakan keamanan sebelum trafik mencapai aplikasi.

2.7.3 Kerentanan dan Perlindungan

Ingress merupakan titik masuk utama yang terekspos ke publik, sehingga rentan terhadap serangan *Distributed Denial of Service (DDoS)*. Penelitian oleh Ramadhani & Ridha (2022) menunjukkan bahwa tanpa konfigurasi keamanan yang tepat, NGINX Ingress dapat mengalami lonjakan penggunaan CPU hingga 99% saat menerima beban tinggi melalui pengujian *stress testing*. Kondisi ini mengindikasikan risiko kelumpuhan sistem akibat *resource exhaustion*.

2.8 Nginx

Nginx merupakan web server *open-source* yang juga dapat digunakan sebagai *reverse proxy* dengan performa yang tinggi. Nginx dikenal karena kemampuannya menangani permintaan dalam jumlah besar dengan penggunaan sumber daya yang relatif ringan. Salah satu keunggulan Nginx adalah arsitektur *event-driven* dan *non-blocking I/O* yang memungkinkannya menangani ribuan koneksi konkuren dengan *overhead* memori yang minimal, sehingga sangat efektif dalam memitigasi

serangan yang mencoba menghabiskan sumber daya *thread* server (Azi dkk., 2023). Selain itu, Nginx dilengkapi fitur keamanan seperti *rate limiting*, dukungan SSL/TLS, dan akses kontrol berbasis IP yang menjadikannya populer untuk pengamanan dan optimasi performa web server.



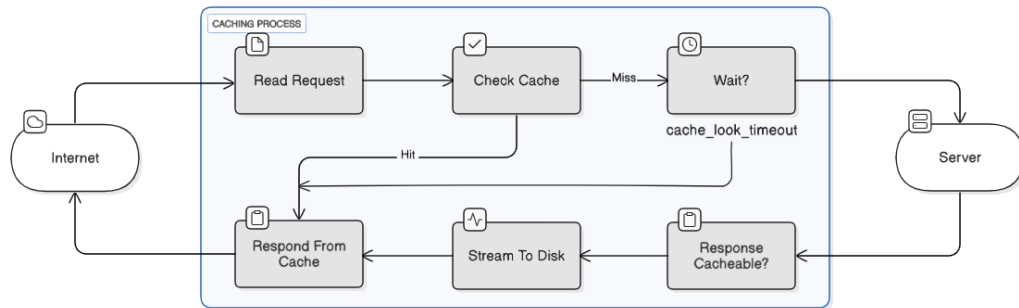
Gambar 2. 11 NGINX (Sumber: NGINX)

Dalam ekosistem Kubernetes, Nginx sering diimplementasikan sebagai Ingress Controller, yang bertindak sebagai titik masuk tunggal (*entry point*) untuk mengelola lalu lintas HTTP(S) dari luar kluster menuju layanan internal.

2.9 Caching

Implementasi caching di Nginx memberikan dampak positif terhadap performa web server, terutama dalam skenario dengan trafik tinggi atau permintaan berulang. Caching di Nginx bekerja dengan menyimpan respons statis atau dinamis untuk melayani permintaan berulang tanpa membebani server backend (Azi dkk., 2023).

Efisiensi ini tidak hanya berdampak pada kecepatan akses pengguna, tetapi juga menjadi strategi penting dalam mitigasi serangan seperti HTTP(S) Flood. Dengan menggunakan mekanisme caching, beban komputasi untuk menghasilkan respons (terutama konten dinamis) dialihkan dari *backend* ke lapisan *proxy*. Hal ini secara signifikan mengurangi penggunaan CPU dan RAM pada server aplikasi saat terjadi serangan *low-and-slow* atau *high-volume* HTTP(S) Flood.



Gambar 2. 12 Caching Process

Mekanisme Caching pada Nginx meliputi:

1. Proxy Cache

Digunakan untuk menyimpan respons HTTP(S) pada disk atau memori. Mekanisme ini memungkinkan Nginx untuk langsung melayani permintaan yang identik tanpa perlu meneruskannya ke server *backend*.

2. FastCGI Cache

Berfungsi untuk mengurangi beban pemrosesan pada aplikasi dinamis seperti PHP (misalnya melalui PHP-FPM). Dengan menyimpan hasil eksekusi skrip sebelumnya, FastCGI cache membantu mengurangi penggunaan sumber daya komputasi di sisi aplikasi secara signifikan.

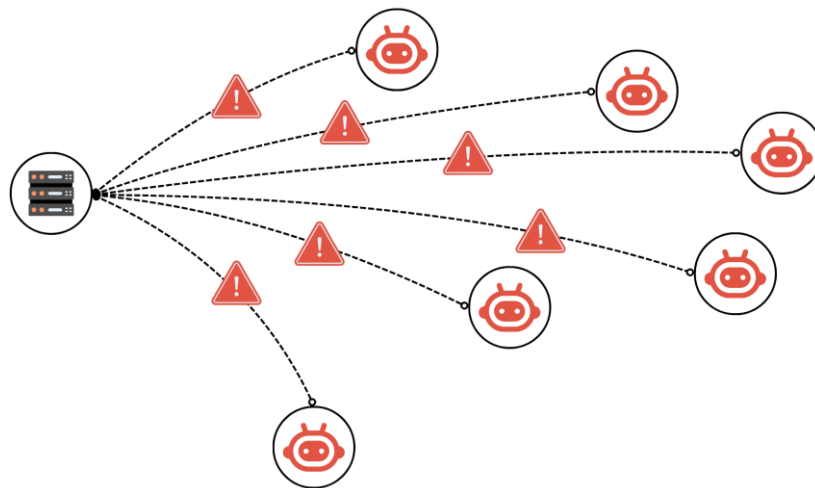
3. Static File Cache

Meskipun Nginx secara *native* sudah sangat efisien dalam melayani file statis (seperti gambar, CSS, dan JavaScript), penggunaan *header caching* yang tepat (seperti Cache-Control atau Expires) dapat meningkatkan performa lebih lanjut dengan menginstruksikan *browser* atau *reverse proxy* tingkat atas untuk menyimpan data tersebut secara lokal.

2.10 DDoS (*Distributed Denial of Service*)

Serangan DDoS (*Distributed Denial of Service*) adalah jenis serangan DoS yang dilakukan oleh banyak komputer secara terkoordinasi untuk mengganggu atau melemahkan server tertentu (Zain dkk., 2023). Serangan DDoS adalah upaya untuk mengganggu ketersediaan layanan dengan cara mengirimkan ribuan permintaan

koneksi (*request*) secara bersamaan hingga membebani atau memenuhi kapasitas jaringan maupun aplikasi target dengan lalu lintas data berlebihan dari berbagai sumber, sehingga mengakibatkan kelumpuhan layanan (*service outage*) di mana server tidak dapat melayani permintaan pengguna sah secara normal (Arman, 2020).



Gambar 2. 13 *DDoS Attack*

Jenis-jenis serangan DDoS meliputi:

a. Volumetric Attacks

Membanjiri bandwidth jaringan menggunakan trafik besar, seperti UDP Flood (mengirim paket UDP ke port acak) atau ICMP Flood (membanjiri dengan ping), untuk memenuhi kapasitas jaringan hingga *bandwidth* habis.

b. Protocol Attacks

Mengeksploitasi kelemahan protokol jaringan, seperti SYN Flood, yang memanfaatkan mekanisme handshake TCP tanpa menyelesaikannya, menyebabkan server kehabisan sumber daya untuk koneksi baru.

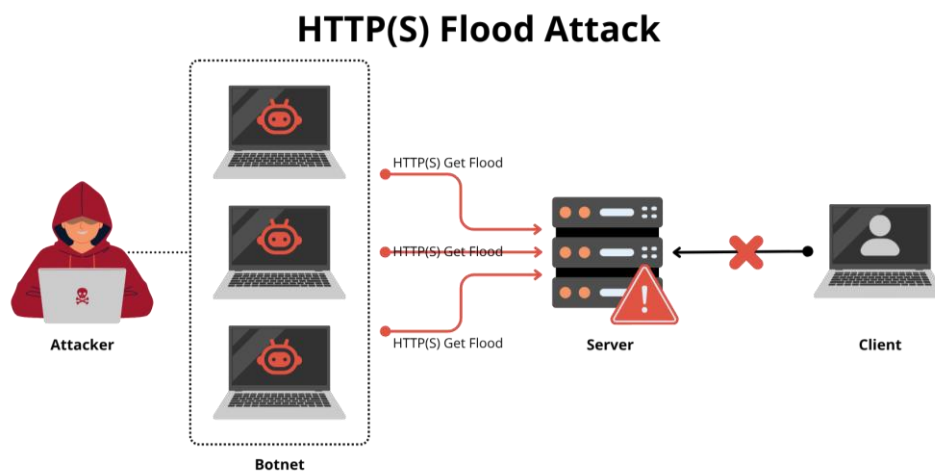
c. Application Layer Attacks

Menyerang lapisan aplikasi, seperti HTTP(S) Flood, yang menjadi fokus penelitian ini karena menargetkan aplikasi web dengan permintaan yang tampak sah atau sulit dideteksi karena menyerupai trafik pengguna asli. Meyerang

dengan cara mengirim banyak permintaan HTTP/HTTPS yang tampak normal namun bertujuan membebani server.

2.11 HTTP(S) Flood

Serangan HTTP(S) Flood adalah jenis serangan DDoS pada lapisan aplikasi yang membanjiri web server dengan permintaan HTTP(S) seperti *GET/POST* yang tampak sah atau menyerupai trafik pengguna asli. Meskipun tidak menghabiskan banyak *bandwidth*, serangan ini sangat efektif karena memaksa server memproses setiap permintaan, sehingga menguras sumber daya seperti CPU, memori, dan thread koneksi. Penyerang sering menggunakan botnet untuk melakukan serangan secara terkoordinasi dari berbagai sumber, membuatnya lebih sulit dideteksi dan dibendung (Sarmah dkk., 2025).



Gambar 2. 14 *HTTP(S) Flood Attack*

Jenis-jenis serangan HTTP(S) Flood meliputi:

a. GET Flood

engirimkan permintaan GET secara berulang untuk mengakses sumber daya seperti gambar, CSS, atau halaman HTML. Tanpa mekanisme *caching* yang optimal, server akan kewalahan memproses permintaan yang terus-menerus ini.

b. POST Flood

Mengirimkan permintaan POST yang sering kali disertai data kompleks (seperti formulir atau akses API). Karena POST memerlukan pemrosesan lebih berat di sisi server (seperti penulisan ke *database* atau validasi), serangan ini lebih cepat menghabiskan sumber daya dibandingkan GET Flood.

c. Slowloris

Metode serangan yang bekerja dengan mengirimkan permintaan HTTP parsial (tidak lengkap) secara perlahan. Tujuannya adalah untuk menahan slot koneksi server selama mungkin hingga mencapai batas maksimal (*connection limit*), sehingga server tidak dapat melayani permintaan baru.

d. RUDY (R U Dead Yet)

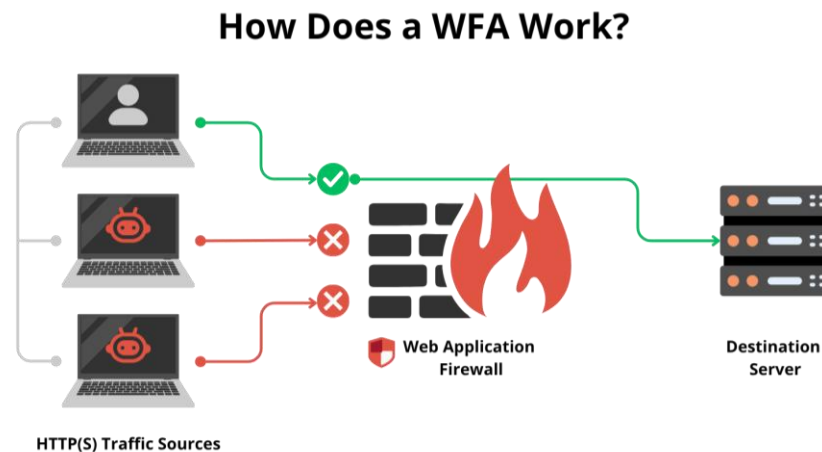
Serangan ini mengirimkan data POST dalam potongan-potongan kecil secara sangat lambat. Hal ini memaksa server untuk mengalokasikan sumber daya dalam waktu yang lama untuk satu koneksi, yang akhirnya menyebabkan kelelahan sumber daya (*resource exhaustion*).

2.12 WAF (*Web Application Firewall*)

Web Application Firewall (WAF) adalah komponen keamanan yang dirancang khusus untuk memantau, menyaring, dan memblokir lalu lintas HTTP(S) yang mencurigakan dari dan menuju aplikasi web. Berbeda dengan *firewall* tradisional yang bekerja pada lapisan jaringan, WAF beroperasi pada lapisan aplikasi (Layer 7) untuk mencegah eksploitasi kerentanan seperti *SQL Injection*, *Cross-Site Scripting* (XSS), dan serangan DDoS tipe HTTP(S) Flood dengan cara memeriksa setiap permintaan dan respons berdasarkan kumpulan aturan keamanan (*ruleset*).

WAF berfungsi mendeteksi pola serangan, menerapkan kebijakan *rate limiting*, serta menyediakan log aktivitas untuk analisis keamanan lebih lanjut. Meskipun efektif dalam memberikan perlindungan tambahan, implementasi WAF dapat meningkatkan latensi sistem atau menyebabkan *false positives* (pemblokiran trafik sah) jika aturan yang diterapkan terlalu ketat (*high paranoia level*). Oleh karena itu, diperlukan penyesuaian (*tuning*) yang mendalam untuk mencapai

keseimbangan antara tingkat keamanan dan produktivitas sistem (Thomas-Reynolds & Butakov, 2020).



Gambar 2. 15 Mekanisme *Web Application Firewall*

2.13 ModSecurity

ModSecurity adalah modul *Web Application Firewall* (WAF) bersifat *open-source* yang berfungsi melindungi aplikasi web dengan melakukan inspeksi lalu lintas HTTP(S) secara *real-time*. Dalam penelitian ini, ModSecurity berperan sebagai instrumen keamanan utama yang diintegrasikan dengan Nginx Ingress Controller untuk mendeteksi dan memitigasi berbagai ancaman pada lapisan aplikasi.

ModSecurity mendukung mitigasi serangan HTTP(S) Flood melalui penerapan berbagai kebijakan keamanan, termasuk penggunaan OWASP *Core Rule Set* (CRS) yang menyediakan aturan berbasis tanda tangan (*signature-based*) dan heuristik yang terus diperbarui untuk memberikan perlindungan terhadap kerentanan kritis. Aturan-aturan di dalam CRS memungkinkan sistem untuk mengidentifikasi pola serangan umum seperti *SQL Injection*, *Cross-Site Scripting* (XSS), serta berbagai varian *Distributed Denial of Service* (DDoS), termasuk serangan berbasis *flooding* yang menargetkan sumber daya aplikasi (Zain dkk., 2023). Dengan kemampuan analisis paket yang mendalam (*Deep Packet Inspection*), ModSecurity mampu memberikan keputusan seketika untuk

mengizinkan, memantau, atau memblokir permintaan yang dianggap mencurigakan sebelum permintaan tersebut membebani server *backend*.



Gambar 2. 16 ModSecurity (Sumber: ModSecurity)

ModSecurity dirancang dengan arsitektur modular yang memungkinkan pemrosesan permintaan HTTP(S) secara *real-time* berdasarkan aturan yang telah dikonfigurasi. Komponen utama dalam arsitektur ModSecurity meliputi:

- a. Memproses permintaan berdasarkan aturan CRS untuk mendeteksi pola serangan.
- b. Parser HTTP(S) atau mengurai *header* dan *body* permintaan untuk analisis keamanan. Dengan proses parsing yang mendalam, ModSecurity mampu mengidentifikasi muatan berbahaya yang tersembunyi dalam permintaan.
- c. Sistem *Logging*, yaitu mencatat detail seperti *log audit*, termasuk alamat IP asal, waktu serangan, jenis permintaan, dan kode kesalahan untuk analisis pasca-serangan.

ModSecurity lebih efisien saat diintegrasikan dengan Nginx karena kemampuan Nginx menangani lalu lintas tinggi. Konfigurasi seperti *SecRuleEngine On* dan *SecAuditLog* memungkinkan pemantauan *real-time*.

Meskipun ModSecurity memiliki kinerja yang baik dalam memblokir serangan, hal tersebut juga mencatat bahwa penggunaan CPU dan waktu respon meningkat saat ModSecurity aktif, yang menandakan adanya beban tambahan pada system (Fadhilah dkk., 2023).

2.14 Apache JMeter

Apache JMeter adalah alat *open-source* untuk pengujian beban dan performa, sering digunakan dalam pengujian sistem berbasis HTTP/HTTPS karena kemampuannya dalam membuat skenario dengan jumlah permintaan yang tinggi untuk mengukur efektivitas solusi mitigasi (Ramadhani & Ridha, 2022). Dalam konteks keamanan siber, JMeter juga digunakan untuk mensimulasikan serangan seperti HTTP(S) Flood guna menguji ketahanan sistem terhadap lonjakan trafik.



Gambar 2. 17 Apache JMeter (Sumber: Apache JMeter)

JMeter mendukung pengujian protokol HTTP/HTTPS, memungkinkan simulasi permintaan *GET/POST* dalam jumlah besar. Fitur seperti *Thread Group* dan *HTTP Request* memungkinkan mensimulasikan lalu lintas serangan atau pengguna normal.

Dalam penelitian ini, JMeter akan digunakan untuk mensimulasikan serangan HTTP(S) Flood dengan konfigurasi seperti 1000 *thread* untuk menghasilkan hingga 1000 permintaan per detik, menguji ketahanan ModSecurity dan Nginx Caching.

2.15 State of the Art

Dalam penelitian ini, beberapa studi terdahulu dijadikan sebagai panduan atau acuan untuk penelitian yang dilakukan, guna mendukung dan memperkuat pelaksanaan serta keberhasilan penelitian ini.

Tabel 2. 1 *State of the Art*

Komponen	Keterangan Jurnal Terdahulu
Peneliti	Ayu Rosyida Zain, Iik Muhamad Malik Matin dan Kevin Kautsar
Judul	Analisis Implementasi Modsecurity dan Reverse Proxy Untuk Pencegahan Serangan Keamanan DDoS pada Web Server
Tahun	2023
Keterangan	Menyatakan bahwa integrasi ModSecurity dan reverse proxy membentuk perlindungan ganda terhadap serangan DDoS, dengan ModSecurity memblokir serangan secara efektif (100% keberhasilan terhadap Bash Script, GoldenEye, dan HULK) dan reverse proxy memperlambat serangan dengan menambah waktu tempuh rata-rata 53,12509 ms.
Peneliti	Fariz Fadhilah, Eka Wahyudi, Eko Fajar Cahyadi
Judul	Analisis Modsecurity Dan Modantiloris Pada Serangan DDoS Slowhttp Terhadap Webserver
Tahun	2023
Keterangan	Menyatakan bahwa ModSecurity dan ModAntiLoris efektif melindungi web server dari serangan DDoS SlowHTTP, dengan ModSecurity memblokir serangan melalui aturan firewall dan ModAntiLoris mencegah koneksi lambat, sehingga meningkatkan ketahanan server terhadap ancaman tersebut.
Peneliti	Zata Ismah Sumayyah, Silva Dimas Surya Permana, Muhammad Tsabit, Aep Setiawan
Judul	Penerapan dan Mitigasi Teknik Slowloris dalam Serangan Distributed Denial-of-Service (DDoS) terhadap Website Ilegal dengan Kali Linux
Tahun	2024
Keterangan	Menyatakan simulasi serangan Slowloris berhasil membuat website target tidak responsif. Teknik mitigasi seperti pengaturan timeout pada NGINX dan firewall terbukti efektif dalam mengurangi dampak serangan. Penelitian menekankan pentingnya strategi mitigasi yang tepat untuk mempertahankan kinerja server.

Peneliti	Molavi Arman
Judul	Metode Pertahanan Web Server Terhadap Distributed Slow HTTP DoS Attack
Tahun	2020
Keterangan	Menyatakan bahwa ModSecurity dan mod_antiloris terbukti efektif dalam melindungi web server dari serangan Slow HTTP DoS. ModSecurity memblokir serangan melalui aturan firewall, sedangkan mod_antiloris mencegah koneksi lambat, sehingga secara signifikan meningkatkan ketahanan server terhadap ancaman tersebut.
Peneliti	Upasana Sarmah, Parthajit Borah, D. K. Bhattacharyya
Judul	Streamlining HTTP Flooding Attack Detection Through Incremental Feature Selection
Tahun	2025
Keterangan	Menyatakan INFS-MICC efektif mendeteksi serangan HTTP Flooding dengan memilih fitur relevan dan non-redundan menggunakan mutual information dan korelasi. Metode ini meningkatkan akurasi klasifikasi secara real-time, mengurangi dampak serangan DDoS pada server web.
Peneliti	Muhammad Naufal Ammar Azi, Bongga Arifwidodo, Eka Wahyudi
Judul	Analisis Performansi Web Server Saat Menangani Permintaan Client Menggunakan Metode Reserve Proxy Caching Nginx dan Varnish
Tahun	2023
Keterangan	Menyatakan Nginx lebih unggul dari Varnish sebagai reverse proxy caching pada web server Apache, dengan RPS 39,6% lebih tinggi, waktu pengujian 28,3% lebih cepat, waktu per request 28,4% lebih rendah, dan transfer rate 38,4% lebih baik. Direkomendasikan untuk lalu lintas tinggi guna cegah downtime dan tingkatkan performa.
Peneliti	Wisnu Ramadhani, Muhammad Arif Fadhly Ridha
Judul	Perbandingan Kinerja Ingress Controller Pada Kubernetes Menggunakan Traefik Dan Nginx
Tahun	2022
Keterangan	Menyatakan Traefik lebih responsif dan cepat dibandingkan Nginx dalam menangani request pada klaster Kubernetes, tetapi

	memerlukan memori lebih besar. Nginx lebih hemat memori, cocok untuk perangkat dengan spesifikasi rendah. Traefik direkomendasikan untuk performa tinggi, sedangkan Nginx untuk efisiensi sumber daya.
Peneliti	Ade Ismail, Ahmadi Yuli Ananta, Sofyan Noor Arief, Elok Nur Hamdana
Judul	Performance Testing Sistem Ujian Online Menggunakan JMeter pada Lingkungan Virtual
Tahun	2023
Keterangan	Menyatakan pengujian performa (<i>performance testing</i>) menggunakan Apache JMeter dengan skenario beban pengguna yang disesuaikan dalam model virtualisasi server. Hasilnya menunjukkan peningkatan alokasi resource hardware dasar kurang berdampak signifikan pada penanganan beban kerja tanpa optimasi konfigurasi tambahan seperti tuning DBMS
Peneliti	Harry Setiawan, Depandi Enda
Judul	Pengujian Load Testing Website Jurusan Teknik Informatika Politeknik Negeri Bengkalis
Tahun	2024
Keterangan	Menyatakan pengujian <i>load testing</i> menggunakan Apache JMeter untuk mencari tahu kapasitas maksimal sistem web akademik dalam mengelola beban akses bersamaan pada tiga jenis infrastruktur server (<i>localhost</i> , <i>shared hosting</i> , dan <i>VPS hosting</i>). Hasil perbaikan lewat teknik meringkas baris kode dan kompresi gambar terbukti mendongkrak performa server secara signifikan.
Peneliti	Marek Šimon, Ladislav Huraj, Nicolas Búčik
Judul	A Comparative Analysis of High Availability for Linux Container Infrastructures
Tahun	2023
Keterangan	Menyatakan analisis komparatif performa infrastruktur <i>high availability</i> berbasis kontainer Linux pada <i>platform</i> Docker Swarm, Kubernetes, dan Proxmox. Pengukuran berfokus pada durasi <i>service recovery time</i> , volume kegagalan <i>request</i> , dan <i>transfer rate</i> . Docker Swarm dinilai efektif untuk klaster umum, sementara Proxmox unggul khusus untuk pemrosesan <i>transfer rate</i> yang tinggi.
Peneliti	Aulia Rahma Annisa, Mirza Ardiana, Putri Nur Rahayu, Thomas Brian, Mohammad Abu Jami'in

Judul	Performance Evaluation of Web Applications Using JMeter Load Testing for Server Capacity and Response Efficiency
Tahun	2026
Keterangan	Menyatakan pengujian efisiensi server web jurnal konferensi akademik lewat metode <i>load testing</i> menggunakan Apache JMeter dengan beban dinamis mencapai 5000 <i>virtual users</i> . Ditemukan bahwa batas stabilitas optimal server lokal berada pada kisaran 200 <i>concurrent users</i> , di mana beban melebihi 300 <i>users</i> memicu lonjakan latensi respons di atas 60 detik serta <i>error rate</i> (1,67%).
Peneliti	Budi Harijanto, Yuri Ariyanto
Judul	Desain dan Analisis Kinerja Virtualisasi Server Menggunakan Proxmox Virtual Environment
Tahun	2015
Keterangan	Menyatakan pengujian performa isolasi resource perangkat keras pada infrastruktur virtualisasi server menggunakan sistem operasi Proxmox VE berbasis KVM dan OpenVZ. Hasil pembebanan membuktikan efisiensi pengelolaan kluster, di mana lonjakan beban kerja yang tinggi (>50%) pada salah satu VM tidak menimbulkan intervensi atau degradasi performa pada VM lainnya.
Peneliti	Widya Tri Wulan Sari, Sugeng Purwanto E.S.G.S, Muhammad Arif Fadhly Ridha
Judul	Pengukuran Kinerja Kubernetes Cluster pada Nested Virtualization Berbasis KVM
Tahun	2023
Keterangan	Menyatakan analisis alokasi sumber daya pada implementasi kluster kontainer Kubernetes yang dibangun di atas lingkungan <i>nested virtualization</i> berbasis KVM. Hasil pengujian menunjukkan arsitektur berlapis ini mampu membatasi utilisasi CPU saat kondisi puncak (<i>busy</i>) hingga lebih rendah (36%) daripada kluster biasa (65%), meskipun menghasilkan beban <i>overhead</i> RAM mencapai 42%.

BAB 3. METODE PENELITIAN

3.1 Waktu dan Tempat

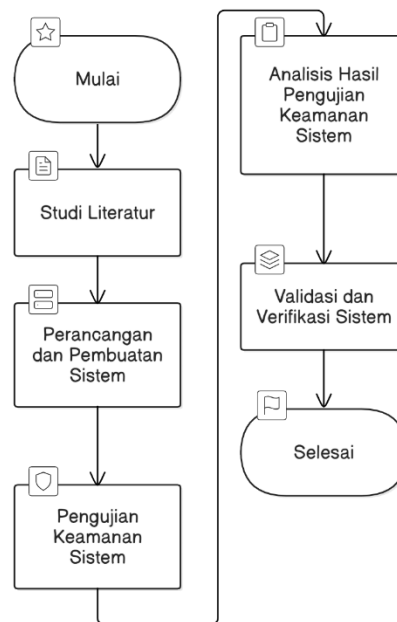
Penelitian ini dilaksanakan selama 8 (delapan) bulan, dari bulan November 2025 hingga Juni 2026, di Jurusan Teknologi Informasi, Politeknik Negeri Jember, yang berlokasi di Jl. Mastrip, Krajan Timur, Sumbersari, Kecamatan Sumbersari, Kabupaten Jember, Jawa Timur. Periode penelitian ini memastikan bahwa waktu cukup untuk melakukan konfigurasi sistem, simulasi serangan, pengumpulan data, dan analisis dari hasil yang sudah diujikan.

Tabel 3. 1 Waktu Penelitian

No	Kegiatan	Tahun							
		2025				2026			
		November	Desember	Januari	Februari	Maret	April	Mei	Juni
1	Studi Literature								
2	Perancangan dan Pembuatan Sistem								
3	Pengujian Keamanan Sistem								
4	Analisis Hasil Pengujian								
5	Validasi dan Verifikasi Sistem								

3.2 Tahapan Penelitian

Tahapan penelitian mencakup langkah-langkah yang dirancang agar penelitian ini dapat berjalan secara terstruktur, serta bertujuan untuk meminimalkan risiko kegagalan. Tahapan penelitian yang akan diterapkan mengikuti urutan proses yang digambarkan dalam diagram alir pada gambar berikut:



Gambar 3. 1 Diagram Alir Tahapan Penelitian

Tahapan penelitian ini dirancang secara sistematis untuk menjamin ketercapaian tujuan mitigasi serangan *HTTP(S) Flood*. Alur kerja penelitian dibagi menjadi empat fase utama sebagai berikut:

3.2.1 Fase Inisiasi dan Studi Literatur

Fase ini mencakup identifikasi masalah dan pengumpulan referensi terkait arsitektur Kubernetes, ModSecurity, dan mekanisme *caching*. Output dari tahap ini adalah parameter serangan yang akan disimulasikan.

3.2.2 Fase Perancangan dan Pembuatan Sistem

Fase ini merupakan inti dari pembangunan infrastruktur. Fokus utama adalah mengkonfigurasi *klaster* Kubernetes pada Proxmox dan mengintegrasikan modul keamanan pada Ingress Nginx. Detail arsitektur dan alur sistem dijelaskan lebih mendalam pada Subbab 3.3 Perancangan dan Pembuatan Sistem.

3.2.3 Fase Pengujian dan Pengumpulan Data

Pada fase ini, dilakukan simulasi serangan *HTTP(S) Flood* menggunakan Apache JMeter dengan berbagai tingkat intensitas. Prosedur pengujian, skenario beban, dan metrik yang diukur dijelaskan secara rinci pada Subbab 3.4 Tingkat Keamanan Sistem.

3.2.4 Fase Analisis, Validasi, dan Penarikan Kesimpulan

Data yang diperoleh kemudian diolah untuk melihat efektivitas mitigasi. Proses validasi dilakukan dengan membandingkan hasil *throughput* dan *error rate* terhadap target keberhasilan penelitian.

3.3 Perancangan Sistem

Perancangan sistem dalam penelitian ini difokuskan pada pembangunan infrastruktur kluster Kubernetes di atas lingkungan virtual. Arsitektur ini dirancang untuk mendukung implementasi *multi-node*.

3.3.1 Perancangan Lingkungan Virtual (*Before Environment*)

Sebelum membangun kluster orkestrasi yang kompleks, rancangan sistem dimulai pada pemetaan lingkungan dasar sistem atau disebut sebagai *Before Environment*. Pada tahap ini, seluruh *node* Kubernetes dirancang untuk dijalankan di atas Proxmox Virtual Environment sebagai lapisan *hypervisor* utama di level *bare-metal*.

Fase *Before Environment* ini berfokus pada penyediaan *resource pool* mentah dari perangkat keras yang bertindak sebagai pondasi virtualisasi. Pemilihan Proxmox sebagai *Hypervisor Type-I* pada fase lingkungan awal ini didasarkan pada beberapa pertimbangan teknis:

1. Efisiensi Resource

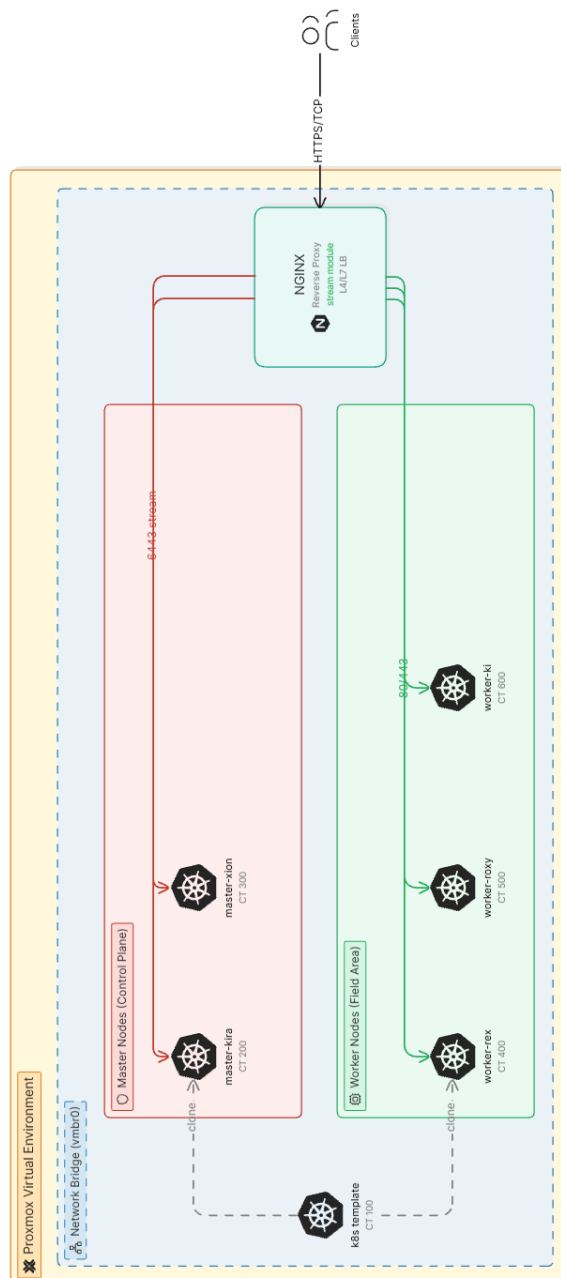
Proxmox memungkinkan pembagian sumber daya fisik (CPU, RAM, dan Storage) secara granular ke dalam beberapa *Virtual Machine* (VM) atau *Linux Containers* (LXC) yang berfungsi sebagai *Master Node* dan *Worker Node*. Hal ini memastikan bahwa meskipun dijalankan pada perangkat keras yang terbatas, kluster *multi-node* tetap mendapatkan isolasi sumber daya yang cukup untuk menjalankan layanan Kubernetes secara stabil.

2. Penerapan Multi-Node

Proxmox dapat memfasilitasi pembuatan infrastruktur *multi-node* yang terisolasi, memungkinkan pengujian mekanisme *High Availability* (HA) dan distribusi beban kerja secara merata.

3. Manajemen dan Monitoring

Proxmox menyediakan *dashboard* manajemen berbasis web yang memudahkan konfigurasi jaringan virtual, manajemen *resource* secara *real-time*, serta pengambilan data *baseline* infrastruktur sebelum pengujian dilakukan.



Gambar 3. 2 Arsitektur Proxmox sebagai Hypervisor (*Before Environment*)

3.3.2 Perancangan Topologi Klaster (*Before Topology*)

Setelah rancangan lingkungan virtual dasar (*Before Environment*) terpenuhi, tahapan dilanjutkan dengan merancang konsep alur jaringan klaster yang bersih atau disebut sebagai *Before Topology*. Kondisi *Before Topology* ini merepresentasikan arsitektur pemetaan *node* virtual di dalam *hypervisor* yang dikonfigurasi menggunakan model *multi-node* bersarang (*nested*), namun dalam keadaan murni (belum ada target aplikasi maupun aturan mitigasi keamanan).

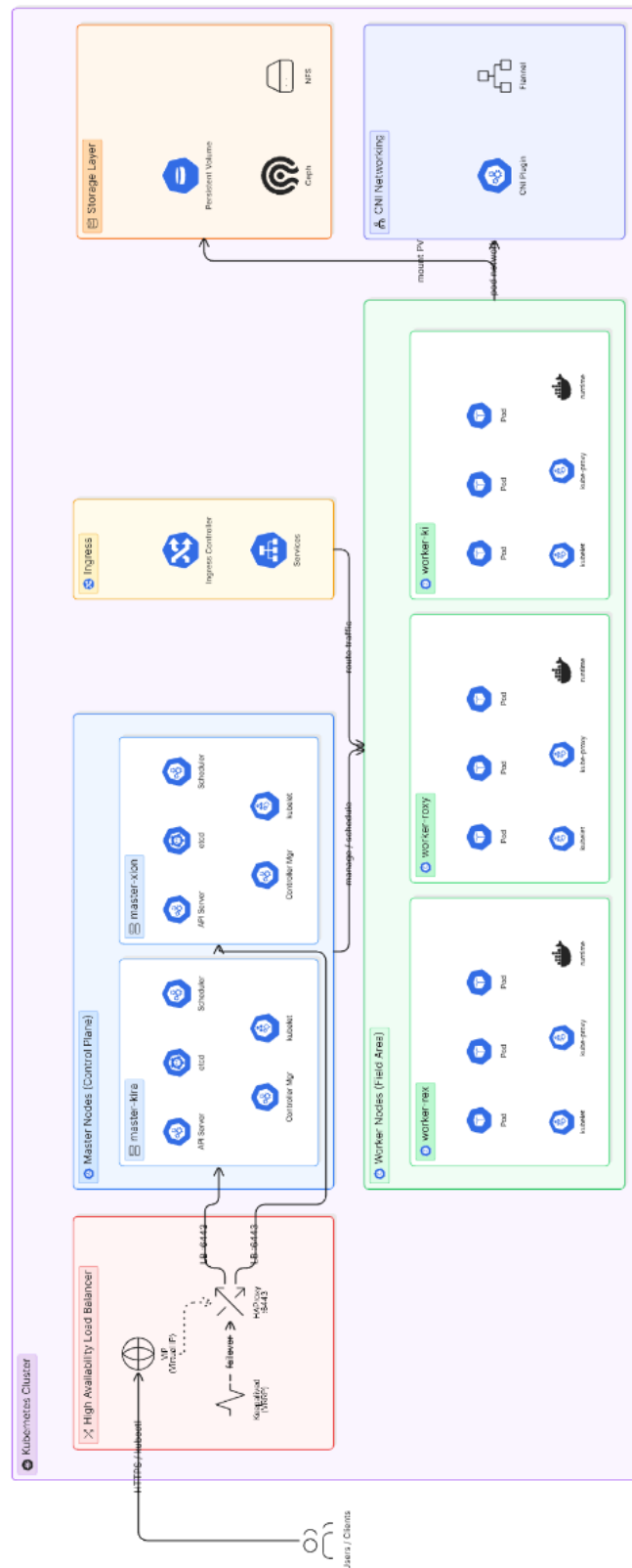
Perancangan *Before Topology* ini bertujuan untuk memisahkan fungsi kontrol dan beban kerja aplikasi guna meminimalkan dampak serangan pada *Master Node* serta memastikan data penggunaan *resource* pada *Worker Node* bersifat akurat:

1. Master Node (Control Plane)

Bertugas mengelola status klaster, penjadwalan *pod*, dan manajemen API server. Dalam lingkungan Proxmox, *Master Node* diberikan alokasi *resource* yang stabil untuk menjaga konsistensi orkestrasi.

2. Worker Nodes (Field Area)

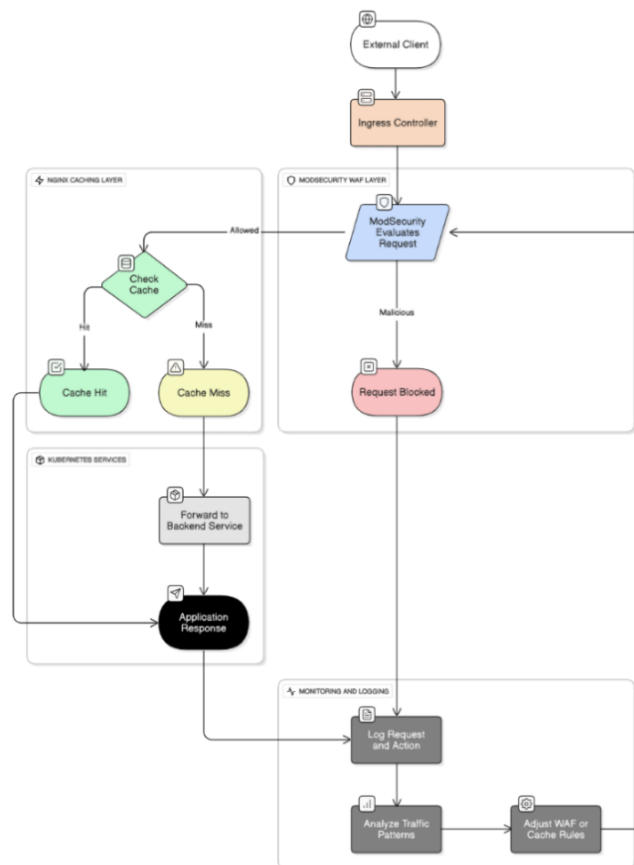
Merupakan unit komputasi tempat aplikasi dan Ingress Controller dijalankan. Penggunaan *multi-node* pada *Worker* memungkinkan pengujian distribusi trafik dan observasi beban CPU/RAM secara spesifik pada *node* yang terdampak serangan.



Gambar 3. 3 Kluster Kubernetes Multi-Node (*Before Topology*)

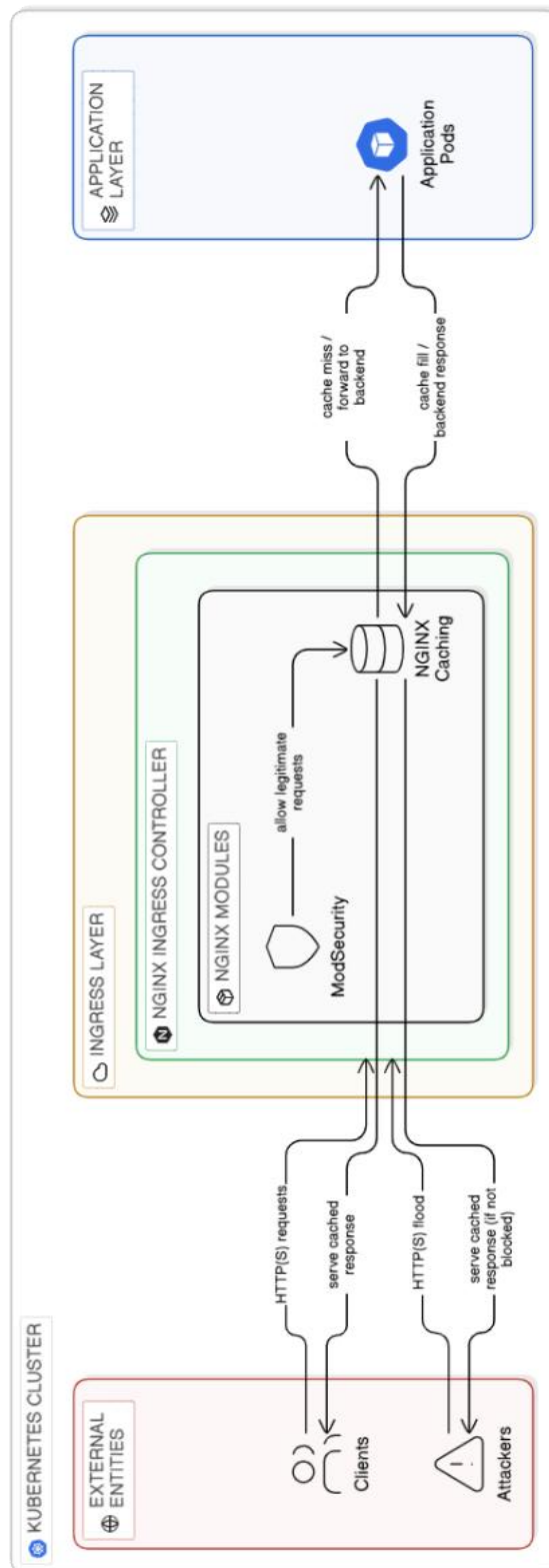
3.3.3 Integrasi ModSecurity dan NGINX Caching pada Ingress

Perancangan keamanan pada lapisan aplikasi dilakukan dengan mengintegrasikan ModSecurity sebagai *Web Application Firewall* (WAF) dan NGINX sebagai penyedia layanan *caching* di dalam Ingress Controller.



Gambar 3. 4 Flowchart Sistem

Flowchart sistem pada Gambar 3.4 menggambarkan alur kerja arsitektur berbasis Kubernetes untuk mendeteksi dan memitigasi serangan *HTTP(S) Flood*. Diagram ini menjelaskan interaksi antara komponen utama, yaitu ModSecurity sebagai *Web Application Firewall* (WAF) dan NGINX Caching untuk optimasi performa.



Gambar 3. 5 Arsitektur Alur Data Sistem Mitigasi

Arsitektur alur data dalam penelitian ini digambarkan melalui diagram pada Gambar 3.5, yang menunjukkan mekanisme kerja berbasis Kubernetes dalam mengintegrasikan ModSecurity dan NGINX Caching guna mitigasi serangan HTTP(S) Flood. Penjelasan komponen dan alur proses logisnya adalah sebagai berikut:

a. External Entities

Mencakup *Clients* (pengguna sah) dan *Attackers* (penyerang). *Clients* mengirimkan permintaan HTTP(S) normal, sementara *Attackers* melancarkan serangan *HTTP(S) Flood* untuk mengganggu layanan. Seluruh trafik masuk melalui lapisan eksternal menuju kluster.

b. Ingress Layer

Lapisan ini dikelola oleh NGINX Ingress Controller yang berfungsi sebagai entry point. Komponen ini bertanggung jawab dalam routing dan load balancing terhadap permintaan yang masuk.

c. NGINX Modules

Terdiri dari dua komponen utama yaitu:

1. NGINX Caching

Berfungsi untuk menyimpan respons aplikasi, baik statis maupun dinamis. Jika permintaan yang sama diterima kembali dan tersedia dalam cache, respons akan diberikan langsung tanpa mengakses backend. Pada kondisi cache miss, permintaan diteruskan ke backend, dan respons yang dihasilkan akan disimpan untuk penggunaan berikutnya (cache fill). Mekanisme ini secara signifikan mengurangi beban CPU pada Worker Node saat terjadi lonjakan trafik.

2. ModSecurity

Berfungsi sebagai filter awal yang memeriksa setiap permintaan HTTP(S) berdasarkan rule set (Core Rule Set/CRS). Penempatan pada level Ingress memastikan serangan dapat difilter sebelum mencapai aplikasi. ModSecurity menganalisis lalu lintas, memblokir permintaan mencurigakan, dan hanya meneruskan trafik yang valid.

d. Application Layer

Lapisan ini berisi *Application Pods*, yaitu kontainer aplikasi yang menjalankan layanan utama. Jika terjadi *cache miss* (respons tidak ada di *cache*), permintaan diteruskan ke *Application Pods* untuk diproses, dan responsnya digunakan untuk mengisi *cache* (*cache fill*) sekaligus dikembalikan ke pengguna.

3.3.4 Alur Proses Kerja Sistem Mitigasi

Alur kerja sistem secara keseluruhan adalah sebagai berikut:

- a. Permintaan HTTP(S) dari *Clients* dan *Attackers* diterima oleh *NGINX Ingress Controller*.
- b. ModSecurity memfilter permintaan, memblokir trafik yang terindikasi sebagai serangan *HTTP(S) Flood*.
- c. Permintaan yang lolos akan diperiksa oleh mekanisme *caching*. Jika *respons* tersedia, maka langsung dikembalikan dari *cache*. jika tidak, permintaan diteruskan ke *Application Pods* untuk diproses yang kemudian mengisi *cache* dengan *respons* baru.
- d. *Respons* akhir dikirimkan kepada *Clients*, sedangkan trafik dari *Attackers* diblokir atau ditangani melalui mekanisme mitigasi yang tersedia.

Arsitektur ini memadukan keamanan dan efisiensi dalam satu alur yang terintegrasi. ModSecurity berperan sebagai gerbang seleksi, sementara *NGINX Caching* bertindak sebagai penyangga beban seperti bendungan yang menahan arus deras, agar sistem tetap tenang meski tekanan meningkat.

3.4 Pengujian Inkremental

Pengujian dilakukan menggunakan pendekatan beban bertahap (*incremental load testing*) untuk mengidentifikasi karakteristik performa sistem mulai dari kondisi minimal (*idle*), beban normal (*baseline*), hingga kondisi ekstrem. Pendekatan ini memungkinkan penentuan *baseline* performa sistem, identifikasi titik awal degradasi, serta batas maksimum kemampuan sistem dalam menangani beban trafik yang datang.

KLASIFIKASI METODE PENGUJIAN



Gambar 3. 6 Bagan Klasifikasi Metode Pengujian

3.4.1 Klasifikasi Metode Pengujian

Metode pengujian dibagi menjadi empat kategori utama untuk mendapatkan gambaran performa sistem secara menyeluruh:

1. *Idle Test*

Pengujian dilakukan dengan beban minimal (1–5 pengguna) untuk mengamati performa dasar sistem dan penggunaan *resource* dalam kondisi hampir tanpa beban. Data ini digunakan sebagai kontrol awal sebelum pengujian lanjutan.

2. *Baseline Test*

Pengujian dengan beban rendah (10–100 pengguna) untuk menentukan kondisi stabil sistem dalam melayani permintaan sah. Hasil pengujian ini digunakan sebagai acuan pembandingan terhadap skenario selanjutnya.

3. *Incremental Load Test*

Pengujian dengan peningkatan beban secara bertahap (misalnya 100, 200, 300, hingga 500 pengguna) untuk memetakan hubungan antara jumlah pengguna dan performa sistem serta mengidentifikasi titik awal degradasi.

4. *Stress Test*

Pengujian dengan beban yang melampaui kapasitas normal sistem (di atas 500 pengguna) untuk mengidentifikasi titik jenuh (*saturation point*), perilaku sistem saat *overload*, serta ketahanan mekanisme mitigasi yang diterapkan.

3.4.2 Penentuan Skenario Pengujian

Skenario pengujian dirancang untuk mensimulasikan kondisi nyata serangan HTTP(S) Flood, termasuk serangan *Distributed Denial of Service* (DDoS) dengan variasi intensitas. Skenario pengujian dilakukan secara berurutan untuk memastikan stabilitas data yang diperoleh. Detail parameter untuk setiap skenario yang diadaptasi dari metodologi pengujian Setiawan & Enda (2024) serta Annisa dkk. (2026) disajikan pada Tabel 3.2.

Tabel 3. 2 Skenario Pengujian Inkremental

Skenario	Metode	Virtual Users (Threads)	Ramp-up (s)	Durasi (s)
I	<i>Idle</i>	1 – 5	30	600
II	<i>Baseline</i>	10 – 100	60	600+
III	<i>Incremental</i>	100 – 500	120	600 – 900
IV	<i>Stress Test</i>	600 – 1.000	120	600 – 900

Sumber: Disesuaikan dari Setiawan & Enda (2024) dan Annisa dkk. (2026)

3.4.3 Prosedur Pengujian

Setiap skenario dalam Tabel 3.2 dijalankan dengan prosedur tetap untuk menjaga konsistensi hasil:

1. Inisialisasi untuk memastikan klaster Kubernetes dalam status sehat melalui perintah `kubectl get nodes`.
2. Eksekusi dalam menjalankan *test plan* pada Apache JMeter sesuai dengan jumlah *thread* yang ditentukan.
3. Logging untuk mencatat penggunaan CPU dan memori pada Node Worker melalui *dashboard* Proxmox secara *real-time*.
4. Cooling Down untuk memberikan jeda waktu 300 detik lebih untuk menghindari carry-over effect antar skenario, setelah setiap skenario selesai

agar penggunaan sumber daya kembali ke kondisi *baseline* sebelum memulai tahap berikutnya.

3.4.4 Metrik Pengukuran

Metrik yang digunakan meliputi:

a. Tingkat Pemblokiran Serangan

Persentase permintaan berbahaya yang berhasil diblokir oleh ModSecurity.

$$\text{Attack Block Rate} = \left(\frac{\sum \text{Requests Diblokir}}{\sum \text{Total Serangan}} \right) \times 100\%$$

b. Latency (Response Time)

Waktu respons rata-rata untuk permintaan sah, diukur dalam milidetik.

$$\text{Response Time} = \frac{\sum (\text{Waktu Selesai} - \text{Waktu Mulai})}{\sum \text{Total Request}}$$

c. Throughput

Jumlah permintaan sah yang diproses per detik oleh sistem.

$$\text{Throughput} = \frac{\sum \text{Requests Berhasil}}{\text{Total Durasi}}$$

d. Error Rate

Tingkat kegagalan layanan dalam memproses permintaan.

$$\text{Error Rate} = \left(\frac{\sum \text{Requests Gagal}}{\sum \text{Total Requests}} \right) \times 100\%$$

e. Cache Hit Ratio

Persentase permintaan yang berhasil dilayani langsung dari *cache* NGINX.

$$\text{Cache Hit Ratio} = \left(\frac{\sum \text{Cache Hits}}{\sum \text{Total Requests}} \right) \times 100\%$$

3.4.5 Hasil yang Diharapkan

Berdasarkan tinjauan literatur, target keberhasilan penelitian ini adalah:

1. Sistem mampu memitigasi serangan hingga beban 1.000 user dengan *error rate* pada sisi pengguna sah di bawah 5%.
2. Penerapan *NGINX Caching* diharapkan mampu menjaga penggunaan CPU pada *Worker Node* tetap stabil (tidak menyentuh 100%) saat terjadi lonjakan trafik.
3. ModSecurity secara akurat membedakan antara trafik sah dan serangan berdasarkan parameter *payload* GET/POST.

Pengujian ini akan memberikan dasar untuk menganalisis efektivitas solusi yang diusulkan dalam mitigasi serangan *HTTP(S) Flood*.

3.5 Analisis Hasil Pengujian

Analisis hasil pengujian keamanan sistem dilakukan untuk mengevaluasi efektivitas arsitektur pertahanan ganda (*dual-layer defense*) menggunakan ModSecurity dan *Nginx Caching* dalam memitigasi serangan *HTTP(S) Flood*. Data yang dianalisis mencakup lapisan aplikasi, infrastruktur kontainer, hingga *hypervisor* fisik.

3.5.1 Pemrosesan Data

Metrik evaluasi diekstrak dari perangkat Apache JMeter untuk mengukur lapisan aplikasi, perintah administratif kubectl untuk mengamati aktivitas kluster Kubernetes, dan metrik Proxmox untuk mengevaluasi saturasi pada lapisan kernel *host*.

Seluruh data mentah yang diperoleh dari ketiga instrumen tersebut kemudian diproses, dibersihkan, dan ditabulasi secara menyilang (*cross-tabulation*). Tabulasi data dilakukan untuk membandingkan parameter performa utama (seperti *Average Response Time*, *Throughput*, *Error Rate*, *WAF Blocked Rate*, serta Utilisasi CPU) terhadap kelima kondisi pengujian utama yang dijalankan, yaitu skenario *Baseline*, *Base Cache*, *Payload ModSecurity*, *Combination*, dan *Stress Test*. Hasil tabulasi komparatif inilah yang nantinya akan dianalisis untuk mengukur tingkat ketahanan layanan (*availability*) dan stabilitas perangkat keras fisik secara objektif.

3.5.2 Perbandingan Skenario

Hasil dari pengujian komparatif dianalisis untuk mengidentifikasi pola performa arsitektur keamanan berdasarkan tingkat intensitas beban:

1. Pengujian *Baseline* (10 – 100 *Users*)

Menganalisis titik jenuh (*saturation point*) sistem aplikasi (WordPress, Ghost, dan Drupal) tanpa mitigasi tambahan.

2. Pengujian *Incremental Load* (500 *Users*)

Membandingkan dampak performa antara mitigasi tunggal (*Nginx Caching* mandiri atau ModSecurity mandiri) melawan mitigasi ganda (Kombinasi).

3. Pengujian *Stress Testing* (600 – 1.000 *Users*)

Mengevaluasi batas atas (*threshold*) dari efektivitas ModSecurity dalam menyaring serangan serta kontribusi *Nginx Caching* dalam meredam degradasi *backend* pada kondisi saturasi perangkat keras ekstrem.

3.5.3 Interpretasi

Tahap interpretasi menyoroti sejauh mana rancangan infrastruktur mampu memenuhi target sasaran mitigasi. Evaluasi ini difokuskan pada tiga pencapaian utama, yaitu: tercapainya tingkat pemblokiran serangan (*WAF Blocked Rate*) secara stabil di atas 90,00%, keberhasilan arsitektur dalam mengisolasi *Application Pods backend* dari lonjakan komputasi CPU dan memori yang berlebih, serta keandalan kluster Kubernetes beserta *kernel host* Proxmox dalam menangani anomali *Context Switches* akibat tingginya intensitas inspeksi aturan keamanan.

3.5.4 Kesimpulan Awal

Berdasarkan sintesis data pengujian, ditarik kesimpulan awal mengenai kekuatan dan kelemahan spesifik dari setiap teknologi *Content Management System* (CMS) yang digunakan, serta perumusan arsitektur keamanan paling optimal yang akan menjadi landasan untuk tahap validasi dan verifikasi sistem.

3.6 Validasi dan Verifikasi Sistem

Tahapan validasi dan verifikasi dilakukan untuk memberikan justifikasi ilmiah bahwa arsitektur yang diimplementasikan pada kluster Kubernetes dapat beroperasi secara konsisten, sesuai dengan tujuan rancangan penelitian, dan dapat diandalkan dalam mereplikasi kondisi ancaman siber yang aktual.

3.6.1 Validasi Sistem

Validasi bertujuan untuk membuktikan bahwa parameter teknis yang diuji benar-benar merepresentasikan keberhasilan mekanisme mitigasi terhadap serangan *HTTP(S) Flood*. Proses ini mencakup:

1. Memastikan bahwa kegagalan layanan yang tercatat (*Error Rate* tinggi pada pengujian) bukanlah representasi dari *server crash*, melainkan validasi dari keberhasilan fungsi ModSecurity dalam mengembalikan kode HTTP 403 Forbidden sebagai wujud pemblokiran intrusi.
2. Memverifikasi parameter *Cache Hit Ratio* dan *Cache Bypass* saat sistem disimulasikan menggunakan parameter *URL query string* acak guna menguji ketahanan lapisan Ingress Controller.
3. Mencocokkan skenario injeksi *payload* berbahaya pada metode *HTTP POST* dengan *Core Rule Set* (CRS) standar dari OWASP guna memastikan bahwa sistem menanggulangi model ancaman peretasan yang relevan dengan kondisi industri.

3.6.2 Verifikasi Sistem

Verifikasi menitikberatkan pada aspek konsistensi fungsional arsitektur dan kemampuan daur ulang rancangan pengujian (*reproducibility*). Proses ini meliputi:

1. Memastikan bahwa eksekusi simulasi menggunakan *Apache JMeter* mampu secara konsisten mempertahankan jumlah koneksi virtual (*concurrent users*) dari skala ringan hingga skala batas atas 1.000 *users* tanpa mengalami kegagalan transmisi dari sisi mesin penyerang.
2. Melakukan pengecekan silang (*cross-verification*) terhadap log *Nginx Ingress Controller*, log *Application Pods*, dan status kesehatan kluster Kubernetes untuk memastikan tidak ada kesalahan konfigurasi perutean jaringan atau kegagalan *cgroup* alokasi RAM/CPU selama serangan berlangsung.

3. Menyusun dokumentasi spesifikasi konfigurasi manifes *Deployment* Kubernetes (berkas YAML), pengaturan virtualisasi Proxmox, serta modul rancangan uji JMeter (berkas .jmx) sedemikian rupa sehingga arsitektur eksperimen ini bersifat saintifik dan dapat direplikasi secara presisi oleh peneliti lain di masa mendatang.

Hasil validasi dan verifikasi ini mengonfirmasi bahwa sistem siap digunakan sebagai solusi mitigasi serangan HTTP(S) Flood dan mendukung kesimpulan penelitian.

BAB 4. HASIL DAN PEMBAHASAN

4.1 Hasil Studi Literatur dan Spesifikasi Infrastruktur

Dalam mengimplementasikan lingkungan virtualisasi ini, diperlukan penentuan spesifikasi perangkat keras dan perangkat lunak yang memadai untuk menangani beban kerja *cluster* Kubernetes serta proses inspeksi trafik oleh ModSecurity. Infrastruktur ini dirancang untuk memastikan sistem mampu menjalankan simulasi mitigasi serangan HTTP(S) Flood secara stabil.

Sebagai landasan ilmiah, penentuan serta pemilihan seluruh komponen infrastruktur pada bab ini didasarkan pada sintesis hasil studi literatur terdahulu (*state of the art*) yang telah dipetakan.

Secara arsitektur keamanan, pemilihan Nginx Ingress Controller dan Web Application Firewall ModSecurity merujuk pada temuan Zain dkk. (2023) serta Azi dkk. (2023) yang membuktikan efektivitas ModSecurity dalam memblokir anomali trafik hingga 100%, serta efisiensi Nginx sebagai *reverse proxy caching* yang sangat optimal untuk perangkat dengan alokasi sumber daya rendah (Ramadhani & Ridha, 2022). Penelitian ini mengintegrasikan kedua modul tersebut secara simultan pada satu gerbang *NGINX Ingress Controller* untuk menganalisis efek sinergisnya.

Sementara itu, perancangan infrastruktur berlapis (*nested virtualization*) di atas *Hypervisor Type-1* Proxmox Virtual Environment didasarkan pada kajian Sari dkk. (2023) dan Šimon dkk. (2023) mengenai keunggulan Proxmox dalam isolasi sumber daya fisik secara granular serta efisiensi pengelolaan klaster saat kondisi beban puncak (Harijanto & Ariyanto, 2015).

Terakhir, penentuan Apache JMeter sebagai instrumen pengujian beban hingga 1.000 *users* mengacu pada metodologi pengujian performa yang divalidasi oleh Ismail dkk. (2023), Setiawan & Enda (2024), serta Annisa dkk. (2026) untuk memetakan titik jenuh (*saturation point*) sistem secara presisi. Penelitian tersebut dominan berfokus pada metrik performa lapisan aplikasi (seperti *response time* dan *throughput*). Sementara itu, penelitian ini melakukan pemantauan berlapis yang

menghubungkan metrik aplikasi, alokasi *resource Pod container* pada level klaster Kubernetes (kubectl top), hingga utilisasi kernel fisik *hypervisor* Proxmox untuk memetakan anomali trafik secara komprehensif.

Berbeda dengan penelitian terdahulu yang umumnya menggunakan satu jenis platform target, eksperimen ini mengevaluasi efektivitas arsitektur mitigasi terhadap tiga karakteristik *runtime* aplikasi *Content Management System* (CMS) yang berbeda, yaitu WordPress (berbasis *process-driven blocking*), Ghost (berbasis *event-driven non-blocking*), dan Drupal (PHP dengan optimasi *caching* internal yang kompleks).

Berdasarkan justifikasi teoretis tersebut, berikut adalah rincian spesifikasi perangkat lunak (*software*) dan perangkat keras (*hardware*) yang digunakan dalam pelaksanaan eksperimen:

4.1.1 *Software*

1. Proxmox Virtual Environment

Platform manajemen virtualisasi berbasis *open-source* (Hypervisor Type-1) yang digunakan untuk membuat dan mengelola mesin virtual (VM) dan *Linux Containers* (LXC). Proxmox berfungsi sebagai *host* utama yang menyediakan isolasi sumber daya bagi setiap *node* di dalam klaster Kubernetes.

2. Kubernetes

Sebagai platform orkestrasi *container* utama yang mengelola penyebaran (*deployment*), penskalaan, dan pengelolaan otomatis aplikasi. Kubernetes menjamin *high availability* dan pemulihan otomatis (*self-healing*) pada *node* eksperimen.

3. ModSecurity

Web Application Firewall (WAF) yang beroperasi pada layer aplikasi untuk memantau dan menyaring permintaan HTTP. ModSecurity berperan dalam mendeteksi dan mencegah serangan melalui konfigurasi *ruleset* (OWASP CRS).

4. Nginx Ingress Controller

Komponen yang mengatur lalu lintas masuk (*ingress traffic*) ke dalam *cluster*. Berfungsi meneruskan permintaan pengguna ke layanan yang tepat, serta mendukung *SSL termination* dan *caching*.

5. Apache JMeter

Alat *open-source* yang digunakan untuk melakukan uji beban dan performa. JMeter mensimulasikan trafik dari banyak pengguna secara bersamaan untuk mengukur ketahanan sistem terhadap serangan HTTP(S) Flood.

4.1.2 Hardware

Infrastruktur eksperimen dalam penelitian ini dibangun menggunakan kombinasi perangkat keras lokal sebagai stasiun pengendali (*client controller*) dan lingkungan infrastruktur komputasi awan berbasis *Infrastructure as a Service* (IaaS) dengan rincian spesifikasi sebagai berikut:

1. Laptop (*Client Workstation*)

- a. Sistem Operasi: Windows 11 Home Single Language.
- b. Prosesor: 13th Gen Intel(R) Core(TM) i5-13420H (12 CPUs), N2.1GHz.
- c. RAM: 24 GB.
- d. Fungsi: Digunakan untuk konfigurasi sistem melalui antarmuka baris perintah (CLI), pengelolaan kluster Kubernetes secara jarak jauh, eksekusi skrip otomatisasi Apache JMeter, dan analisis data eksperimen.

2. Server/VPS/IaaS (*Proxmox Host*)

- a. Penyedia Layanan: Amazon Web Services (AWS) – Instans Amazon EC2.
- b. Sistem Operasi: Debian GNU/Linux 13 (trixie)/Proxmox VE 9.1.7.
- c. Kernel: Linux 6.12.63+deb13-amd64/Linux 6.17.13-2-pve
- d. Prosesor: Intel(R) Xeon(R) Platinum 8488C (2) @ 2.40 GHz.
- e. RAM: 8 GB.
- f. Fungsi: Menjalankan sistem *Bare-Metal Hypervisor Type-1* untuk menopang dan mengisolasi siklus kerja *Multi-Node container cluster* Kubernetes (Master Node dan Worker Node) di dalam lingkungan virtualisasi bersarang (*nested virtualization*).

```

root@kiraryuuki:/home/kira# fastfetch
.://:~
`hMMMMMMd/`      `dMMMMMMh`
`sMMMMMMMd:      `mMMMMMMMs`
`-/+++/:'`yMMMMMMh-`-hMMMMMMMy`-/+++/-'
`:oooooooo/-`-hMMMMMMMyMMMMMMh-`/oooooooo:
`/oooooooo:~mMMMMMMMMMMhM:~:ooooooooo/
`./oooooooo+~+NMMMMMMMN+~+ooooooooo/
`+oooooooo+~`oNMMMMMMNo~+oooooooo+
`+oooooooo/.`sMMs`./oooooooo+
:ooooooooo/..`/ooooooooo:
:ooooooooo/..`/ooooooooo:
`+oooooooo/.`sMMs`./oooooooo+
`+oooooooo+~+NMMMMMMMN+~+oooooooo+
`/oooooooo/-`-hMMMMMMMMMN+~:ooooooooo/
`:oooooooo:~mMMMMMMMMMMhM:~:ooooooooo/
`-/+++/:'`yMMMMMMh-`-hMMMMMMMy`-/+++/-'
`hMMMMMMMd/`      `dMMMMMMh`
.://:~

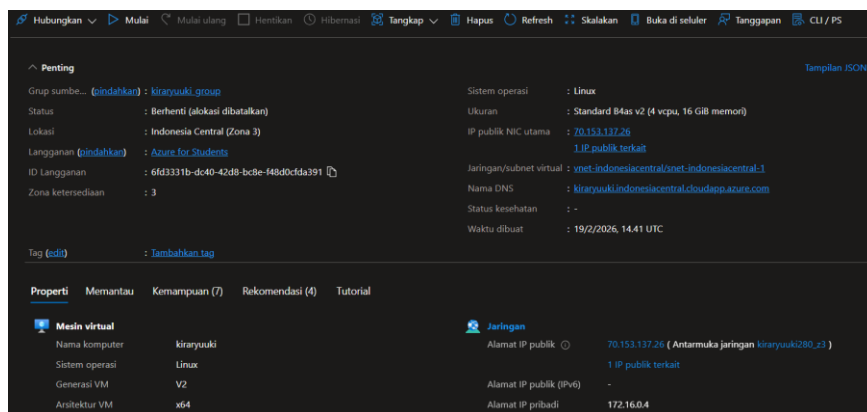
root@kiraryuuki
-----
OS: Proxmox VE 9.1.7 x86_64
Host: m7i-flex.large
Kernel: Linux 6.17.13-2-pve
Uptime: 7 mins
Packages: 957 (dpkg)
Shell: bash 5.2.37
Terminal: /dev/pts/1
CPU: Intel(R) Xeon(R) Platinum 8488C (2) @ 2.40 GHz
GPU: Amazon.com, Inc. Device 1111 (VGA compatible)
Memory: 1.67 GiB / 7.68 GiB (23%)
Swap: Disabled
Disk (/): 25.94 GiB / 98.19 GiB (26%) - ext4
Local IP (enp3s0): 172.31.8.47/20
Locale: C.UTF-8

```

Gambar 4. 1 Spesifikasi Sistem

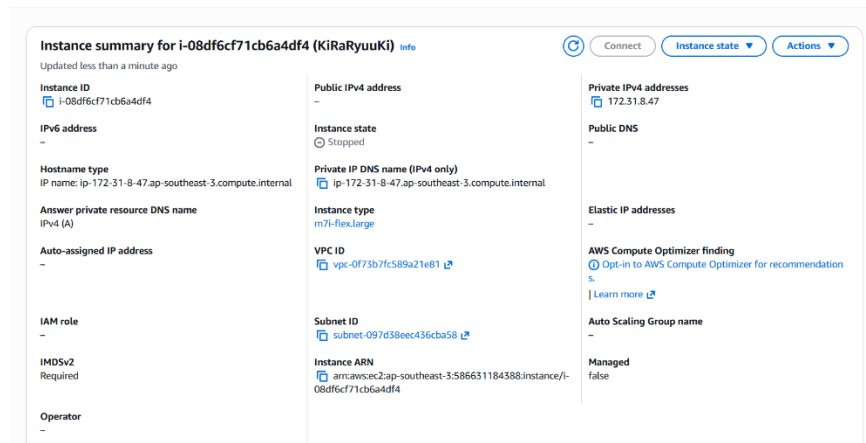
3. Riwayat Analisis dan Pemilihan Infrastruktur

Sebelum menetapkan Amazon Web Services (AWS) sebagai platform komputasi utama, telah dilakukan tahapan uji coba pra-eksperimen menggunakan penyedia layanan Microsoft Azure melalui program *Azure for Students* serta layanan VPS dari Hostinger.



Gambar 4. 2 Antarmuka Manajemen pada Microsoft Azure

Namun, penggunaan kedua platform tersebut tidak dilanjutkan untuk pemrosesan data eksperimen utama karena adanya batasan masa berlaku. Durasi akses gratis dan alokasi kredit pada program *Azure for Students* telah habis sebelum seluruh rangkaian pengujian keamanan kluster selesai dieksekusi.



Gambar 4. 3 Antarmuka Manajemen pada Amazon Web Services

Oleh karena itu, infrastruktur dialihkan menggunakan layanan AWS (Amazon Web Services) dengan memanfaatkan program kredit gratis khusus pembelajaran mahasiswa (*AWS Student Cloud Credits*) seperti yang terdokumentasi pada Gambar 4.3. Platform AWS EC2 ini dipilih karena menyediakan durasi penggunaan serta alokasi sumber daya virtual yang memadai untuk menyelesaikan seluruh tahapan simulasi mitigasi serangan HTTP(S) Flood di atas lingkungan virtualisasi bersarang (*nested virtualization*) secara legal dan stabil. Adapun spesifikasi awal dari instans AWS EC2 yang berhasil disewa pada program pembelajaran mahasiswa ini adalah menggunakan prosesor Intel(R) Xeon(R) Platinum 8488C dengan kapasitas 2 vCPU dan RAM sebesar 8 GB, sebagaimana telah dirinci pada poin spesifikasi perangkat keras utama di atas.

4.2 Implementasi dan Konfigurasi Sistem

4.2.1 Pemasang Proxmox

Sistem dibangun menggunakan platform virtualisasi Proxmox Virtual Environment sebagai fondasi utama untuk menjalankan *cluster* Kubernetes. Pemilihan Proxmox didasarkan pada efisiensi pengelolaan *Linux Containers* (LXC) serta fleksibilitas dalam alokasi sumber daya (*resource*) dan konfigurasi jaringan virtual.

Tahap konfigurasi diawali dengan validasi alamat IP statis pada *host* untuk menjamin stabilitas komunikasi antar-node dalam jaringan. Sinkronisasi identitas

sistem dilakukan melalui modifikasi berkas `/etc/hosts` guna memastikan resolusi `hostname` berjalan optimal tanpa bergantung pada DNS eksternal.

```
GNU nano 8.4 /etc/hosts
127.0.0.1    localhost
::1         localhost ip6-localhost ip6-loopback
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters

# add IP address, and your hostname, then your /etc/hosts file could look like:
172.31.8.47  kiraryuuki.systems  kiraryuuki
```

Gambar 4. 4 File `/etc/hosts`

Untuk mendukung orkestrasi Kubernetes di dalam kontainer, fitur *nesting* diaktifkan serta dilakukan pemuatan modul kernel `br_netfilter` dan `overlay`. Hal ini dilakukan agar mekanisme *networking* berbasis *bridge* dan *container runtime* dapat beroperasi sesuai spesifikasi Kubernetes.

```
GNU nano 8.4 /etc/modules
# /etc/modules is obsolete and has been replaced by /etc/modules-load.d/.
# Please see modules-load.d(5) and modprobe.d(5) for details.
#
# Updating this file still works, but it is undocumented and unsupported.

overlay
br_netfilter
ip_vs
nf_nat
xt_conntrack
```

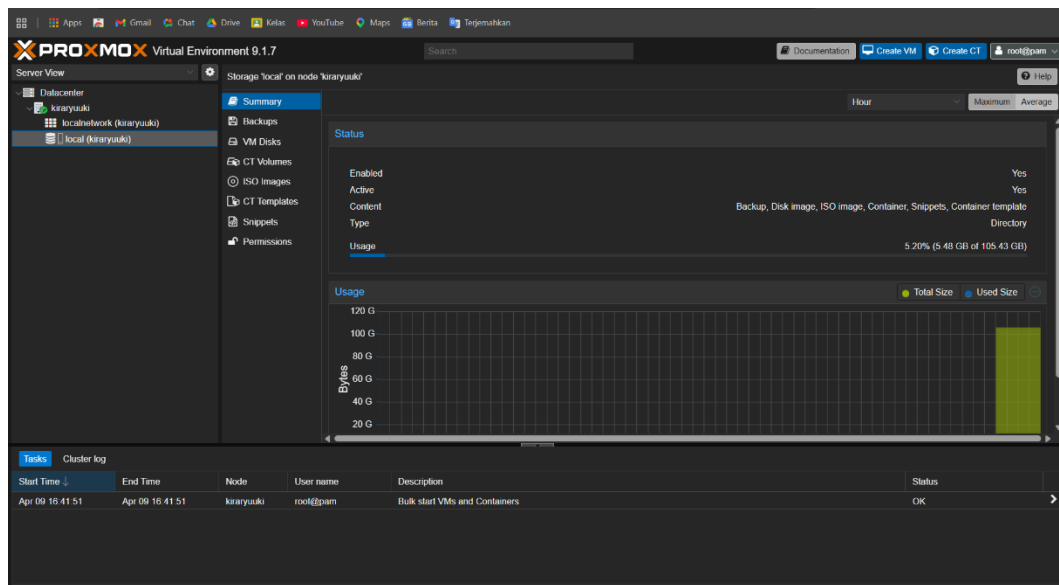
Gambar 4. 5 Konfigurasi Modules

Proses instalasi diakhiri dengan pembaruan repositori resmi Proxmox dan pemasangan layanan pendukung seperti `chrony` untuk sinkronisasi waktu dan `open-iscsi` untuk manajemen penyimpanan. Setelah sistem direstart, verifikasi versi dilakukan untuk memastikan infrastruktur siap digunakan sebagai *host* virtualisasi.

```
root@kiraryuuki:/home/kira# pveversion
pve-manager/9.1.7/16b139a017452f16 (running kernel: 6.17.13-2-pve)
root@kiraryuuki:/home/kira#
```

Gambar 4. 6 Status `pveversion`

Setelah proses instalasi melalui terminal selesai, dilakukan verifikasi akses manajemen melalui antarmuka berbasis web (GUI). Akses dilakukan dengan membuka alamat IP *host* melalui protokol HTTP(S) pada *port* 8006. Keberhasilan akses ke *dashboard* Proxmox menandakan bahwa seluruh layanan virtualisasi telah siap digunakan untuk pengelolaan *node* dan *container*.



Gambar 4. 7 Antarmuka Manajemen Proxmox

Setelah *host* virtualisasi siap, tahap selanjutnya adalah melakukan *provisioning* terhadap mesin virtual yang akan digunakan sebagai *node* dalam kluster Kubernetes.

4.2.2 Konfigurasi Antarmuka Jaringan

Konfigurasi jaringan dilakukan untuk menjembatani komunikasi antara *host*, *container*, dan jaringan eksternal. Tahap ini melibatkan pembuatan *virtual bridge* (vbr0) yang berfungsi sebagai *switch* virtual untuk menghubungkan seluruh *node* dalam kluster ke jaringan luar.

```

GNU nano 8.4
# network interface settings; autogenerated
# Please do NOT modify this file directly, unless you know what
# you're doing.
#
# If you want to manage parts of the network configuration manually,
# please utilize the 'source' or 'source-directory' directives to do
# so.
# PVE will preserve these directives, but will NOT read its network
# configuration from sourced files, so do not attempt to move any of
# the PVE managed interfaces into external files!

auto lo
iface lo inet loopback

iface enp39s0 inet manual

# create vbr0
auto vbr0
iface vbr0 inet static
    address 192.168.0.1/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0

```

Gambar 4. 8 Konfigurasi *Virtual Bridge (vbr0)*

Bridge ini dikonfigurasi menggunakan alamat IP statis, sehingga setiap container yang terhubung dapat berkomunikasi melalui gateway yang telah ditentukan. Selain itu, fitur IP forwarding diaktifkan untuk memungkinkan paket data diteruskan antar interface jaringan.

```

*filter
:INPUT ACCEPT [131385:29635419]
:FORWARD ACCEPT [15326:2020093]
:OUTPUT ACCEPT [89512:31310143]
-A INPUT -i lo -j ACCEPT
-A FORWARD -p 112 -j ACCEPT
-A FORWARD -i vbr0 -o enp39s0 -j ACCEPT
-A FORWARD -i enp39s0 -o vbr0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
COMMIT

```

Gambar 4. 9 Aktivasi *IP Forwarding* pada Konfigurasi Kernel Sistem

Untuk mendukung akses keluar jaringan, diterapkan mekanisme Network Address Translation (NAT) menggunakan iptables dengan metode *masquerading*. Mekanisme ini memungkinkan container dalam jaringan internal mengakses jaringan eksternal melalui alamat IP host.

```

*nat
:PREROUTING ACCEPT [8869:535662]
:INPUT ACCEPT [6143:367774]
:OUTPUT ACCEPT [3118:191887]
:POSTROUTING ACCEPT [9415:492697]
-A POSTROUTING -s 192.168.45.0/24 -o enp39s0 -j MASQUERADE
COMMIT

```

Gambar 4. 10 Konfigurasi *Postrouting NAT*

Selain itu, aturan port forwarding juga diterapkan untuk mengarahkan trafik tertentu menuju layanan yang berjalan di dalam container, seperti layanan web dan caching.

Selain pengaturan *routing* dasar, infrastruktur klaster ini dilengkapi dengan komponen *Load Balancing* Lapisan Transport (L4) menggunakan modul *nginx-stream*. *Reverse proxy* ini diinstal langsung pada *host* Proxmox dan berfungsi sebagai pintu gerbang (*gateway*) utama lalu lintas HTTP (Port 80) dan HTTPS (Port 443).

```
stream {
    map $ssl_preread_server_name $backend_name {
        hostnames;
        default                    local_nginx_https;

        proxmox.kiraryuuki.systems  local_nginx_https;
        wordpress.kiraryuuki.systems k8s_ingress_https;
        ghost.kiraryuuki.systems     k8s_ingress_https;
        drupal.kiraryuuki.systems     k8s_ingress_https;
    }

    upstream local_nginx_https {
        server 127.0.0.1:4433;
    }

    upstream k8s_ingress_https {
        server 192.168.45.5:30919;
    }

    server {
        listen 443;
        proxy_pass $backend_name;
        ssl_preread on;
    }
}
```

Gambar 4. 11 Konfigurasi NGINX Stream Reverse Proxy

Aturan *forwarding* pada modul *stream* ini dikonfigurasi agar secara transparan meneruskan (*pass-through*) seluruh beban trafik jaringan yang diterima oleh *host* langsung menuju IP internal layanan *Nginx Ingress Controller* di dalam klaster Kubernetes tanpa mengubah isi *header* paket data. Arsitektur jaringan berlapis ini dirancang untuk menciptakan ekosistem pengujian yang menyerupai

kondisi kluster produksi aktual di industri. Konfigurasi ini menjadi bagian penting dalam mensimulasikan arsitektur layanan yang akan diuji pada tahap berikutnya.

4.2.3 Persiapan Kontainer untuk Kubernetes

4.2.3.1 Runtime Container

Untuk menjalankan Kubernetes, diperlukan container runtime sebagai komponen utama dalam mengeksekusi workload berbasis container. Pada penelitian ini, digunakan containerd sebagai runtime karena memiliki performa ringan dan kompatibilitas tinggi dengan Kubernetes.

```
root@master-kira:/# systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset:
   Active: active (running) since Sat 2026-04-18 04:59:15 UTC; 8h ago
   Invocation: 86e69922e07b42118de242a18fdb78cd
```

Gambar 4. 12 Status Service containerd

Selain itu, komponen tambahan seperti runc dan CNI (Container Network Interface) juga diinstal untuk mendukung eksekusi container dan komunikasi jaringan antar pod.

Konfigurasi dilakukan dengan mengaktifkan sistem cgroup berbasis systemd untuk memastikan manajemen resource berjalan optimal. Pengaturan ini dilakukan melalui modifikasi berkas config.toml pada *containerd* agar selaras dengan *init system* yang digunakan oleh sistem operasi

```
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]
  BinaryName = ""
  CriuImagePath = ""
  CriuPath = ""
  CriuWorkPath = ""
  IoGid = 0
  IoUid = 0
  NoNewKeyring = false
  NoPivotRoot = false
  Root = ""
  ShimCgroup = ""
  SystemdCgroup = true
```

Gambar 4. 13 Konfigurasi cgroup

4.2.3.2 Menginstal kubeadm, kubectl, dan kubelet

Komponen inti Kubernetes yang meliputi *kubeadm*, *kubelet*, dan *kubectl* diinstal guna membangun serta mengelola kluster secara terintegrasi. Setiap komponen memiliki peran spesifik dalam arsitektur sistem yaitu *kubeadm* berfungsi sebagai instrumen inisialisasi kluster, *kubelet* bertindak sebagai agen eksekusi pada setiap *node*, dan *kubectl* berperan sebagai antarmuka kontrol administratif.

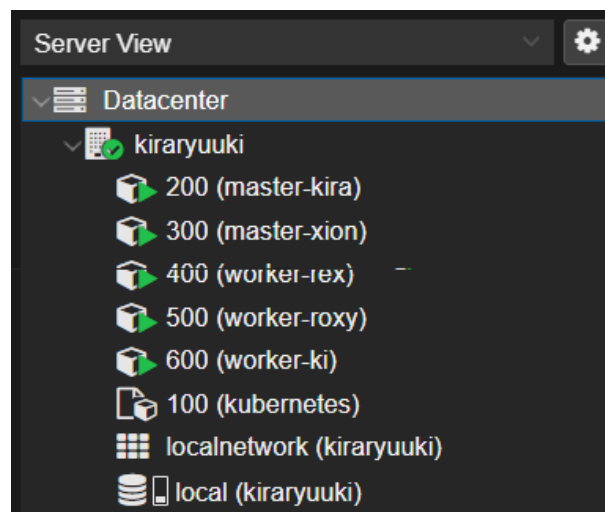
Instalasi dilakukan melalui repositori resmi Kubernetes untuk memastikan kompatibilitas versi yang digunakan.

```
root@master-kira:/# kubeadm version && kubectl version --client && kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"32", GitVersion:"v1.32.13", GitCommit:"6172d7357c6287643358a4fc7e848f24098f2a1b", GitTreeState:"clean", BuildDate:"2026-02-26T20:22:27Z", GoVersion:"go1.24.13", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.32.13
Kustomize Version: v5.5.0
Kubernetes v1.32.13
root@master-kira:/#
```

Gambar 4. 14 Verifikasi Versi Komponen Kubernetes

4.2.3.3 Pembuatan Template LXC

Untuk efisiensi, dibuat *template* kontainer LXC yang sudah terinstal dependensi dasar Kubernetes. *Template* ini kemudian di-*cloning* menjadi *master node* dan *worker node* untuk memastikan konsistensi lingkungan dan mempercepat proses *deployment*.



Gambar 4. 15 Daftar LXC pada Dashboard Proxmox

Pendekatan ini memungkinkan replikasi lingkungan yang konsisten serta mempercepat proses deployment cluster secara keseluruhan.

```
root@master-kira:/# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master-kira	Ready	control-plane	9d	v1.32.13
master-xion	Ready	control-plane	8d	v1.32.13
worker-ki	Ready	field-area	8d	v1.32.13
worker-rex	Ready	field-area	8d	v1.32.13
worker-roxy	Ready	field-area	8d	v1.32.13

```
root@master-kira:/#
```

Gambar 4. 16 Daftar *nodes* di Kubernetes

4.2.3.4 Penyeimbang beban pada Master Node

Untuk meningkatkan ketersediaan layanan, diterapkan mekanisme load balancing menggunakan HAProxy dan Keepalived. Kombinasi ini memungkinkan distribusi trafik ke node master secara merata, serta menyediakan virtual IP sebagai endpoint utama cluster.

```
2: eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
link/ether bc:24:11:ff:b8:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
inet 192.168.45.20/24 scope global eth0
    valid_lft forever preferred_lft forever
inet 192.168.45.5/24 scope global secondary proto keepalived eth0
    valid_lft forever preferred_lft forever
inet6 fe80::bc24:11ff:feff:b802/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
```

Gambar 4. 17 Keepalived *Virtual IP* Klaster

Konfigurasi ini penting dalam mensimulasikan kondisi sistem yang mendekati lingkungan produksi, di mana redundansi dan ketersediaan menjadi faktor utama.

4.2.3.5 Instalasi Network Plugin

Setelah cluster diinisialisasi, dilakukan instalasi network plugin untuk mengatur komunikasi antar pod dalam Kubernetes.


```

root@master-kira:/# kubectl get pods -A
NAMESPACE      NAME                                                    READY   STATUS    RESTARTS   AGE
ingress-nginx   ingress-nginx-controller-6994f4fdf7-qhjk9             1/1     Running   3 (18h ago)  9d
kube-flannel    kube-flannel-ds-7jj2h                                1/1     Running   2 (7h48m ago)  8d
kube-flannel    kube-flannel-ds-8vh6h                                1/1     Running   3 (18h ago)  9d
kube-flannel    kube-flannel-ds-9f9rg                                1/1     Running   2 (7h48m ago)  8d
kube-flannel    kube-flannel-ds-d7shg                                1/1     Running   18 (8h ago)  9d
kube-flannel    kube-flannel-ds-lf7dw                                1/1     Running   2 (7h48m ago)  8d
kube-system     coredns-668d6bf9bc-tjwhx                             1/1     Running   3 (18h ago)  9d
kube-system     coredns-668d6bf9bc-znq6b                             1/1     Running   3 (18h ago)  9d
kube-system     etcd-master-kira                                       1/1     Running   14 (9h ago)  9d
kube-system     etcd-master-xion                                       1/1     Running   21 (8h ago)  8d
kube-system     kube-apiserver-master-kira                             1/1     Running   26 (9h ago)  9d
kube-system     kube-apiserver-master-xion                             1/1     Running   28 (8h ago)  8d
kube-system     kube-controller-manager-master-kira                   1/1     Running   8 (18h ago)  9d
kube-system     kube-controller-manager-master-xion                   1/1     Running   13 (8h ago)  9d
kube-system     kube-proxy-64s65                                       1/1     Running   2 (7h48m ago)  8d
kube-system     kube-proxy-fvjwp                                       1/1     Running   2 (7h48m ago)  8d
kube-system     kube-proxy-m95ps                                       1/1     Running   2 (7h48m ago)  8d
kube-system     kube-proxy-xpf5s                                       1/1     Running   7 (8h ago)  9d
kube-system     kube-proxy-zrkks                                       1/1     Running   3 (18h ago)  9d
kube-system     kube-scheduler-master-kira                             1/1     Running   8 (18h ago)  9d
kube-system     kube-scheduler-master-xion                             1/1     Running   12 (8h ago)  9d
kube-system     metrics-server-56ff78d5b7-d2cp5                       1/1     Running   5 (18h ago)  8d
root@master-kira:/#

```

Gambar 4. 18 Status *Ready* Setelah Instalasi *Network Plugin*

4.2.3.6 Pemasangan Network File Sistem

Mengingat arsitektur Kubernetes yang diusung menggunakan beberapa *Worker Node*, implementasi volume penyimpanan harus bersifat *ReadWriteMany* agar fail data dapat diakses oleh beberapa *pod* secara serentak, terutama untuk penyimpanan media WordPress, Drupal, serta pangkalan data (*database*).

```

root@kiraryuuki:/home/kira# ls -lh /mnt/data
total 24K
drwxr-xr-x  9  999 root    4.0K Apr 29 22:33 drupal-data
drwxrwxrwx 11 1000 nogroup 4.0K Apr 29 19:33 ghost-data
drwxr-xr-x  8  999 root    4.0K Apr 29 23:04 mysql-drupal
drwxr-xr-x  8  999 root    4.0K Apr 30 01:18 mysql-ghost
drwxrwxrwx 11  999 nogroup 4.0K Apr 30 17:16 mysql-wordpress
drwxr-xr-x  5  999 www-data 4.0K Apr 30 17:15 wordpress-data
root@kiraryuuki:/home/kira#

```

Gambar 4. 19 Direktori Penyimpanan NFS

Sebagai solusinya, layanan *Network File System* (NFS) diimplementasikan langsung pada *host* utama. Penyimpanan ini dipetakan dan dibagi secara spesifik agar data antaraplikasi tidak saling bercampur

Seluruh direktori tersebut kemudian didistribusikan (*network share*) secara eksklusif ke dalam rentang IP jaringan privat klaster. Pendekatan ini memastikan bahwa setiap *node worker* di dalam Kubernetes dapat

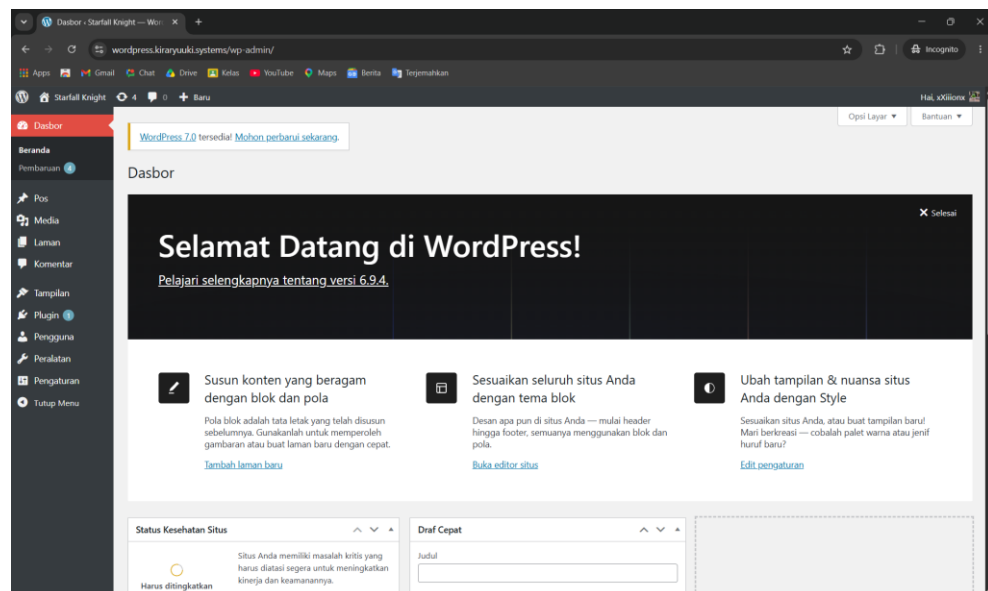
melakukan *mounting* ke penyimpanan utama secara langsung dan aman tanpa terkendala masalah izin akses (*permission*).

Tahap ini sekaligus menandai bahwa fondasi infrastruktur kluster Kubernetes telah sepenuhnya siap untuk tahap peluncuran (*deployment*) aplikasi dan pengujian.

4.2.4 Deployment Aplikasi pada Kubernetes

1. Implementasi pada Aplikasi WordPress

WordPress digunakan sebagai objek pengujian utama yang diimplementasikan ke dalam infrastruktur kluster Kubernetes. Pemilihan platform ini didasarkan pada karakteristik WordPress sebagai aplikasi web dinamis berbasis PHP dan MySQL, yang merepresentasikan arsitektur web tradisional sehingga sangat relevan untuk mensimulasikan dampak serangan HTTP(S) Flood pada lapisan aplikasi.



Gambar 4. 20 Antarmuka Manajemen WordPress

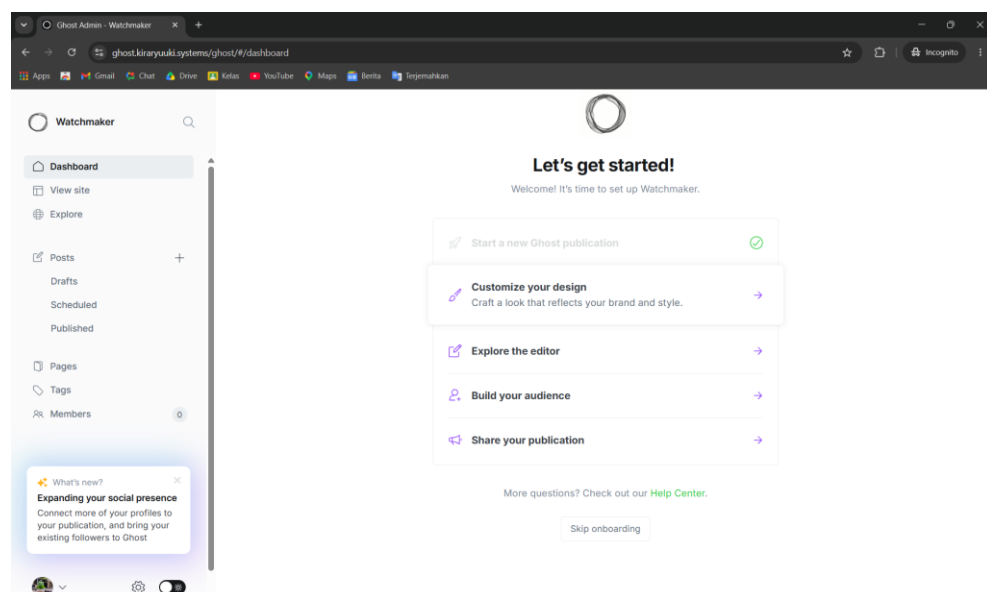
Deployment dilakukan menggunakan objek Pod dan Service, sehingga aplikasi dapat diakses secara internal dalam jaringan kluster. Selain itu, integrasi dengan Nginx Ingress Controller memungkinkan pengelolaan trafik

masuk sekaligus menjadi titik implementasi ModSecurity serta mekanisme caching.

Secara arsitektur, WordPress dalam penelitian ini menggunakan PHP-FPM (FastCGI Process Manager). PHP-FPM bekerja dengan model *process-driven* bersifat *blocking synchronous*, di mana setiap permintaan (*request*) akan mengikat satu proses pekerja (*worker process*) secara eksklusif hingga selesai. Karakteristik ini membuat performa sistem sangat bergantung pada ketersediaan daya prosesor (*CPU time*). Akibatnya, saat klaster menerima lonjakan trafik yang masif, proses pekerja akan cepat mencapai titik jenuh (*worker saturation*), sehingga permintaan berikutnya tertahan dalam antrian (*buffering queue*) dan memicu pembengkakan latensi respons saat pengujian beban.

2. Implementasi pada Aplikasi Ghost

Aplikasi Ghost digunakan sebagai representasi aplikasi web modern berbasis Node.js. Berbeda dengan WordPress, Ghost dipilih karena arsitekturnya yang bersifat *non-blocking I/O*, sehingga memberikan perspektif berbeda dalam penanganan beban request tinggi pada lingkungan kontainer.

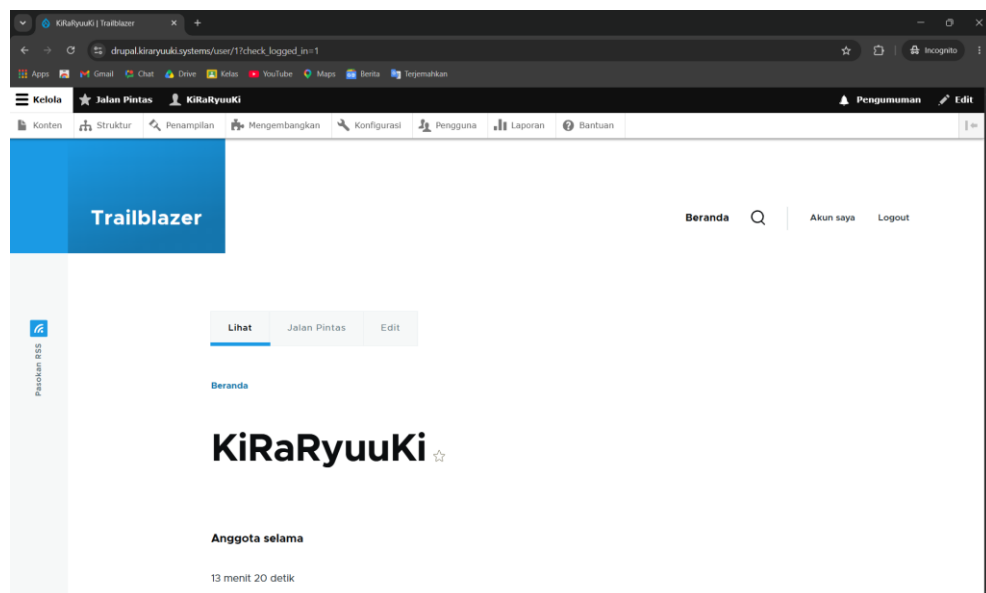


Gambar 4. 21 Antarmuka Manajemen Ghost

Dalam pengujian ini, Ghost di-deploy menggunakan *image* Docker resmi dengan basis database MySQL. Konfigurasi Service diarahkan melalui Ingress Controller yang sama untuk memastikan seluruh trafik tetap melalui lapisan keamanan ModSecurity dan optimasi Nginx Cache sebelum mencapai *backend* Node.js.

3. Implementasi pada Aplikasi Drupal

Sebagai pembanding ketiga, digunakan aplikasi Drupal yang juga berbasis PHP namun memiliki manajemen *cache* dan struktur sistem yang lebih kompleks dibandingkan WordPress. Drupal dipilih untuk menguji sejauh mana mekanisme mitigasi (WAF dan Cache) dapat menangani aplikasi dengan beban pemrosesan *backend* yang lebih berat.



Gambar 4. 22 Antarmuka Manajemen Drupal

Implementasi dalam klaster Kubernetes melibatkan penggunaan Persistent Volume untuk pengelolaan file statis dan sinkronisasi database. Melalui skenario ini, diharapkan dapat terlihat efisiensi *resource* Kubernetes dalam mengelola aplikasi dengan kebutuhan memori (RAM) yang lebih tinggi saat berada di bawah beban trafik yang intens.

Ketiga aplikasi tersebut mewakili variasi teknologi backend (PHP dan Node.js) serta kompleksitas aplikasi yang berbeda, guna mendapatkan data performa mitigasi yang komprehensif.

Seluruh komponen arsitektur klaster telah berhasil dikonfigurasi pada infrastruktur virtual Proxmox dengan status *Ready*. Implementasi keamanan dilakukan melalui aktivasi modul ModSecurity pada Ingress Nginx, diikuti dengan penerapan kebijakan *caching* untuk mengoptimalkan pengiriman konten statis. Verifikasi awal menunjukkan bahwa seluruh komponen pendukung, termasuk *Pod backend* dan *Service*, telah beroperasi sesuai dengan spesifikasi rancangan sistem.

4.3 Skenario Pengujian

4.3.1 Model Serangan HTTP(S) Flood

Serangan yang disimulasikan mengacu pada karakteristik *HTTP(S) Flood* yang dijelaskan dalam literatur, di mana penyerang mengirimkan permintaan HTTP(S) dalam jumlah besar yang menyerupai trafik normal guna menghabiskan sumber daya *server*.

Dalam implementasinya, serangan difokuskan pada kombinasi metode HTTP(S) GET dan HTTP(S) POST. Metode GET digunakan untuk menghasilkan volume permintaan tinggi dengan *overhead* minimal pada sisi penyerang, sementara metode POST ditujukan untuk membebani proses autentikasi pada sisi *backend*.

Tabel 4. 1 Konfigurasi Payload

Metode	Endpoint Target	Jenis Data (Payload)	Tujuan Pengujian
GET	/ (Homepage)	–	Membebani <i>caching</i> dan Ingress
POST	/ (Login Page)	Email/Username & Password	Membebani proses <i>backend</i> & ModSecurity

Pola permintaan yang digunakan bersifat kontinu tanpa jeda signifikan (*no delay*). Hal ini bertujuan untuk mensimulasikan kondisi serangan intensitas tinggi yang secara agresif menargetkan ketersediaan (*availability*) layanan aplikasi.

4.3.2 Parameter Beban dan Intensitas Serangan

Untuk merepresentasikan variasi tingkat serangan, digunakan beberapa tingkat intensitas beban yang dihasilkan melalui perangkat Apache JMeter. Intensitas serangan diukur berdasarkan jumlah permintaan per detik (*request rate*) yang dihasilkan selama pengujian berlangsung.

Penetapan parameter *Ramp-up Period* dilakukan secara proporsional (selama 120 detik) pada beberapa skenario untuk memastikan akselerasi beban yang konsisten dan memberikan waktu adaptasi bagi infrastruktur kluster Kubernetes dalam melakukan alokasi sumber daya.

Tabel 4. 2 Intensitas Beban

Intensitas	Number of Threads	Ramp-up (s)	Loop Count	Target Perilaku / Batasan Evaluasi
Low	100+	120	Kontinu	Mengevaluasi titik jenuh <i>baseline</i> sistem serta ketahanan awal lapisan <i>Nginx Caching</i> murni.
Medium	500+	120	Kontinu	Menganalisis sensitivitas deteksi awal aturan ModSecurity dan mengukur dampak <i>cache bypass</i> .
High	1.000+	120	Kontinu	Menguji batas atas stabilitas orkestrasi kluster <i>nested</i> serta mengamati fenomena <i>CPU throttling</i> pada kernel <i>host</i> .

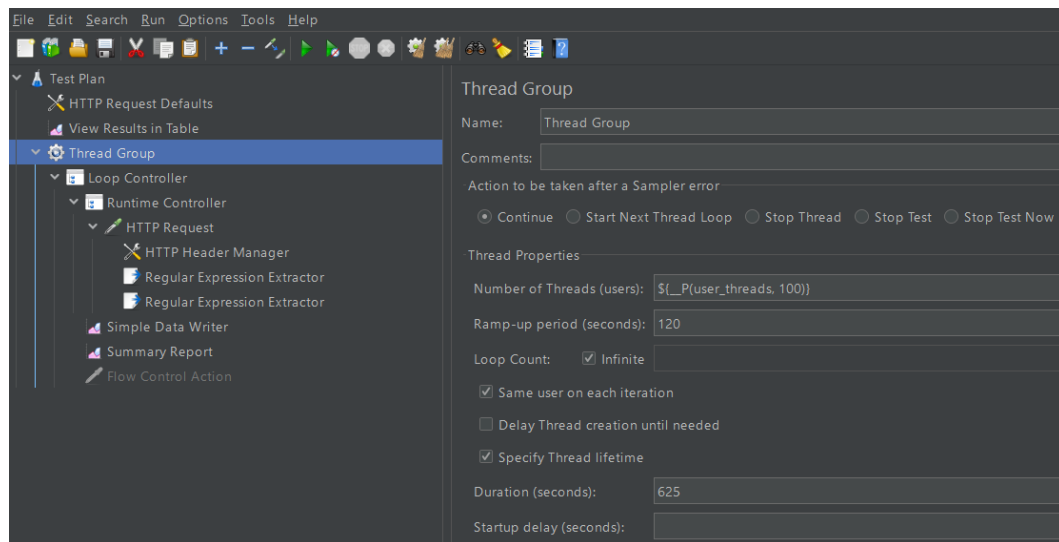
Pendekatan ini digunakan untuk mengamati bagaimana sistem merespons peningkatan beban secara bertahap, serta mengidentifikasi titik mula terjadinya degradasi performa (*saturation point*).

4.3.3 Konfigurasi Pengujian pada Apache JMeter

Simulasi serangan dilakukan menggunakan Apache JMeter dengan konfigurasi parameter yang disesuaikan untuk setiap skenario. Pengaturan utama dilakukan pada dua level komponen, yaitu *Thread Group* untuk mengatur beban trafik dan *HTTP Request* untuk mendefinisikan target serangan.

4.3.3.1 Konfigurasi Thread Group

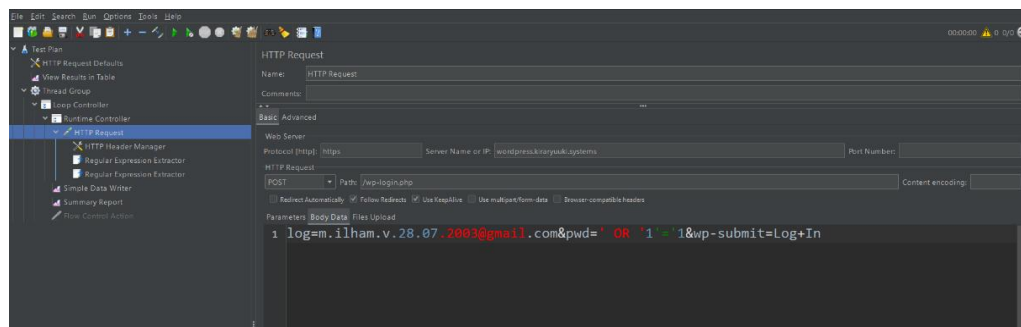
Komponen ini digunakan untuk mensimulasikan sejumlah pengguna virtual secara simultan. Berdasarkan Gambar 4.21, konfigurasi dilakukan pada aspek *Number of Threads* (jumlah pengguna), *Ramp-up Period* (durasi akselerasi beban), serta durasi pengujian melalui fitur *Scheduler*.



Gambar 4. 23 Konfigurasi Thread Group

4.3.3.2 Konfigurasi HTTP Request

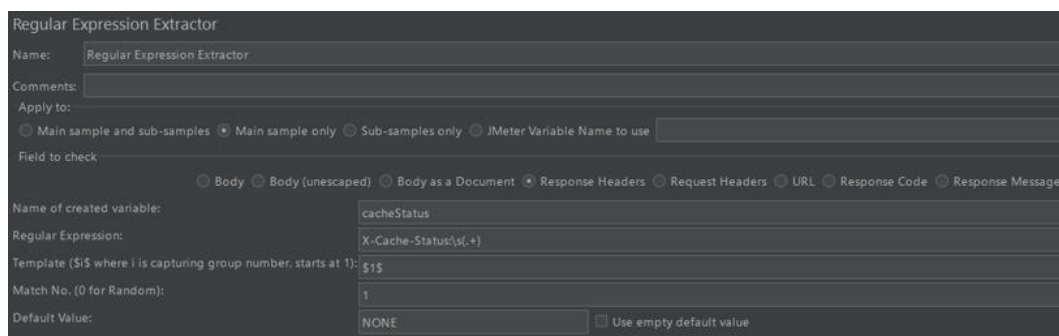
Sampler ini berfungsi untuk mengarahkan trafik serangan ke *endpoint* spesifik pada aplikasi. Sesuai dengan Gambar 4.22, konfigurasi mencakup alamat IP target, protokol HTTP(S), serta metode yang digunakan (GET/POST).



Gambar 4. 24 Konfigurasi HTTP Request

4.3.3.3 Konfigurasi Ekstraksi Respons

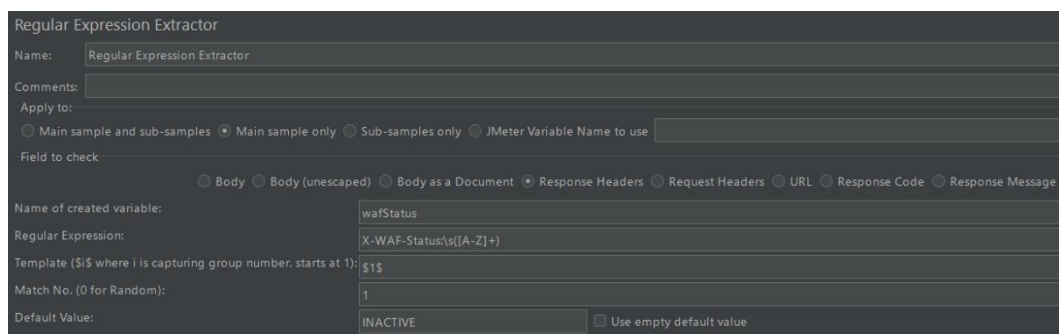
Untuk mengidentifikasi status keberhasilan mekanisme mitigasi dari setiap paket HTTP(S) yang dikembalikan oleh *server*, pengujian ini memanfaatkan komponen *Regular Expression Extractor* pada JMeter. Komponen ini berfungsi untuk memindai dan mengekstrak nilai spesifik dari *header* atau *body* respons.



The screenshot shows the 'Regular Expression Extractor' configuration window. The 'Name' field is 'Regular Expression Extractor'. The 'Apply to' section has 'Main sample only' selected. The 'Field to check' section has 'Response Headers' selected. The 'Name of created variable' is 'cacheStatus'. The 'Regular Expression' is 'X-Cache-Status:(.+)' and the 'Template' is '\$1\$'. The 'Match No.' is '1' and the 'Default Value' is 'NONE'.

Gambar 4. 25 Regular Expression Extractor Cache Status

Berdasarkan Gambar 4.23, ekstraktor dikonfigurasi untuk menangkap status *caching* (misalnya dengan memindai atribut X-Cache-Status) guna menentukan apakah suatu permintaan berhasil dilayani oleh *cache* (*HIT*) atau justru diteruskan menuju *backend* (*MISS/BYPASS*).



The screenshot shows the 'Regular Expression Extractor' configuration window. The 'Name' field is 'Regular Expression Extractor'. The 'Apply to' section has 'Main sample only' selected. The 'Field to check' section has 'Response Headers' selected. The 'Name of created variable' is 'wafStatus'. The 'Regular Expression' is 'X-WAF-Status:s([A-Z]+)' and the 'Template' is '\$1\$'. The 'Match No.' is '1' and the 'Default Value' is 'INACTIVE'.

Gambar 4. 26 Regular Expression Extractor WAF Status

Sementara itu, pada Gambar 4.24, ekstraktor WAF *Status* bertugas mendeteksi pola respons penolakan, seperti keberadaan teks spesifik ModSecurity atau kode status HTTP 403 *Forbidden*. Ekstraksi ini mutlak diperlukan untuk memvalidasi secara logis apakah paket data yang dikirimkan berhasil diblokir oleh *Web Application Firewall* (WAF) atau lolos menuju kerentanan aplikasi.

Konfigurasi ini memungkinkan pengaturan jumlah permintaan, *concurrency*, dan durasi pengujian secara terkontrol, sehingga menghasilkan pola trafik yang konsisten pada setiap skenario. Pendekatan menggunakan JMeter dipilih karena kemampuannya dalam mengontrol *concurrency* dan durasi secara presisi, serta menghasilkan laporan statistik (*reproducibility*) yang lebih valid untuk dianalisis, sehingga validitas data yang dihasilkan tetap terjaga.

4.3.3.4 Otomatisasi Eksekusi Pengujian

Menjalankan simulasi serangan hingga 1.000 pengguna virtual melalui antarmuka grafis (GUI) JMeter berpotensi memicu *overhead* memori pada mesin pengujian (klien Windows), yang dapat mendistorsi validitas pengiriman paket. Oleh karena itu, seluruh komponen pengaturan (mulai dari *Thread Group*, *HTTP Request*, hingga *Regular Expression*) disimpan dan dibungkus ke dalam sebuah dokumen *test plan* berbasis XML. Dokumen ini bertindak sebagai cetak biru (*blueprint*) rancangan serangan yang komprehensif.

Eksekusi dokumen *test plan* tersebut kemudian dijalankan melalui antarmuka baris perintah (Command-Line Interface / CLI) yang diotomatisasi menggunakan Batch script. Skrip ini dirancang dengan struktur perulangan yang secara otomatis menaikkan nilai parameter pengguna virtual dari 100 hingga 1.000 dengan interval kenaikan konstan sebesar 100 *users* pada setiap iterasinya. Setelah satu siklus serangan selesai dieksekusi, skrip ini menginstruksikan mesin klien untuk melakukan penundaan eksekusi selama 300 detik (5 menit). Jeda waktu yang diatur pada skrip klien ini dirancang agar tersinkronisasi secara sempurna dengan

jeda pembersihan kernel (*drop caches*) pada sisi infrastruktur kluster Kubernetes. Adapun dokumen cetak biru XML *test plan* JMeter beserta *source code* skrip Batch otomatisasi ini telah didokumentasikan dan dapat dilihat pada bagian Lampiran.

4.3.4 Pengambilan dan Ekstraksi Data

Mengingat durasi pengujian berlangsung secara intensif (625 detik per skenario) dengan peningkatan *users* yang dinamis (100 hingga 1.000 *users*), proses ekstraksi data tidak dilakukan secara manual guna menghindari kesalahan manusia (*human error*) dan ketidaksinkronan cap waktu (*timestamp*). Pengambilan data dilakukan secara otomatis menggunakan skrip berbasis *Bash* dengan interval rekam setiap 5 detik. Terdapat dua titik pemantauan utama dalam infrastruktur ini:

4.3.4.1 Pengambilan Data Kernel

Pemantauan pada sisi perangkat keras dieksekusi langsung di level sistem operasi *host* Proxmox untuk merekam beban komputasi mentah (*bare-metal*). Pengambilan data ini diotomatisasi menggunakan skrip pemantauan internal yang mengombinasikan beberapa perintah utilitas Linux, yaitu *vmstat* (mengestraksi kalkulasi CPU, interupsi, dan *Context Switches*), *free* (memantau sisa memori dan *cache* kernel), *iostat* (menghitung operasi baca-tulis *disk storage*), serta *uptime* (memetakan *Load Average*).

4.3.4.2 Pengambilan Data Kubernetes

Bersamaan dengan pemantauan tingkat *host*, sebuah skrip pemantauan sekunder dijalankan pada lingkungan *Master Node* untuk mengevaluasi perilaku aplikasi di dalam kontainer. Skrip ini memanfaatkan perintah *kubectl top pods* untuk mengukur utilitas CPU dan RAM yang digunakan oleh *Application Pods* (WordPress, Ghost, Drupal) serta lapisan *Nginx Ingress Controller*. Selain itu, skrip ini secara dinamis melakukan inspeksi catatan sistem (memanfaatkan perintah *kubectl logs | grep*) pada komponen *Ingress* untuk mendeteksi keberadaan interupsi dari *ModSecurity*, yang kemudian diklasifikasikan ke dalam status pemblokiran (BLOCKED atau NONE).

Untuk mengotomatisasi seluruh siklus eksperimen, skrip pemantauan kernel dan skrip pemantauan Kubernetes diikat menggunakan skrip *runner* (pengekseskusi utama). Skrip *runner* ini tidak hanya mengeksekusi perekaman data secara inkremental (kelipatan 100 *users*), tetapi juga menanamkan jeda paksa (*sleep interval*) selama 300 detik antar-skenario pengujian. Waktu jeda ini merupakan perwujudan dari metodologi Zero-Baseline, yang secara saintifik memberikan kesempatan bagi prosesor dan RAM untuk melakukan pendinginan (*cooling down*) dan membersihkan sisa pemrosesan beban (*garbage collection*) sebelum dihantam oleh intensitas serangan pada tahap berikutnya. Untuk menjaga fokus pembahasan pada inti hasil penelitian, seluruh *source code* skrip Bash (*runner*, pemantauan kernel, dan pemantauan Kubernetes) beserta contoh log mentah hasil ekstraksinya disertakan secara terpisah pada bagian Lampiran.

4.3.5 Validasi Perhitungan Metrik Pengujian

Untuk menjamin validitas dan transparansi data sebelum seluruh hasil komparasi disajikan, dilakukan pembuktian matematis melalui perhitungan manual. Validasi ini menggunakan sampel 10 data log mentah pertama yang terekam pada Apache JMeter dengan skenario beban 100 *users* pada aplikasi WordPress. Data mentah tersebut dirangkum dalam Tabel 4.3.

Tabel 4. 3 Sampel Data Pengujian JMeter

timeStamp	elapsed (ms)	responseCode	success	cacheStatus	wafStatus
1781055320196	65	403	false	NONE	ACTIVE
1781055320260	63	403	false	NONE	ACTIVE
1781055320311	49	403	false	NONE	ACTIVE
1781055320380	68	403	false	NONE	ACTIVE
1781055320381	54	403	false	NONE	ACTIVE
1781055320436	51	403	false	NONE	ACTIVE
1781055320436	45	403	false	NONE	ACTIVE
1781055320461	44	403	false	NONE	ACTIVE
1781055320462	42	403	false	NONE	ACTIVE
1781055320521	44	403	false	NONE	ACTIVE

Metode perhitungan manual untuk masing-masing metrik pengukuran yang mengacu pada subbab 3.4.4 dijabarkan sebagai berikut:

a. Tingkat Pemblokiran Serangan

1. Jumlah data dengan **wafStatus** = **ACTIVE** dan **responseCode** = 403 = 10 data
2. Total sampel data = 10 data

$$\text{Attack Blocked Rate} = \left(\frac{10}{10}\right) \times 100\% = 100,00\%$$

b. *Latency (Response Time)*

1. $\sum \text{elapsed} = 65 + 63 + 49 + 68 + 54 + 51 + 45 + 44 + 42 + 44 = 535 \text{ ms}$
2. $N = 10$

$$\text{Response Time} = \frac{535}{10} = 53,50 \text{ ms}$$

c. *Throughput*

1. *timeStamp* Data ke-1 = 1781055320196
2. *timeStamp* Data ke-10 = 1781055320521
3. *elapsed* Data ke-10 = 44 ms
4. Total *Requests* = 10

$$\text{Total Durasi} = (1781055320521 - 1781055320196) + 44 \text{ ms}$$

$$\text{Total Durasi} = 325 \text{ ms} + 44 \text{ ms} = 369 \text{ ms}$$

Jika dikonversi ke dalam satuan *second* (detik):

$$\text{Total Durasi} = \frac{369}{1.000} = 0,369 \text{ s}$$

Setelah total durasi jendela waktu didapatkan sebesar 0,369 detik, maka nilai *throughput* murni dari 10 data *request* tersebut dapat dihitung dengan rumus berikut:

$$\text{Throughput} = \frac{10}{0,369 \text{ s}} = 27,10 \text{ req/s}$$

d. *Error Rate*

1. Jumlah data dengan success = false = 10 data
2. Total sampel data = 10 data

$$\text{Error Rate} = \left(\frac{10}{10}\right) \times 100\% = 100,00\%$$

e. *Cache Hit Rasio*

1. Jumlah data dengan **cacheStatus** = **HIT** = 0 data
2. Total sampel data = 10 data

$$\text{Cache Hit Ratio} = \left(\frac{0}{10}\right) \times 100\% = 0,00\%$$

Berdasarkan penjabaran perhitungan manual di atas, seluruh metrik yang dihasilkan telah terverifikasi secara matematis. Sebagai acuan format pelaporan, hasil dari perhitungan sampel 10 data pada skenario beban 100 *users* tersebut dirangkum pada Tabel 4.4. Format tabel ini merupakan standar penyajian data performa yang akan digunakan secara konsisten pada seluruh subbab hasil pengujian berikutnya.

Tabel 4. 4 Validasi Perhitungan Manual Metrik Pengujian

User	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	Cache Hit (%)	WAF Active (%)
100	0.00	100.00	53.50	27.10	0.00	100.00

4.4 Hasil Pengujian Idle Sistem

Sebelum skenario pengujian beban bertahap (*Incremental Load Test*) dan simulasi serangan yang lebih intens dijalankan, dilakukan pengukuran kondisi *Idle* dengan rentang beban minimal 1 hingga 5 *users*. Pengujian ini bertujuan untuk mendapatkan parameter acuan (*Baseline*) performa murni dari masing-masing

aplikasi (WordPress, Ghost, dan Drupal) saat berjalan tanpa adanya beban trafik eksternal yang agresif.

Data ini berfungsi sebagai kontrol untuk mengukur sejauh mana beban kerja nantinya memengaruhi stabilitas kluster Kubernetes. Tanpa mengetahui performa standar, degradasi sistem yang ditimbulkan saat pengujian beban dan serangan terjadi tidak dapat diukur secara objektif.

4.4.1 Performa Idle WordPress

Karakteristik performa dan penggunaan sumber daya kluster untuk lingkungan WordPress dalam kondisi beban rendah dirangkum pada Tabel 4.5 dan Tabel 4.6.

Tabel 4. 5 Performa Idle WordPress

Users	Success Rate (%)	Error Rate (%)	Avg Response (ms)	Avg Latency (ms)	Throughput (req/s)	APDEX
1	100.00	0.00	137.39	108.22	7.27	1.00
2	100.00	0.00	158.27	124.27	12.48	1.00
3	99.97	0.03	240.21	146.97	12.28	0.99
4	100.00	0.00	238.82	174.31	16.44	0.98
5	99.99	0.01	306.89	199.90	15.97	0.97
Average	99.99	0.01	216.32	150.73	12.89	0.99

Tabel 4. 6 Konsumsi Daya pada Pod WordPress

Users	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)	Samples
1	128.96	297.00	291.59	455.00	288.00
2	178.64	448.00	247.72	458.00	369.00
3	180.78	543.00	236.38	460.00	364.00
4	178.13	787.00	219.76	459.00	512.00
5	169.48	638.00	223.14	460.00	516.00
Average	167.20	542.60	243.72	458.40	409.80

Pada kondisi *Idle*, WordPress menunjukkan rata-rata waktu respons sebesar 216,32 ms dengan indeks kepuasan pengguna (APDEX) sebesar 0,99. Dari sisi

penggunaan *resource*, rata-rata komputasi CPU yang dihabiskan berada di angka 167,20 *millicores* (m), dengan kecenderungan konsumsi RAM yang relatif rendah dan stabil di kisaran 243,72 MiB.

4.4.2 Performa Idle Ghost

Data performa pengujian serta alokasi sumber daya untuk platform Ghost berbasis Node.js dirangkum pada Tabel 4.7 dan Tabel 4.8.

Tabel 4. 7 Performa Idle Ghost

Users	Success Rate (%)	Error Rate (%)	Avg Response (ms)	Avg Latency (ms)	Throughput (req/s)	APDEX
1	100.00	0.00	80.10	73.20	12.47	1.00
2	100.00	0.00	90.80	84.15	21.74	1.00
3	100.00	0.00	92.71	86.75	31.80	1.00
4	100.00	0.00	110.61	98.40	35.59	1.00
5	99.99	0.01	117.02	102.98	42.21	1.00
Average	100.00	0.00	98.25	89.10	28.76	1.00

Tabel 4. 8 Konsumsi Daya pada Pod Ghost

Users	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)	Samples
1	104.63	243.00	300.48	397.00	276.00
2	159.79	356.00	305.07	398.00	264.00
3	176.18	476.00	285.70	444.00	380.00
4	176.89	469.00	279.53	451.00	396.00
5	216.43	595.00	273.85	454.00	375.00
Average	166.78	427.80	288.93	428.80	338.20

Arsitektur *non-blocking I/O* pada Ghost memberikan efisiensi pemrosesan permintaan yang tinggi, menghasilkan *throughput* rata-rata sebesar 28,76 req/s dengan nilai APDEX sempurna (1,00). Alokasi memori rata-rata berada pada batas aman 288,93 MiB dengan pemrosesan CPU yang seimbang di angka 166,78 m.

4.4.3 Performa Idle Drupal

Performa hasil pengujian serta visualisasi alokasi *resource* pada lingkungan aplikasi Drupal dirangkum pada Tabel 4.9 dan Tabel 4.10.

Tabel 4. 9 Performa Idle Drupal

Users	Success Rate (%)	Error Rate (%)	Avg Response (ms)	Avg Latency (ms)	Throughput (req/s)	APDEX
1	100.00	0.00	74.49	68.31	13.41	1.00
2	99.99	0.01	95.58	82.25	20.65	1.00
3	100.00	0.00	94.15	85.51	31.32	1.00
4	99.99	0.01	107.54	91.94	36.48	1.00
5	99.99	0.01	101.96	85.33	48.04	1.00
Average	99.99	0.01	94.74	82.67	29.98	1.00

Tabel 4. 10 Konsumsi Daya pada Pod Drupal

Users	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)	Samples
1	40.88	92.00	315.35	430.00	368.00
2	43.48	124.00	318.90	431.00	476.00
3	84.56	152.00	304.18	435.00	282.00
4	95.11	176.00	305.83	438.00	280.00
5	123.01	246.00	307.62	440.00	276.00
Average	77.41	158.00	310.38	434.80	336.40

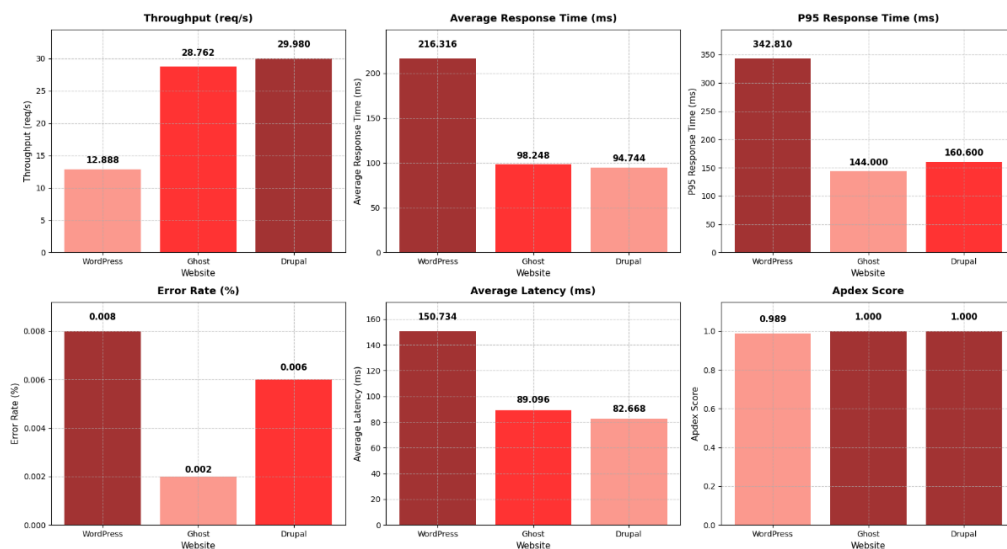
Drupal menunjukkan optimasi manajemen pemrosesan instruksi yang sangat baik dengan tingkat rata-rata penggunaan komputasi CPU paling rendah, yaitu sebesar 77,41 m. Meskipun demikian, struktur data internal Drupal membutuhkan alokasi memori RAM dasar tertinggi di antara ketiga CMS, dengan nilai rata-rata sebesar 310,38 MiB.

4.4.4 Hasil Komparasi Idle Sistem

Untuk mempermudah identifikasi perbedaan karakteristik performa, Tabel 4.11 menyajikan perbandingan metrik dari ketiga aplikasi pada kondisi *Idle*.

Tabel 4. 11 Komparasi Idle Sistem

Website	Avg Response (ms)	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	APDEX
WordPress	216.316	150.734	0.008	12.888	0.989
Ghost	98.248	89.096	0.002	28.762	1.000
Drupal	94.744	82.668	0.006	29.980	1.000



Gambar 4. 27 Grafik Komparasi Idle Sistem

Berdasarkan data pada Tabel 4.11, terlihat perbedaan performa yang cukup kontras. Drupal dan Ghost menunjukkan hasil yang sangat impresif dengan waktu respons rata-rata di bawah 100 ms. Sebaliknya, WordPress membutuhkan waktu yang lebih lama, yaitu sekitar 216,32 ms. Hal ini menunjukkan bahwa dalam kondisi tanpa beban, arsitektur Ghost yang berbasis Node.js serta optimasi mesin internal Drupal lebih ringan dalam memproses permintaan masuk dibandingkan dengan WordPress.

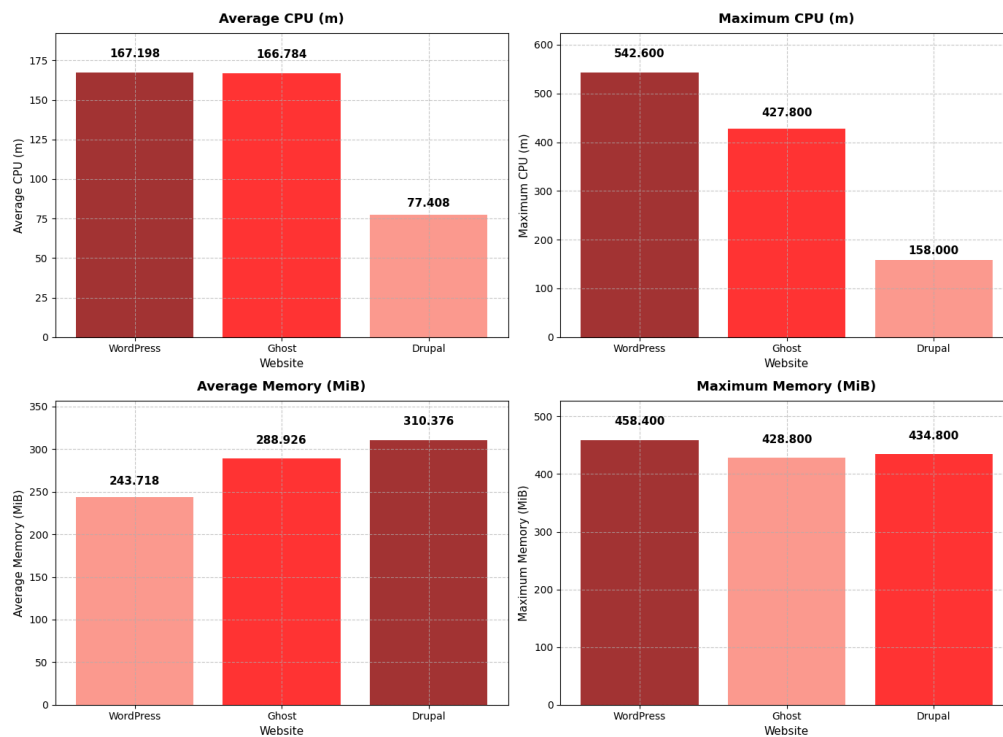
Nilai *throughput* pada Ghost dan Drupal mencapai hampir 30 req/s, dua kali lipat lebih tinggi dari WordPress yang berada di angka 12,89 req/s. Angka APDEX yang menyentuh nilai sempurna (1,000) pada Ghost dan Drupal menegaskan bahwa pengguna akan merasakan pengalaman akses yang sangat lancar pada kondisi ini.

4.4.5 Hasil Komparasi Alokasi Resource Idle Sistem

Evaluasi penggunaan sumber daya komputasi fisik pada tingkat Pod aplikasi dirangkum dalam Tabel 4.12 untuk menganalisis efisiensi masing-masing arsitektur pada level *node worker*.

Tabel 4. 12 Hasil Komparasi Resource Idle Sistem (Node)

Website	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)
WordPress	167.198	542.600	243.718	458.400
Ghost	166.784	427.800	288.926	428.800
Drupal	77.408	158.000	310.376	434.800



Gambar 4. 28 Grafik Komparasi Resource Idle Sistem

Melalui data pada Tabel 4.12, penggunaan sumber daya memberikan gambaran mengenai strategi manajemen memori dan CPU pada masing-masing aplikasi. Satu hal yang menonjol adalah penggunaan CPU pada Drupal yang sangat rendah, hanya sekitar 77,41 m, menjadikannya yang paling efisien di antara ketiganya dalam menjaga status *idle* agar tidak membebani prosesor.

Meskipun Drupal paling hemat dalam penggunaan daya CPU, platform ini membutuhkan alokasi RAM dasar tertinggi (310,38 Mi) berdasarkan pemetaan data pada Tabel 4.12. Sebaliknya, WordPress menggunakan RAM paling rendah (243,72 Mi) namun mengonsumsi daya CPU tertinggi (167,20 m). Fenomena ini membuktikan adanya karakteristik *trade-off* teknis: Drupal cenderung menggunakan memori lebih besar untuk mekanisme *caching* internal demi meringankan beban kerja kalkulasi CPU, sementara WordPress lebih mengandalkan proses komputasi aktif secara langsung saat menerima permintaan.

Di sisi lain, Ghost menunjukkan konsistensi efisiensi yang sangat baik. Sebagai platform berbasis Node.js yang menerapkan arsitektur *non-blocking I/O*, penggunaan memori dan CPU-nya berada di angka menengah yang stabil, menjadikannya sistem yang paling seimbang antara alokasi *resource* dan kecepatan respons yang dihasilkan.

4.5 Hasil Pengujian Baseline Sistem

Setelah mendapatkan parameter acuan pada kondisi *Idle*, pengujian dilanjutkan ke tahap *Baseline* untuk mengamati respons sistem terhadap peningkatan trafik yang disimulasikan. Skenario pengujian ini diimplementasikan dengan memberikan beban kerja berupa *Incremental Load Testing* mulai dari 10 hingga 100 pengguna virtual (*concurrent users*) secara bertahap dengan kenaikan konstan.

Hal ini dilakukan untuk memetakan stabilitas layanan serta pola konsumsi sumber daya, baik pada tingkat *Pod* aplikasi, *Node* klaster (Kubernetes), maupun sumber daya pada kernel *host* (Proxmox). Melalui tahapan ini, diperoleh data komparatif mengenai daya tahan WordPress, Ghost, dan Drupal saat beban kerja ditingkatkan secara berkelanjutan sebelum diterapkan mekanisme mitigasi (*non-mitigation state*).

4.5.1 Performa Baseline WordPress

Karakteristik performa, penggunaan sumber daya tingkat kontainer (*Pod*), dan kondisi kernel *host* untuk lingkungan WordPress dirangkum pada Tabel 4.13, Tabel 4.14, dan Tabel 4.15.

Tabel 4. 13 Performa Baseline WordPress

Users	Success Rate (%)	Error Rate (%)	Avg Response (ms)	Avg Latency (ms)	Throughput (req/s)	APDEX
10	100.00	0.00	368.95	314.58	25.89	0.92
20	99.98	0.02	875.57	693.82	21.77	0.60
30	99.96	0.04	1281.87	1022.99	22.70	0.52
40	99.96	0.04	1693.82	1585.52	22.49	0.42
50	99.96	0.04	2051.56	1901.07	23.23	0.35
60	99.97	0.03	2514.37	2401.69	22.72	0.35
70	99.96	0.04	3013.59	2864.99	22.13	0.32
80	99.93	0.07	3645.20	3468.17	20.91	0.23
90	99.91	0.09	3868.07	3676.38	22.20	0.28
100	99.87	0.13	4211.02	3998.76	22.64	0.31
Average	99.95	0.05	2352.40	2192.80	22.67	0.43

Tabel 4. 14 Konsumsi Sumber Daya Pod WordPress

Users	Replicas	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)
10	4	222.45	979.00	233.59	504.00
20	4	260.11	868.00	244.45	543.00
30	4	203.78	869.00	248.72	554.00
40	4	286.30	985.00	271.39	556.00
50	4	267.29	973.00	280.78	533.00
60	4	302.44	999.00	308.01	536.00
70	4	287.94	966.00	313.39	538.00
80	5	268.67	999.00	321.24	665.00
90	4	299.18	990.00	348.23	613.00
100	5	268.75	819.00	337.51	719.00
Average	4	266.69	944.70	290.73	576.10

Tabel 4. 15 Utilisasi Resource Kernel WordPress

Users	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
10	67.35	100.00	4057.73	4454.00	713.44	17608.00	23169
20	76.87	101.00	3919.27	4343.00	511.54	9924.00	23441
30	64.65	101.00	3869.46	4301.00	357.07	1496.00	21239
40	83.54	101.00	3731.29	4300.00	402.15	3416.00	23545
50	80.54	101.00	3665.55	4293.00	932.91	48156.00	23267
60	88.59	101.00	3516.53	4264.00	378.13	1647.52	24182
70	82.53	101.00	3504.07	4236.00	398.31	2600.00	23021
80	87.92	101.00	3320.16	4263.00	580.71	9424.00	23643
90	87.84	101.00	3301.93	4222.00	385.25	1560.00	23601
100	87.16	101.00	3239.32	4139.00	370.15	2164.00	23441
Average	80.70	100.90	3612.53	4281.50	502.97	9799.55	23255

Berdasarkan data performa pada Tabel 4.13, WordPress mengalami tekanan performa yang sangat signifikan seiring bertambahnya pengguna simultan. Waktu respons melonjak drastis dari 368,95 ms pada beban 10 *users* menjadi 4.211,02 ms saat mencapai beban 100 *users*. Menariknya, meskipun beban *users* meningkat 10 kali lipat, nilai *throughput* WordPress cenderung tertahan (statis) di kisaran 22 req/s. Hal ini mengindikasikan bahwa kapasitas pemrosesan aplikasi telah mencapai titik jenuh (*saturation point*).

Dari sisi infrastruktur klaster Tabel 4.14, Kubernetes berupaya mempertahankan ketersediaan layanan dengan menjaga alokasi *Horizontal Pod Autoscaler* (HPA) pada tingkat 4 replika *pod* untuk beban mayoritas, dan baru melakukan *scaling* maksimal hingga 5 replika *pod* saat beban mencapai puncaknya di 100 *users*. Namun, penambahan replika ini tidak mampu menahan lonjakan beban karena batasan fisik pada kernel *host* Proxmox Tabel 4.15. Utilisasi CPU pada level kernel *host* langsung menyentuh rata-rata 80,70% dengan *Max CPU* mencapai 100,90% (mengalami *oversubscription*). Beban kerja yang berat ini juga ditunjukkan oleh tingginya nilai *Context Switches* (rata-rata 23.255), yang berarti

prosesor menghabiskan banyak waktu hanya untuk berganti-ganti tugas mengelola antrian instruksi WordPress yang belum terselesaikan.

4.5.2 Performa Baseline Ghost

Karakteristik performa, alokasi pod, dan kondisi kernel *host* untuk platform Ghost dirangkum dalam Tabel 4.16, Tabel 4.17, dan Tabel 4.18.

Tabel 4. 16 Performa Baseline Ghost

Users	Success Rate (%)	Error Rate (%)	Avg Response (ms)	Avg Latency (ms)	Throughput (req/s)	APDEX
10	100.00	0.00	127.48	120.29	74.89	1.00
20	100.00	0.00	229.80	217.24	82.90	0.98
30	99.99	0.01	359.90	340.80	79.34	0.90
40	99.99	0.01	494.55	465.44	76.97	0.81
50	99.99	0.01	593.13	566.60	80.21	0.78
60	99.98	0.02	717.21	670.63	80.98	0.74
70	99.98	0.02	843.21	811.10	79.13	0.69
80	99.94	0.06	1205.10	876.15	72.85	0.67
90	99.99	0.01	1156.43	1134.58	74.13	0.60
100	99.96	0.04	1286.51	1243.84	73.99	0.55
Average	99.98	0.02	701.33	644.67	77.54	0.77

Tabel 4. 17 Konsumsi Sumber Daya Pod Ghost

Users	Replicas	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)
10	3	305.80	724.00	271.77	460.00
20	4	264.79	843.00	234.75	474.00
30	4	315.46	874.00	223.01	479.00
40	4	302.04	862.00	223.33	485.00
50	4	312.96	945.00	224.15	488.00
60	4	212.62	915.00	212.72	494.00
70	4	278.61	932.00	225.35	504.00
80	4	295.68	898.00	230.12	508.00
90	4	206.62	964.00	219.73	516.00

100	4	331.04	888.00	235.49	500.00
Average	4	282.56	884.50	230.04	490.80

Tabel 4. 18 Utilisasi Resource Kernel Ghost

Users	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
10	74.26	97.00	4027.16	4308.00	615.37	5552.00	19684
20	76.89	100.00	3903.61	4318.00	466.74	3288.00	19776
30	88.42	100.00	3868.11	4286.00	566.08	7296.00	20670
40	83.87	101.00	3857.71	4280.00	619.82	7776.00	20466
50	90.62	100.00	3814.40	4278.00	634.03	6516.00	20813
60	65.88	101.00	3862.24	4275.00	568.75	9896.00	18873
70	85.32	101.00	3867.49	4259.00	742.84	10228.00	20518
80	55.99	101.00	4071.88	4517.00	760.60	14320.00	18002
90	62.00	101.00	3994.74	4379.00	786.25	11988.00	18694
100	88.33	100.00	3958.92	4345.00	876.78	9612.00	20722
Average	77.16	100.20	3922.63	4324.50	663.73	8647.20	19822

Data pada Tabel 4.16 menunjukkan bahwa platform Ghost yang berbasis Node.js memiliki ketahanan yang lebih baik dalam menangani antrean trafik dibandingkan WordPress. Pada beban 10 *users*, Ghost mencatatkan waktu respons yang cukup cepat sebesar 127,48 ms, dan meningkat secara linear hingga 1.286,51 ms pada beban 100 *users*. Keunggulan Ghost terlihat pada nilai *throughput*-nya yang mampu meroket hingga rata-rata 77,54 req/s (hampir 3,5 kali lipat lebih tinggi dari WordPress).

Pada level kontainer Tabel 4.17, kluster Kubernetes mengalokasikan rata-rata 4 replika *pod* Ghost dengan konsumsi memori yang sangat efisien dan stabil di kisaran 230,04 MiB. Karakteristik *non-blocking I/O* pada arsitektur Node.js terbukti efektif meminimalkan penggunaan RAM pada *Pod*. Meskipun demikian, pada level kernel *host* Proxmox Tabel 4.18, utilisasi CPU tetap terdorong tinggi hingga rata-rata 77,16% dengan puncaknya menyentuh 100,20%. Nilai *Context*

Switches pada Ghost (19.822) tercatat lebih rendah dibandingkan WordPress, menandakan proses pergantian eksekusi instruksi di dalam CPU berjalan sedikit lebih efisien.

4.5.3 Performa Baseline Drupal

Karakteristik performa, alokasi pod, dan kondisi kernel *host* untuk platform Drupal dirangkum dalam Tabel 4.19, Tabel 4.20, dan Tabel 4.21.

Tabel 4. 19 Performa Baseline Drupal

Users	Success Rate (%)	Error Rate (%)	Avg Response (ms)	Avg Latency (ms)	Throughput (req/s)	APDEX
10	100.00	0.00	92.80	86.22	102.82	1.00
20	100.00	0.00	119.66	113.97	159.12	1.00
30	96.85	3.15	166.99	159.91	170.94	1.00
40	88.47	11.53	196.58	186.39	193.65	0.99
50	89.34	10.66	242.57	225.62	196.11	0.98
60	99.99	0.01	442.80	376.13	128.86	0.89
70	89.93	10.07	456.12	388.15	146.02	0.87
80	92.79	7.21	452.19	401.58	168.31	0.88
90	96.51	3.49	630.57	523.64	135.70	0.71
100	99.99	0.01	694.80	562.48	137.97	0.68
Average	95.39	4.61	349.51	302.41	153.95	0.90

Tabel 4. 20 Konsumsi Sumber Daya Pod Drupal

Users	Replicas	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)
10	2	214.99	425.00	299.78	515.00
20	3	278.23	567.00	283.46	562.00
30	2	277.49	635.00	302.13	574.00
40	3	226.25	796.00	277.06	578.00
50	3	244.79	806.00	277.44	585.00
60	2	212.93	607.00	302.63	587.00
70	3	190.52	574.00	301.77	589.00
80	3	184.32	761.00	299.61	600.00

90	3	201.56	617.00	312.08	599.00
100	2	203.44	590.00	326.71	599.00
Average	3	223.45	637.80	298.27	578.80

Tabel 4. 21 Utilisasi Resource Kernel Drupal

Users	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
10	47.36	81.00	4234.03	4349.00	1189.71	12640.00	22864
20	64.34	100.00	4115.43	4310.00	1208.13	16004.00	26855
30	64.64	99.00	4071.51	4237.00	1094.51	11208.00	26851
40	61.08	101.00	3999.92	4236.00	1492.84	7964.00	25650
50	69.21	101.00	3926.05	4218.00	1553.58	7648.00	27211
60	53.67	100.00	4035.48	4197.00	1158.21	6644.00	24017
70	49.13	101.00	3995.40	4202.00	1219.23	13748.00	23276
80	50.36	100.00	4022.88	4245.00	1319.51	8844.00	22950
90	51.95	101.00	3999.62	4225.00	1161.82	12416.00	23702
100	51.51	100.00	4008.38	4235.00	1096.10	8172.00	23464
Average	56.33	98.40	4040.87	4245.40	1249.36	10528.80	24684

Melalui data Tabel 4.19, Drupal menunjukkan performa kecepatan pemrosesan terbaik di antara ketiga CMS dalam kondisi pengujian *baseline*. Rata-rata waktu respons Drupal pada beban 100 *users* hanya menyentuh angka 694,80 ms dengan raihan *throughput* tertinggi yang sangat impresif, yaitu rata-rata 153,95 req/s. Efisiensi ini menghasilkan indeks kepuasan pengguna (APDEX) yang tetap bertahan tinggi di angka 0,90. Namun, efisiensi kecepatan ini harus dibayar dengan munculnya *Error Rate* rata-rata sebesar 4,61%, yang mengindikasikan bahwa beberapa *request* mulai digugurkan oleh server demi menjaga kecepatan respons permintaan lainnya.

Pada level Pod kontainer Tabel 4.20, Drupal hanya membutuhkan alokasi rata-rata 3 replika *pod*, dengan rata-rata konsumsi CPU kontainer paling rendah (223,45 m). Penghematan daya komputasi ini juga berlanjut hingga ke level kernel

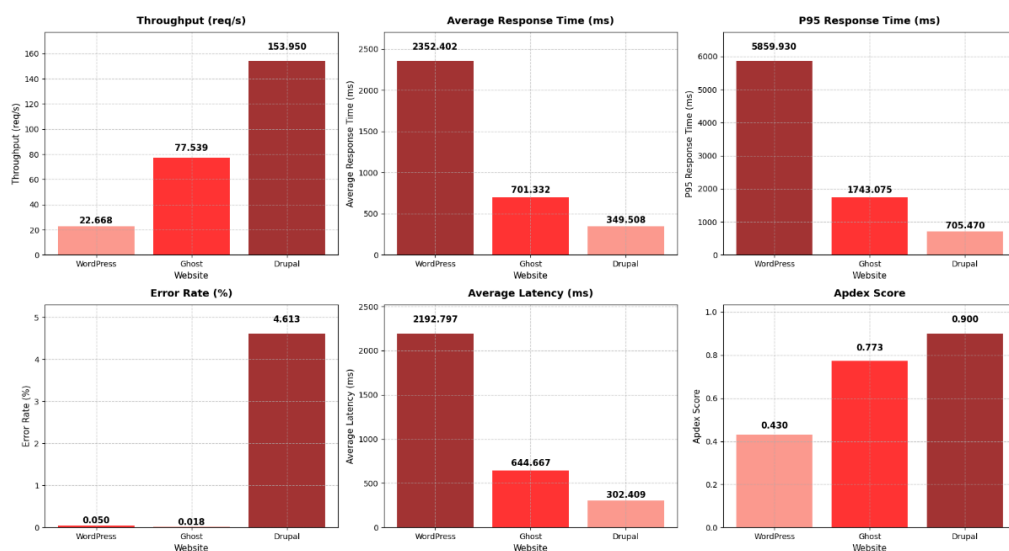
host Proxmox Tabel 4.21, di mana rata-rata utilisasi CPU *host* hanya berada di angka 56,33% (paling rendah di antara ketiga CMS). Kendati demikian, performa tinggi Drupal ini disokong oleh aktivitas *Input/Output* yang sangat tinggi pada kernel, mencatatkan nilai rata-rata *Avg I/O* sebesar 1.249,36 S. Hal ini membuktikan bahwa mekanisme *caching* internal Drupal melakukan pembacaan dan penulisan data ke memori/disk secara agresif untuk meringankan kerja prosesor.

4.5.4 Hasil Komparasi Baseline Sistem

Untuk mengevaluasi perbedaan ketahanan antarsistem saat menerima beban kerja bertahap, Tabel 4.22 menyajikan rekapitulasi performa rata-rata pada lapisan aplikasi.

Tabel 4. 22 Hasil Komparasi Baseline Sistem

Website	Avg Response (ms)	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	APDEX
WordPress	2352.402	2192.797	0.050	22.668	0.430
Ghost	701.332	644.667	0.018	77.539	0.773
Drupal	349.508	302.409	4.613	153.950	0.900



Gambar 4. 29 Grafik Komparasi Baseline Sistem

Berdasarkan data pada Tabel 4.22, terlihat penurunan performa yang signifikan pada ketiga sistem saat beban ditingkatkan hingga 100 *users*. WordPress mengalami degradasi paling parah dengan *Average Response Time* mencapai 2.352,40 ms dan skor APDEX yang merosot tajam ke angka 0,430, yang masuk ke dalam kategori *Frustrated*.

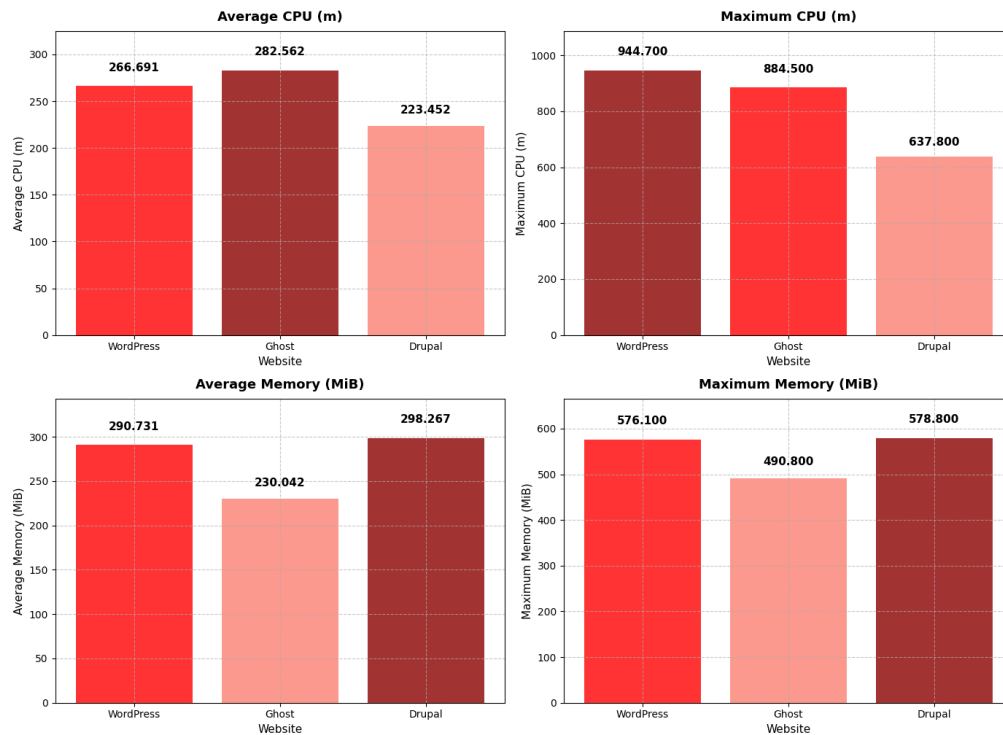
Di sisi lain, Drupal menunjukkan ketahanan yang unggul dengan raihan *throughput* rata-rata tertinggi mencapai 153,95 req/s dan *Response Time* yang masih terjaga di angka 349,51 ms. Hal ini mengindikasikan bahwa arsitektur internal Drupal jauh lebih siap menangani antrean permintaan (*request*) yang padat dibandingkan WordPress dalam kondisi tanpa optimasi tambahan.

4.5.5 Hasil Komparasi Alokasi Resource Baseline Sistem

Evaluasi penggunaan sumber daya pada level infrastruktur kontainer di dalam klaster disajikan pada Tabel 4.23 untuk meninjau efisiensi alokasi CPU dan memori RAM.

Tabel 4. 23 Komparasi Resource Baseline Sistem

Website	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)
WordPress	266.691	944.700	290.731	576.100
Ghost	282.562	884.500	230.042	490.800
Drupal	223.452	637.800	298.267	578.800



Gambar 4. 30 Grafik Komparasi Resource Baseline Sistem

Pengamatan pada level Pod kontainer melalui data Tabel 4.23 menunjukkan bahwa lonjakan trafik secara langsung memaksa penggunaan unit pemrosesan CPU ke titik maksimal. WordPress mencatatkan *Max CPU* hingga 944,70 m, yang mengindikasikan beban kerja telah mendekati batas alokasi 1 *core* penuh.

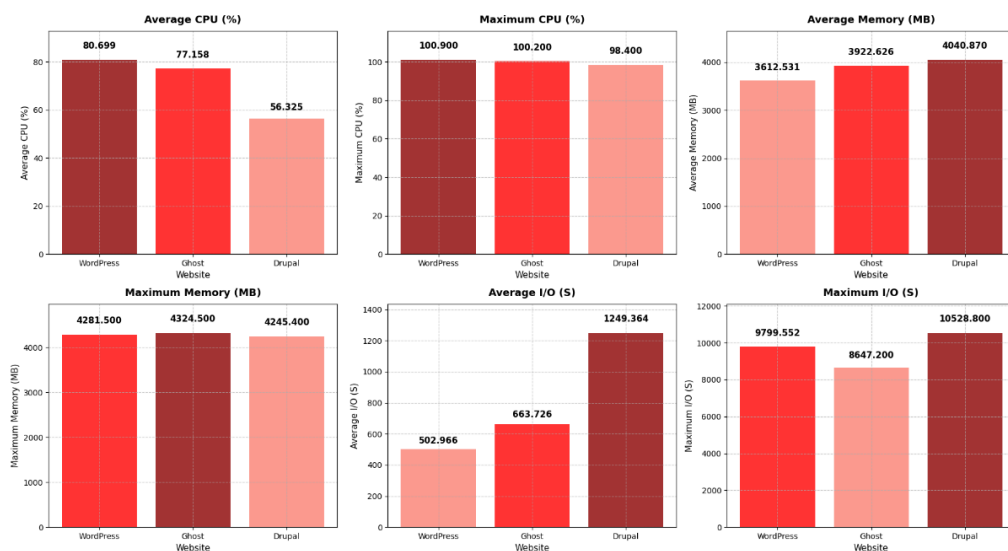
Meskipun Ghost memiliki rata-rata penggunaan CPU yang sedikit lebih tinggi dari WordPress (282,56 m) pada data Tabel 4.23, platform berbasis Node.js tersebut mampu mempertahankan manajemen memori yang jauh lebih efisien dengan rata-rata konsumsi hanya sebesar 230,04 Mi, dibandingkan Drupal dan WordPress yang telah melampaui ambang batas 290 Mi.

4.5.6 Hasil Komparasi Utilisasi Resource Baseline Sistem

Analisis mendalam pada lapisan *hypervisor* fisik Proxmox disajikan pada Tabel 4.24 untuk mengungkap dampak penanganan trafik langsung terhadap sistem operasi *host*.

Tabel 4. 24 Komparasi Utilisasi Resource Baseline Sistem

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	80.699	100.900	3612.531	4281.500	502.966	9799.552	23254.881
Ghost	77.158	100.200	3922.626	4324.500	663.726	8647.200	19821.862
Drupal	56.325	98.400	4040.870	4245.400	1249.364	10528.800	24683.979



Gambar 4. 31 Grafik Komparasi Kernel Baseline Sistem

Analisis mendalam pada level kernel *host* melalui Tabel 4.24 mengungkap penyebab utama terjadinya degradasi performa pada sistem. Kenaikan beban trafik memaksa penggunaan kapasitas CPU Host hingga menyentuh batas kritis 100% (*Max CPU*). Tingginya nilai *Context Switches* yang menembus angka 24.684 pada Drupal dan 23.255 pada WordPress menandakan bahwa kernel *hypervisor* Proxmox sedang melakukan alokasi penjadwalan proses (*process scheduling*) secara masif untuk mengelola antrean instruksi yang masuk secara bersamaan.

Selain itu, nilai I/O pada Drupal mencatatkan angka rata-rata tertinggi sebesar 1.249,36 S, yang berkorelasi linier dengan tingginya *throughput* yang dihasilkan pada lapisan aplikasi. Namun, tingginya beban CPU hingga mencapai 100% ini menjadi sinyal jelas bahwa infrastruktur telah mencapai titik jenuh (*saturation*

point). Tanpa adanya mekanisme mitigasi seperti *WAF filtering* atau *caching layer* eksternal di depan klaster, infrastruktur ini sangat rentan mengalami kegagalan layanan total (*down time*) jika beban terus ditingkatkan melampaui batas 100 pengguna virtual.

4.6 Hasil Pengujian Incremental Load Sistem

Berdasarkan data pengujian *Baseline* pada Tabel 4.22 hingga Tabel 4.24, ditemukan bahwa infrastruktur mengalami degradasi performa yang sangat ekstrem saat beban mencapai 100 pengguna virtual. Hal ini disebabkan oleh keterbatasan sumber daya fisik pada *host* Proxmox yang hanya dibekali dengan 2 *Core* CPU dan 8 GB RAM, namun harus menopang beban orkestrasi dari 5 *node* klaster Kubernetes (2 *Master Node* dan 3 *Worker Node*).

Mengingat adanya limitasi perangkat keras tersebut, pengujian pada tahap beban kerja inkremental dengan intensitas 500 pengguna virtual dipastikan tidak akan mampu mencapai pemrosesan *throughput* yang optimal jika tetap menggunakan metode pemrosesan standar (*non-mitigation state*). Oleh karena itu, pada tahap ini diterapkan mekanisme mitigasi berupa *Nginx Caching* dan *Web Application Firewall* (WAF) ModSecurity secara terpisah maupun kombinasi untuk menjaga ketersediaan layanan (*availability*).

4.6.1 Pengujian Incremental Cache Only

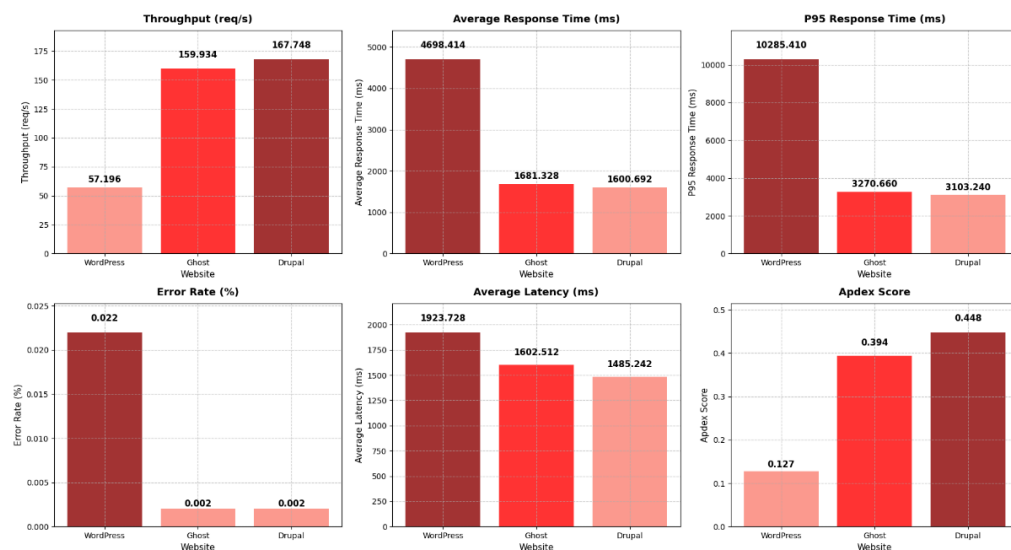
Skenario ini menguji efisiensi klaster ketika hanya mengandalkan lapisan *caching* pada Ingress Controller untuk melayani permintaan tanpa menyentuh interpreter PHP dan basis data (*backend*). Pengujian dibagi menjadi dua kondisi, yaitu tanpa parameter (*Base Cache*) dan dengan manipulasi parameter URL (*Cache Inject Query*).

4.6.1.1 Pengujian Base Cache

Karakteristik performa aplikasi, konsumsi sumber daya *Pod*, dan metrik kernel *host* Proxmox dirangkum dalam Tabel 4.25, Tabel 4.26, dan Tabel 4.27.

Tabel 4. 25 Performa Aplikasi pada Skenario Base Cache

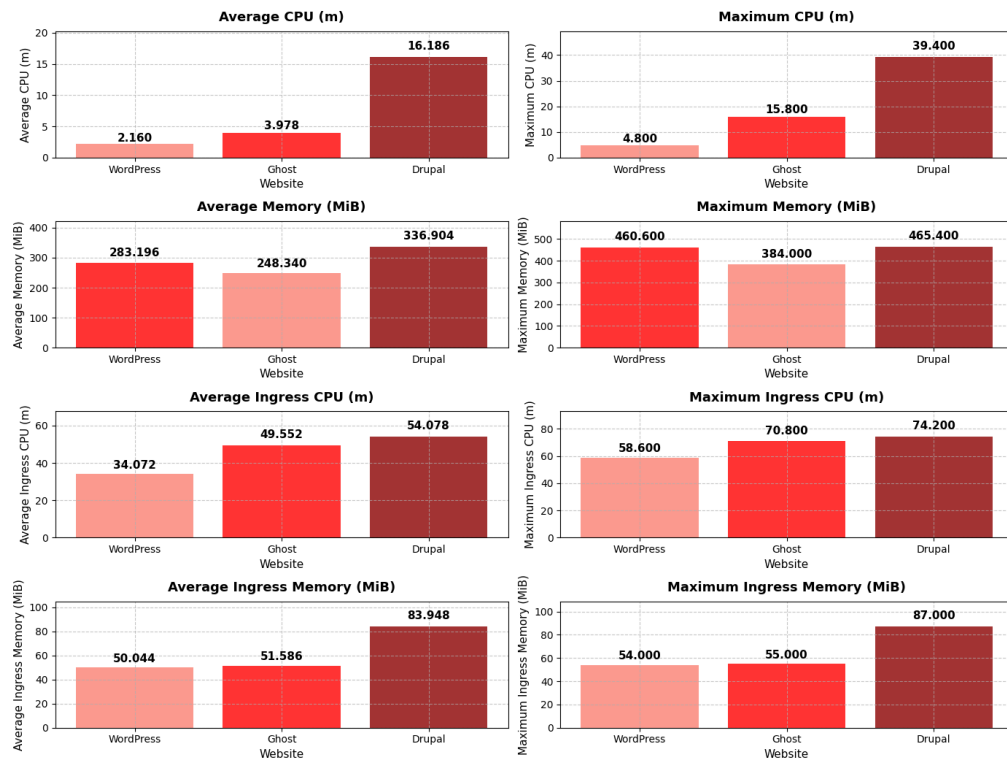
Website	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	Cache Hit (%)	APDEX
WordPress	1923.728	0.022	57.196	100.000	0.127
Ghost	1602.512	0.002	159.934	100.000	0.394
Drupal	1485.242	0.002	167.748	100.000	0.448



Gambar 4. 32 Grafik Hasil Komparasi Base Cache Sistem

Tabel 4. 26 Konsumsi Resource Pod pada Skenario Base Cache

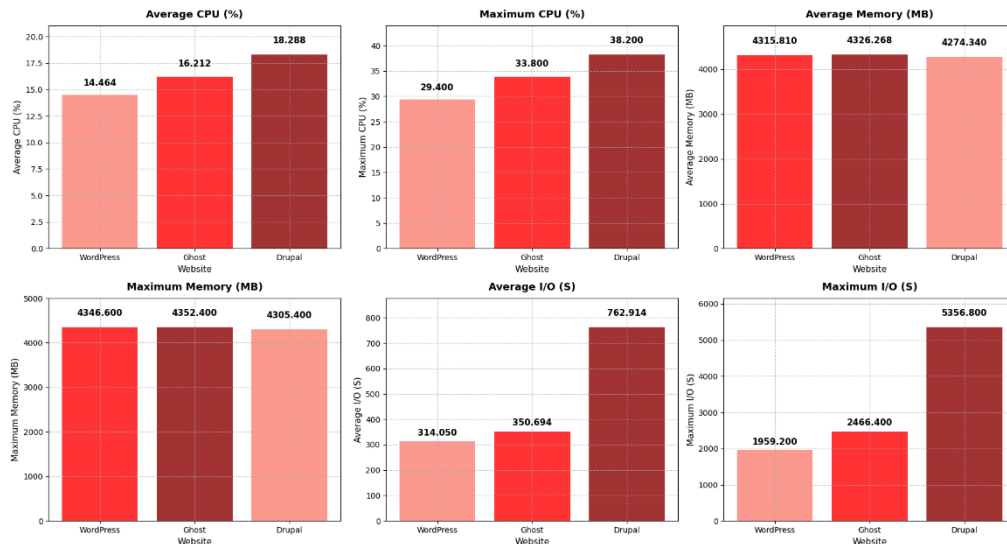
Website	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)
WordPress	2.160	4.800	283.196	460.600	34.072	50.044
Ghost	3.978	15.800	248.340	384.000	49.552	51.586
Drupal	16.186	39.400	336.904	465.400	54.078	83.948



Gambar 4. 33 Grafik Komparasi Resource Base Cache

Tabel 4. 27 Utilisasi Kernel Host Proxmox Base Cache

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	14.464	29.400	4315.810	4346.600	314.050	1959.200	16277.658
Ghost	16.212	33.800	4326.268	4352.400	350.694	2466.400	18157.316
Drupal	18.288	38.200	4274.340	4305.400	762.914	5356.800	18897.426



Gambar 4. 34 Grafik Komparasi Kernel Base Cache

Data pada Tabel 4.25 hingga Tabel 4.27 membuktikan keberhasilan fungsi *Nginx Caching* dalam melindungi ketersediaan layanan sistem. Berdasarkan metrik performa aplikasi pada Tabel 4.25, ketika *Cache Hit Ratio* menyentuh angka sempurna yaitu 100,000%, seluruh permintaan dari 500 *users* dipotong langsung pada lapisan pintu gerbang klaster (*Nginx Ingress Controller*).

Dampak positif yang sangat masif terlihat pada tingkat *Pod* aplikasi Tabel 4.26, di mana rata-rata utilisasi CPU kontainer merosot tajam hingga di bawah 20 *millicores* (m) untuk ketiga CMS. WordPress yang pada pengujian *baseline* (100 *users*) menarik CPU hingga mendekati batas 1 *core* penuh (944,70 m), kini hanya memerlukan rata-rata 2,16 m karena tidak ada instruksi yang diteruskan ke interpreter PHP.

Pada level kernel *host* Proxmox Tabel 4.27, utilisasi rata-rata CPU terjaga sangat aman di bawah 20,00% dengan penurunan nilai *Context Switches* secara drastis ke kisaran 16.000–18.000 pengalihan proses. Hal ini membuktikan bahwa *caching* mampu meminimalkan beban komputasi *host* secara signifikan. Namun, dari sisi kenyamanan pengguna (APDEX), nilai indeks masih berada di tingkat rendah (*Frustrated*) dengan latensi rata-rata di atas 1,4 detik. Hal ini mengindikasikan adanya antrean pemrosesan koneksi (*connection backlog*) pada

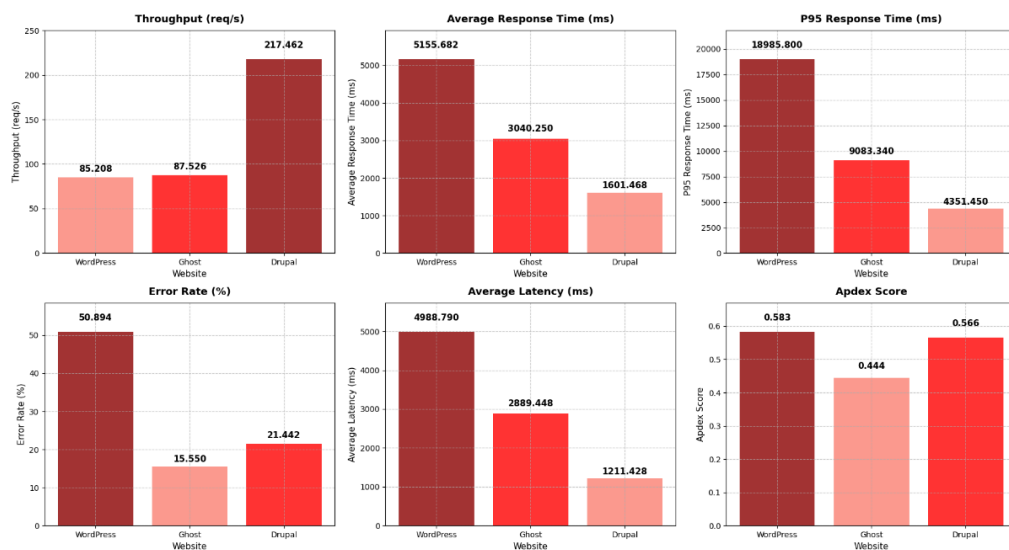
network socket Ingress akibat keterbatasan alokasi CPU fisik yang hanya memiliki 2 core.

4.6.1.2 Pengujian Cache Inject Query

Skenario ini mensimulasikan taktik penyerang yang memanipulasi parameter acak pada URL (`/?id=1' OR '1'='1`) untuk memaksa Nginx mengabaikan salinan data lama dan melakukan *cache miss/bypass*. Parameter performa dan konsumsi daya sistem dirangkum dalam Tabel 4.28, Tabel 4.29, dan Tabel 4.30.

Tabel 4. 28 Performa Aplikasi pada Skenario Cache Inject Query

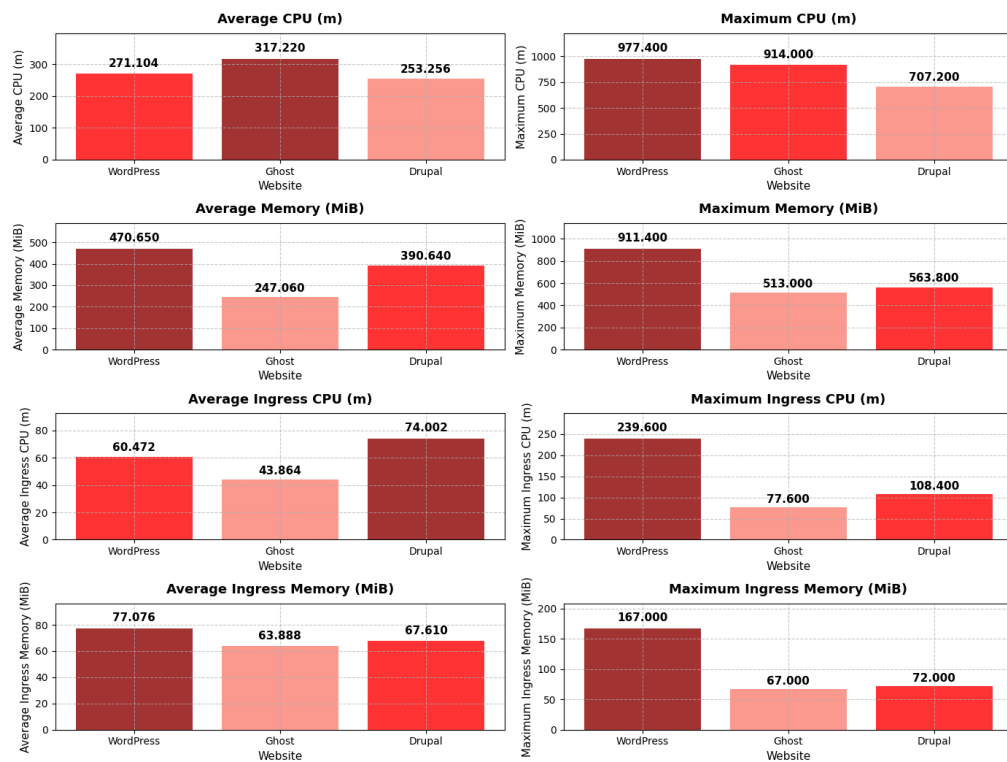
Website	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	Cache Bypass (%)	APDEX
WordPress	4988.790	50.894	85.208	100.000	0.583
Ghost	2889.448	15.550	87.526	100.000	0.444
Drupal	1211.428	21.442	217.462	100.000	0.566



Gambar 4. 35 Grafik Hasil Komparasi Cache Inject Sistem

Tabel 4. 29 Resource Pod pada Skenario Cache Inject Query

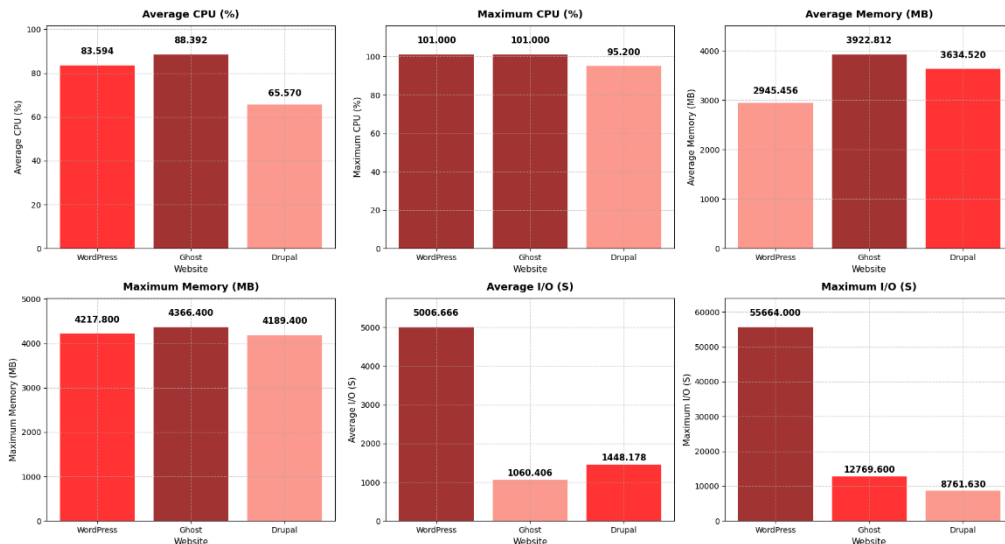
Website	Avg CPU (m)	Max CPU (m)	Avg Memory (Mi)	Max Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)
WordPress	271.104	977.400	470.650	911.400	60.472	77.076
Ghost	317.220	914.000	247.060	513.000	43.864	63.888
Drupal	253.256	707.200	390.640	563.800	74.002	67.610



Gambar 4. 36 Grafik Komparasi Resource Cache Inject

Tabel 4. 30 Utilisasi Kernel Host Proxmox Cache Inject Query

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	83.594	101.000	2945.456	4217.800	5006.666	55664.000	29180.502
Ghost	88.392	101.000	3922.812	4366.400	1060.406	12769.600	21073.616
Drupal	65.570	95.200	3634.520	4189.400	1448.178	8761.630	27306.848



Gambar 4. 37 Grafik Komparasi Kernel Cache Inject

Melalui data pada Tabel 4.28 hingga Tabel 4.30, terlihat kelemahan fatal dari taktik pertahanan jika hanya mengandalkan *caching layer*. Ketika penyerang menyuntikkan *query string* unik, tingkat *Cache Bypass* menyentuh angka 100,00%. Kondisi ini memaksa Ingress Controller untuk meneruskan seluruh beban trafik 500 *users* langsung menuju *Application Pods*.

Dampaknya langsung memicu kegagalan sistem berskala besar. Tingkat kegagalan layanan (*Error Rate*) pada WordPress meroket tajam hingga mencapai angka 50,894% pada Tabel 4.28. Hal ini terjadi karena interpreter *backend* mengalami kegagalan alokasi memori, yang ditunjukkan oleh lonjakan *Max Memory Pod* WordPress hingga mencapai 911,400 Mi pada Tabel 4.29.

Pada lapisan *hypervisor* Proxmox Tabel 4.30, manipulasi URL ini memaksa utilitas CPU kembali ke titik kritis maksimal 101,00% dengan nilai *Context Switches* yang meningkat ekstrem hingga melampaui 29.000. Tingginya aktivitas penulisan dan pembacaan *cache miss* pada WordPress juga memicu beban kerja *disk storage* rata-rata (*Avg I/O*) melompat ke angka 5.006,67 S. Hal ini membuktikan bahwa mekanisme *Cache-Only* tidak efektif untuk memitigasi serangan *HTTP Flood* yang dinamis karena penyerang dapat memanipulasi *cache key* dengan mudah.

4.6.2 Pengujian Incremental ModSecurity Only

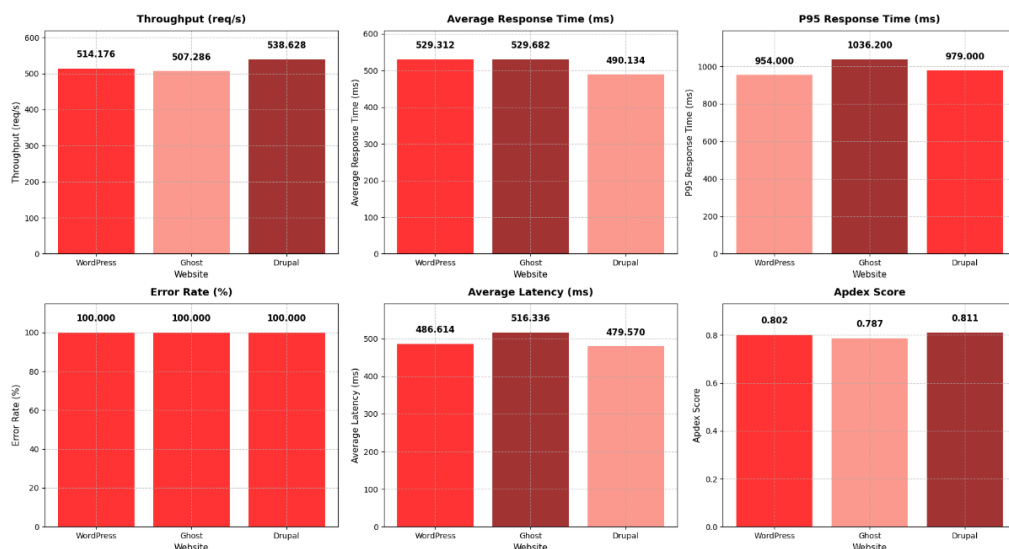
Skenario ini menguji ketahanan klaster ketika mekanisme *caching* dinonaktifkan, dan sistem sepenuhnya bergantung pada filter aturan *Core Rule Set* (CRS) ModSecurity untuk memeriksa serta memblokir paket serangan pada lapisan aplikasi.

4.6.2.1 Pengujian Bypass ModSecurity

Kondisi ini menguji performa filter ketika menerima lonjakan trafik masif dari pengguna virtual sah tanpa muatan aturan eksploitasi berbahaya. Parameter performa sistem dirangkum dalam Tabel 4.31, Tabel 4.32, dan Tabel 4.33.

Tabel 4. 31 Performa Aplikasi pada Skenario Bypass ModSecurity

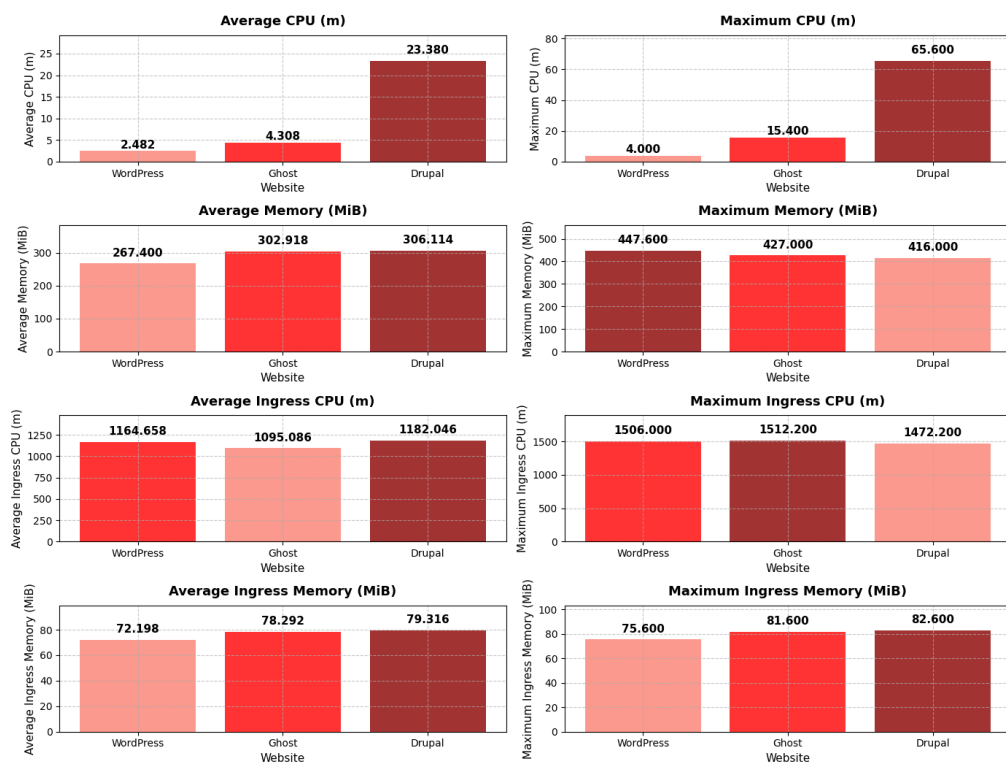
Website	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	WAF Active (%)	APDEX
WordPress	486.614	100.000	514.176	99.968	0.802
Ghost	516.336	100.000	507.286	99.984	0.787
Drupal	479.570	100.000	538.628	99.986	0.811



Gambar 4. 38 Grafik Komparasi Bypass ModSecurity Sistem

Tabel 4. 32 Resource Pod pada Skenario Bypass ModSecurity

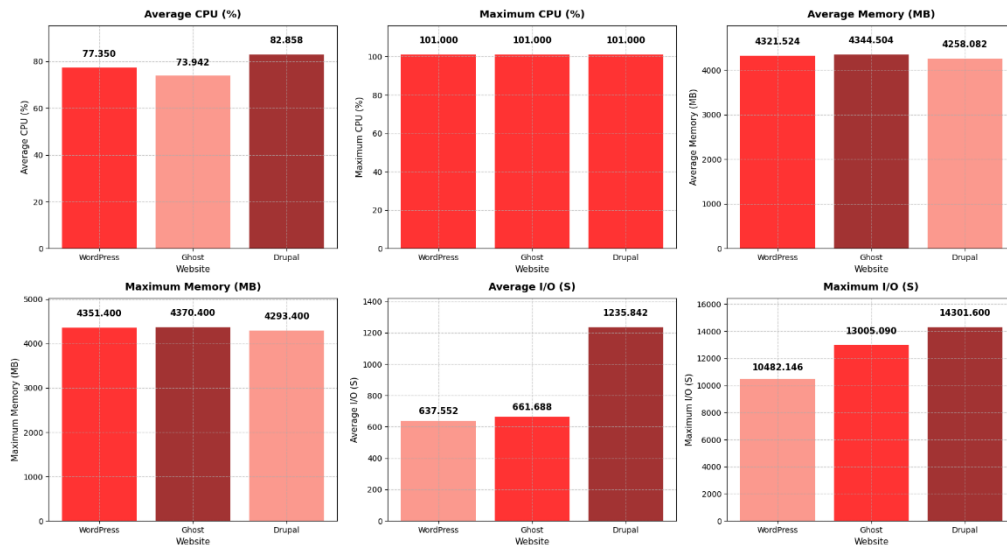
Website	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
WordPress	2.482	267.400	1164.658	72.198	95.786
Ghost	4.308	302.918	1095.086	78.292	97.272
Drupal	23.380	306.114	1182.046	79.316	95.314



Gambar 4. 39 Grafik Komparasi Resource Bypass ModSecurity

Tabel 4. 33 Utilisasi Kernel Host Proxmox Bypass ModSecurity

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	77.350	101.000	4321.524	4351.400	637.552	10482.146	25478.866
Ghost	73.942	101.000	4344.504	4370.400	661.688	13005.090	24762.788
Drupal	82.858	101.000	4258.082	4293.400	1235.842	14301.600	26699.420



Gambar 4. 40 Grafik Komparasi Kernel Bypass ModSecurity

Berdasarkan data komparasi pada Tabel 4.31 hingga Tabel 4.33, tercatat fenomena performa yang unik pada fungsi WAF. Angka *Error Rate* menunjukkan nilai mutlak 100,00% untuk seluruh sistem Tabel 4.31. Namun, nilai kegagalan ini bukan disebabkan oleh server yang *crash*, melainkan karena ModSecurity secara agresif menganggap tingginya intensitas trafik dari *IP Address* penguji sebagai anomali serangan *DDoS/Flood*, sehingga memicu aksi pemblokiran masif dengan tingkat keberhasilan pemblokiran (*ModSecurity Blocked*) rata-rata di atas 95,00%.

Tindakan pemblokiran ini menghasilkan respons balik berupa kode HTTP 403 Forbidden. Karena Ingress Controller langsung memutus koneksi dan memberikan respons balik kesalahan di level pintu gerbang klaster, beban kerja *Pod* aplikasi internal berhasil ditekan hingga di bawah 24 m, meskipun konsumsi CPU Ingress melonjak drastis hingga melampaui 1.100 m pada Tabel 4.32.

Meskipun demikian, proses inspeksi berkas aturan (*rule parsing*) terhadap 500 koneksi masuk secara simultan menuntut kinerja komputasi yang sangat tinggi dari komponen WAF. Akibatnya, beban kerja bergeser ke level Ingress Controller, di mana konsumsi CPU *Ingress* melonjak drastis hingga melampaui 1.100 m Tabel 4.32. Kenaikan beban komputasi filter ini juga berimbas pada lapisan kernel *host* Proxmox Tabel 4.33 dengan utilisasi rata-rata CPU mencapai kisaran 73,00%–

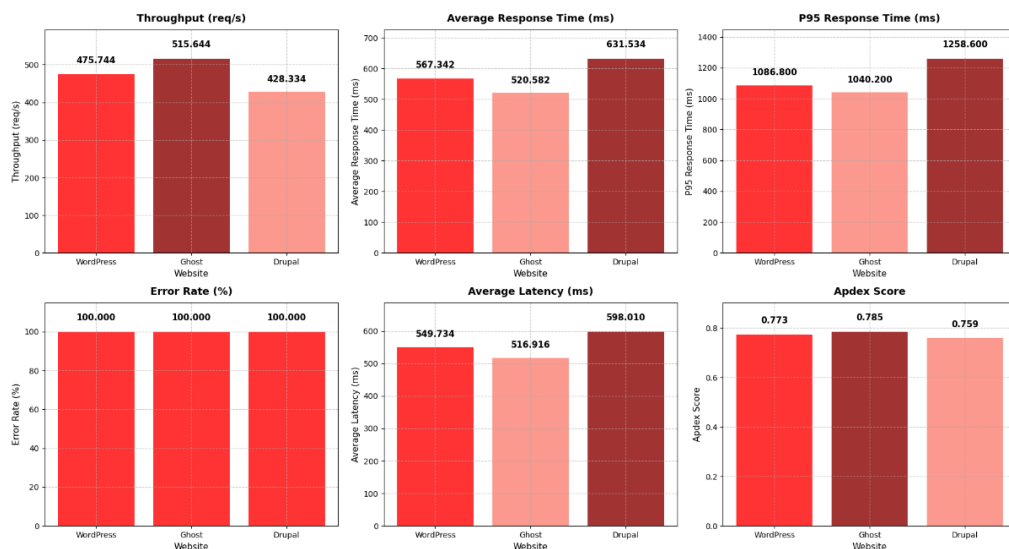
82,00% dan nilai *Context Switches* bertahan tinggi di angka 25.000 pengalihan proses.

4.6.2.2 Pengujian ModSecurity Payload

Skenario ini dilakukan dengan menyisipkan muatan teks eksekusi berbahaya (*malicious payload*) pada metode HTTP POST menuju halaman autentikasi aplikasi untuk menguji fungsi inspeksi mendalam (*Deep Packet Inspection*) dari WAF. Parameter performa sistem dirangkum dalam Tabel 4.34, Tabel 4.35, dan Tabel 4.36.

Tabel 4. 34 Performa Aplikasi pada Skenario ModSecurity Payload

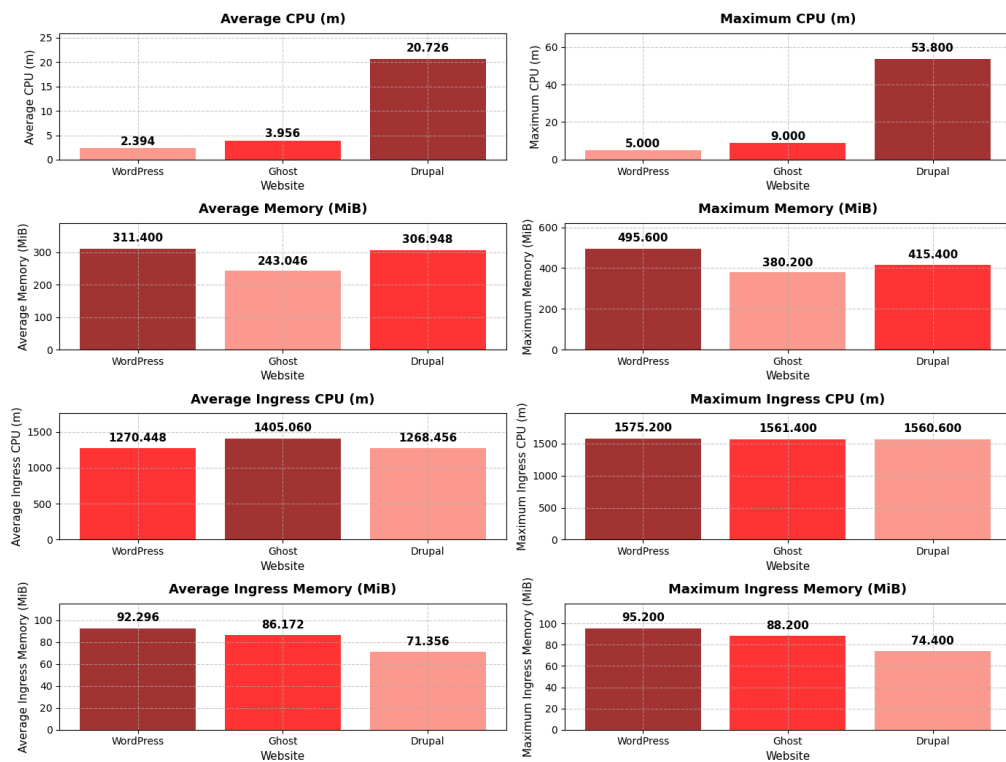
Website	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	WAF Active (%)	APDEX
WordPress	549.734	100.000	475.744	99.982	0.773
Ghost	516.916	100.000	515.644	99.982	0.785
Drupal	598.010	100.000	428.334	99.960	0.759



Gambar 4. 41 Grafik Komparasi ModSecurity Payload Sistem

Tabel 4. 35 Resource Pod pada Skenario ModSecurity Payload

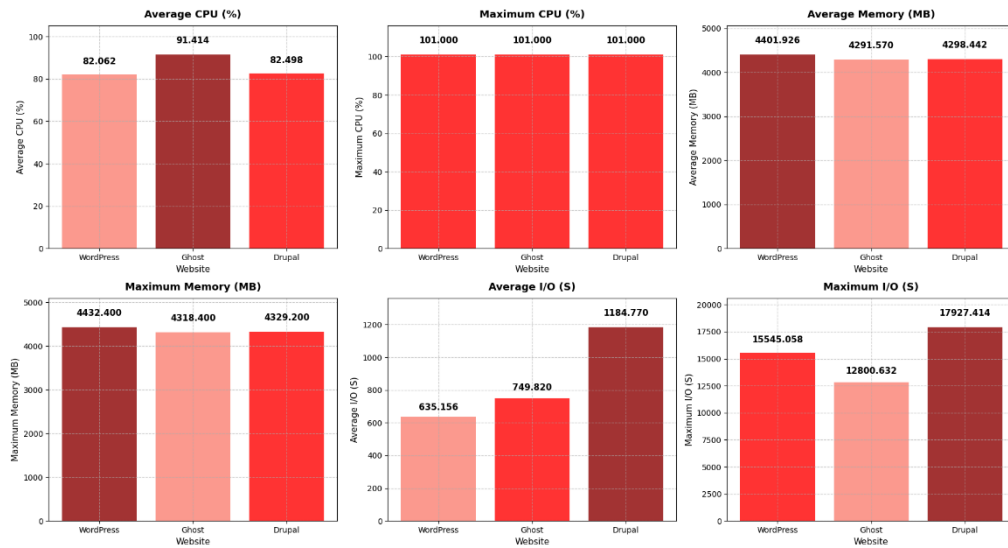
Website	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
WordPress	2.394	311.400	1270.448	92.296	97.272
Ghost	3.956	243.046	1405.060	86.172	99.520
Drupal	20.726	306.948	1268.456	71.356	94.962



Gambar 4. 42 Grafik Komparasi Resource ModSecurity Payload

Tabel 4. 36 Utilisasi Kernel Host Proxmox ModSecurity Payload

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	82.062	101.000	4401.926	4432.400	635.156	15545.058	25359.730
Ghost	91.414	101.000	4291.570	4318.400	749.820	12800.632	27554.666
Drupal	82.498	101.000	4298.442	4329.200	1184.770	17927.414	24904.028



Gambar 4. 43 Grafik Komparasi Kernel ModSecurity Payload

Berdasarkan visualisasi data pada Tabel 4.34 hingga Tabel 4.36, penyuntikan muatan data berbahaya (*payload injection*) memicu beban komputasi paling tinggi pada sistem keamanan WAF. Ketika mendeteksi string ilegal pada metode POST, tingkat akurasi pemblokiran ModSecurity meningkat hingga mencapai angka tertinggi sebesar 99,52% pada platform Ghost Tabel 4.35.

Meskipun *Application Pods* internal di dalam kluster tetap aman dari bahaya *exploit* (utilitas CPU internal kontainer <21 m), kompensasi biaya komputasi yang harus ditanggung oleh infrastruktur *host* fisik menjadi sangat besar. Karena ModSecurity harus mengurai dan mencocokkan setiap baris parameter POST terhadap ratusan pustaka *signature* aturan, rata-rata konsumsi CPU pada Ingress Controller membubung tinggi hingga menyentuh puncaknya di angka 1.405,06 m pada lingkungan Ghost Tabel 4.35.

Kondisi tersebut berdampak langsung pada level kernel *host* Proxmox Tabel 4.36, di mana utilitas rata-rata CPU meningkat tajam hingga menyentuh angka 91,41% dengan puncaknya mencapai batas maksimal alokasi perangkat keras, yaitu 101,00%. Tingginya nilai *Context Switches* (mencapai 27.554,67) mempertegas bahwa keterbatasan spesifikasi prosesor fisik *host* (2 *core*) menjadi *bottleneck*

utama ketika WAF dipaksa melakukan kalkulasi inspeksi paket secara mendalam di bawah kondisi beban kerja yang padat.

4.6.3 Pengujian Incremental Kombinasi

Skenario kombinasi ini merupakan tahap pengujian puncak untuk mengevaluasi kinerja infrastruktur kluster Kubernetes ketika lapisan keamanan *Web Application Firewall* (WAF) ModSecurity dan optimasi *Nginx Caching* diaktifkan secara simultan. Pengujian dilakukan secara inkremental dengan meningkatkan beban dari 100 hingga 500 pengguna virtual (*concurrent users*).

Tujuan dari skenario ini adalah menganalisis bagaimana kedua modul saling melengkapi: ModSecurity bertindak sebagai penyaring muatan berbahaya (*payload filtering*), sementara *Nginx Caching* memotong jalur pemrosesan trafik berulang agar tidak membebani interpreter *backend*.

4.6.3.1 Performa Kombinasi WordPress

Karakteristik performa aplikasi, konsumsi sumber daya *Pod*, dan metrik kernel *host* Proxmox untuk lingkungan WordPress dirangkum dalam Tabel 4.37, Tabel 4.38, dan Tabel 4.39.

Tabel 4. 37 Performa Kombinasi WordPress

Users	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	WAF Active (%)	APDEX
100	0.00	100.00	208.64	429.28	99.99	0.99
200	0.00	100.00	399.03	450.45	99.99	0.88
300	0.00	100.00	589.73	456.24	99.97	0.71
400	0.00	100.00	773.90	463.25	99.96	0.63
500	0.00	100.00	956.75	470.09	99.97	0.58
Average	0.00	100.00	585.61	453.86	99.98	0.76

Tabel 4. 38 Resource Pod pada Skenario Kombinasi WordPress

Users	Replicas	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
100	1	2.23	267.26	1336.42	81.70	100.00
200	1	2.18	267.50	1389.45	84.22	100.00

300	1	2.22	267.50	1414.17	86.32	99.20
400	1	2.22	267.50	1393.74	89.12	99.20
500	1	2.22	267.50	1398.80	91.11	88.00
Average	1	2.21	267.45	1386.52	86.49	97.28

Tabel 4. 39 Utilisasi Kernel Host Proxmox Kombinasi WordPress

Users	Avg CPU (%)	CPU IOWait	Load Avg	Avg Memory (MB)	Avg I/O (S)	Context Switches
100	91.67	0.31	4.11	4319.25	1076.46	26311
200	93.70	0.21	7.33	4315.50	822.07	26746
300	95.02	0.14	8.62	4324.89	497.26	27050
400	94.02	0.16	9.91	4340.81	619.98	26680
500	93.89	0.16	10.76	4330.41	472.49	26845
Average	93.66	0.20	8.15	4326.17	697.65	26726

4.6.3.2 Performa Kombinasi Ghost

Karakteristik performa, alokasi pod, dan kondisi kernel *host* untuk platform Ghost dalam kondisi mitigasi terintegrasi dirangkum dalam Tabel 4.40, Tabel 4.41, dan Tabel 4.42.

Tabel 4. 40 Performa Kombinasi Ghost

Users	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	WAF Active (%)	APDEX
100	0.00	100.00	185.36	484.96	99.99	1.00
200	0.00	100.00	356.45	504.53	99.99	0.91
300	0.00	100.00	508.86	529.99	99.99	0.77
400	0.00	100.00	674.41	532.68	99.98	0.67
500	0.00	100.00	894.74	502.28	99.97	0.60
Average	0.00	100.00	523.96	510.89	99.98	0.79

Tabel 4. 41 Resource Pod pada Skenario Kombinasi Ghost

Users	Replicas	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
100	1	4.72	274.40	1345.61	80.37	99.20
200	1	4.08	276.26	1427.60	83.61	99.20
300	1	3.75	276.75	1441.20	85.66	97.60
400	1	4.16	275.92	1438.13	88.35	88.00
500	1	4.00	276.77	1385.30	90.50	100.00
Average	1	4.14	276.02	1407.57	85.70	96.80

Tabel 4. 42 Utilisasi Kernel Host Proxmox Kombinasi Ghost

Users	Avg CPU (%)	CPU IOWait	Load Avg	Avg Memory (MB)	Avg I/O (S)	Context Switches
100	92.86	0.30	3.94	4295.29	386.26	27496
200	93.14	0.26	6.24	4300.01	925.24	27893
300	92.54	0.23	6.42	4299.56	633.93	28465
400	95.30	0.18	8.76	4286.07	963.61	28784
500	92.52	0.18	7.89	4305.59	952.40	27169
Average	93.27	0.23	6.65	4297.30	772.29	27961

4.6.3.3 Performa Kombinasi Drupal

Karakteristik performa, alokasi internal kontainer, dan parameter kernel operasi *host* Proxmox pada lingkungan Drupal dirangkum dalam Tabel 4.43, Tabel 4.44, dan Tabel 4.45.

Tabel 4. 43 Performa Kombinasi Drupal

Users	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	WAF Active (%)	APDEX
100	0.00	100.00	221.62	323.72	99.99	0.98
200	0.00	100.00	448.35	401.07	99.99	0.82
300	0.00	100.00	652.08	413.86	99.98	0.69
400	0.00	100.00	810.97	441.93	99.95	0.63

500	0.00	100.00	948.95	474.28	99.97	0.60
Average	0.00	100.00	616.39	410.97	99.98	0.74

Tabel 4. 44 Resource Pod pada Skenario Kombinasi Drupal

Users	Replicas	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
100	1	19.55	361.03	1237.73	76.80	93.84
200	1	20.73	366.06	1358.45	79.99	99.20
300	1	21.10	367.10	1381.88	82.32	100.00
400	1	21.96	367.88	1272.63	84.26	95.20
500	1	23.20	368.08	1195.64	86.33	90.40
Average	1	21.31	366.03	1289.27	81.94	95.73

Tabel 4. 45 Utilisasi Kernel Host Proxmox Kombinasi Drupal

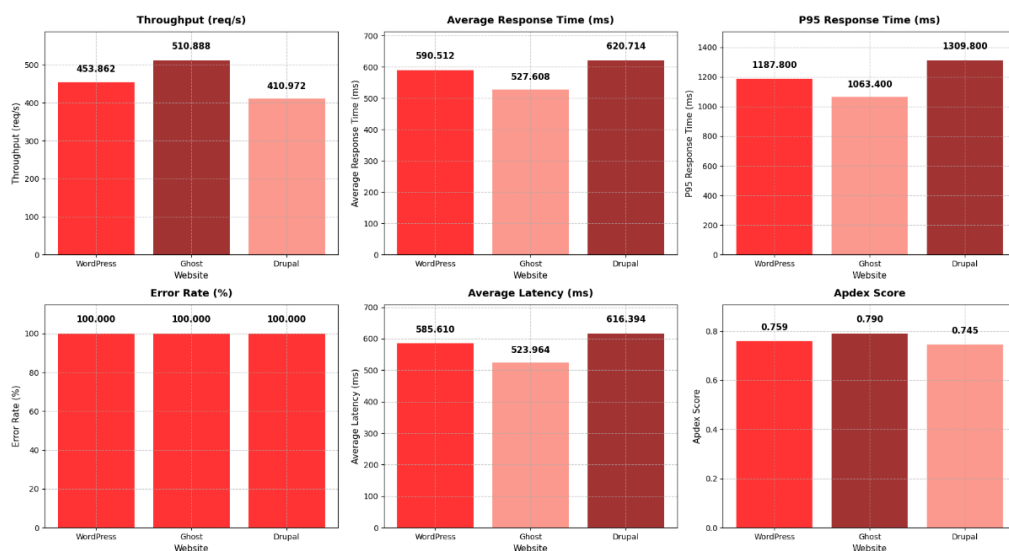
Users	Avg CPU (%)	CPU IOWait	Load Avg	Avg Memory (MB)	Avg I/O (S)	Context Switches
100	85.61	0.52	4.22	4226.21	1608.88	24714
200	94.39	0.26	8.21	4201.04	1141.86	25514
300	93.58	0.36	9.75	4203.41	1660.87	26162
400	88.70	0.83	10.31	4220.90	1411.79	26061
500	85.43	0.31	10.16	4206.31	1799.67	26186
Average	89.54	0.46	8.53	4211.57	1524.61	25727

4.6.3.4 Hasil Komparasi Kombinasi Sistem

Untuk mengevaluasi efisiensi relatif antar website di bawah implementasi arsitektur pertahanan ganda, Tabel 4.46 menyajikan rekapitulasi performa pada level aplikasi.

Tabel 4. 46 Hasil Komparasi Performa Skenario Kombinasi

Website	Avg Response (ms)	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	WAF Active (%)	APDEX
WordPress	590.512	585.610	100.000	453.862	99.976	0.759
Ghost	527.608	523.964	100.000	510.888	99.984	0.790
Drupal	620.714	616.394	100.000	410.972	99.976	0.745



Gambar 4. 44 Grafik Komparasi Kombinasi Sistem

Data komparatif pada Tabel 4.46 memperlihatkan bahwa aktivasi pertahanan ganda menghasilkan pola performa yang sangat seragam di lapisan aplikasi. Seluruh CMS mencatatkan tingkat kegagalan (*Error Rate*) mutlak sebesar 100,00%. Seperti halnya pada skenario *WAF Only*, kondisi ini bukan indikasi kegagalan *crash* pada server aplikasi, melainkan merupakan representasi keberhasilan ModSecurity dalam mencegah paket data serangan dan langsung mengembalikan kode status HTTP 403 Forbidden kepada pengguna virtual penyerang (*WAF Active* mencapai rata-rata 99,98%).

Integrasi *caching* memberikan kontribusi yang sangat signifikan dalam mempertahankan kecepatan respons sistem. Waktu respons rata-rata (*Avg Response Time*) dapat ditekan dengan stabil di kisaran 520 ms hingga 620 ms untuk seluruh

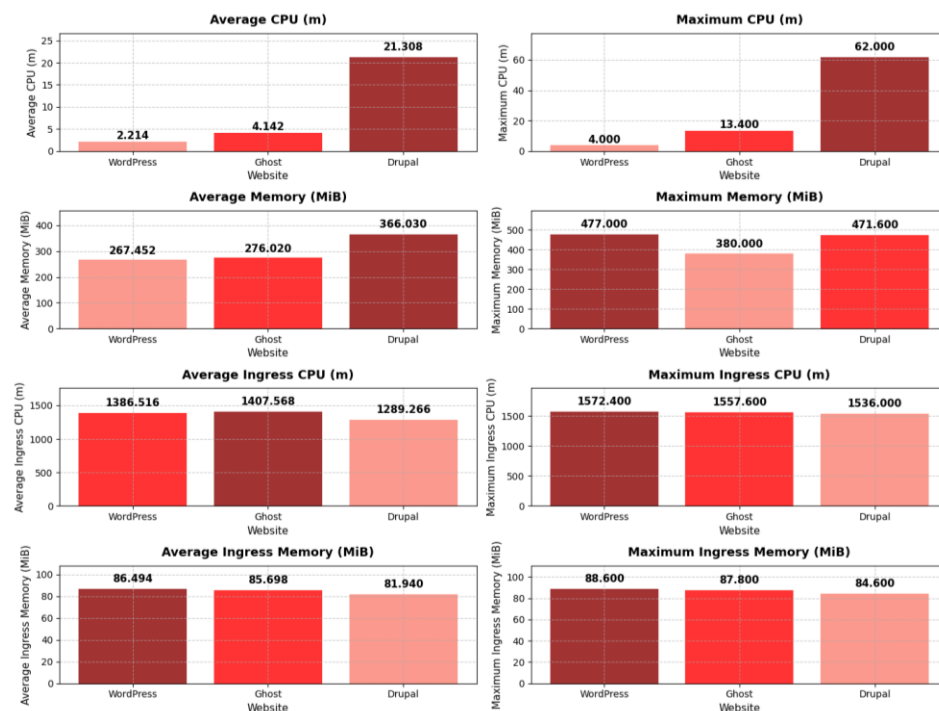
CMS. Ghost mencatatkan performa terbaik dengan waktu tunggu terendah sebesar 527,61 ms dan *throughput* tertinggi mencapai 510,89 req/s, disusul oleh WordPress (453,86 req/s) dan Drupal (410,97 req/s). Kecepatan penanganan pemutusan koneksi ini secara dramatis mendongkrak nilai indeks kepuasan pengguna (APDEX) ke tingkat menengah atas (rata-rata 0,74–0,79) jika dibandingkan dengan kondisi tanpa mitigasi.

4.6.3.5 Hasil Komparasi Alokasi Resource Kombinasi Sistem

Pengukuran alokasi daya komputasi internal pada lingkungan kontainer kluster dirangkum dalam Tabel 4.47 untuk menganalisis tingkat perlindungan terhadap *backend*.

Tabel 4. 47 Hasil Komparasi Resource Pod Skenario Kombinasi

Website	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
WordPress	2.214	267.452	1386.516	86.494	97.280
Ghost	4.142	276.020	1407.568	85.698	96.800
Drupal	21.308	366.030	1289.266	81.940	95.728



Gambar 4. 45 Grafik Komparasi Resource Kombinasi Sistem

Analisis pada Tabel 4.47 memperlihatkan efektivitas kolaborasi arsitektur dua lapis dalam mengamankan unit pemrosesan *backend*. Berkat kombinasi ini, *Application Pods* murni terisolasi dari beban komputasi luar. Rata-rata penggunaan CPU kontainer WordPress hanya memerlukan daya sebesar 2,21 m dan Drupal sebesar 21,31 m. Angka ini sangat identik dengan kondisi sistem saat *Idle*, yang menandakan tidak ada kalkulasi PHP/proses dinamis yang sampai ke kontainer WordPress akibat sebagian besar trafik diputus langsung di level Ingress (ModSecurity Blocked rata-rata >95%).

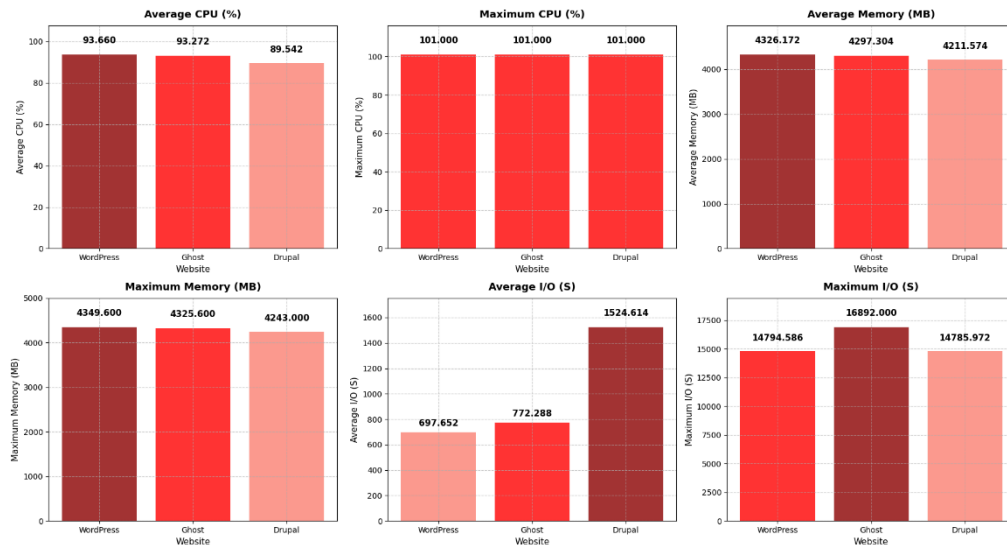
Kompensasi dari perlindungan ketat ini menyebabkan beban kerja terpusat sepenuhnya pada NGINX Ingress Controller yang menjalankan mesin *filter* WAF sekaligus manajemen *cache*. Data Tabel 4.47 mencatatkan konsumsi rata-rata CPU Ingress yang sangat tinggi, di mana lingkungan WordPress menghabiskan daya sebesar 1.386,516 m dan lingkungan Ghost menyentuh angka 1.407,568 m. Hal ini menunjukkan bahwa beban kerja penguraian paket data (*deep packet parsing*) yang dilakukan ModSecurity tetap menuntut kalkulasi prosesor yang intensif di level pintu gerbang.

4.6.3.6 Hasil Komparasi Utilisasi Resource Kombinasi Sistem

Analisis menyeluruh pada tingkat sistem operasi *hypervisor* fisik Proxmox dirangkum dalam Tabel 4.48 untuk memetakan dampak gabungan terhadap perangkat keras.

Tabel 4. 48 Hasil Komparasi Kernel Host Skenario Kombinasi

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	93.660	101.000	4326.172	4349.600	697.652	14794.586	26726.122
Ghost	93.272	101.000	4297.304	4325.600	772.288	16892.000	27961.406
Drupal	89.542	101.000	4211.574	4243.000	1524.614	14785.972	25727.350



Gambar 4. 46 Grafik Komparasi Kernel Kombinasi Sistem

Evaluasi mendalam pada level kernel *host* melalui data Tabel 4.48 memberikan jawaban konklusif mengenai limitasi infrastruktur virtual. Meskipun kombinasi WAF dan *caching* berhasil menyelamatkan *Application Pods* dari kelumpuhan memori dan mempertahankan kestabilan latensi aplikasi di angka 500 ms, utilisasi rata-rata CPU pada *hypervisor* Proxmox tetap terdorong hingga menyentuh batas kritis atas, yaitu 93,66% pada WordPress dan 93,27% pada Ghost, dengan puncaknya mencapai *Max CPU* 101,00%.

Kondisi beban kerja yang tinggi ini disebabkan oleh dua faktor utama: tingginya nilai kalkulasi penguraian aturan (*rule parsing*) pada ModSecurity serta tingginya intensitas pergantian tugas prosesor fisik yang ditunjukkan oleh nilai *Context Switches* yang menembus kisaran angka 25.000 hingga 27.000 pengalihan proses secara kontinu.

Meskipun demikian, terdapat perbedaan mendasar dibandingkan dengan kondisi *Cache Inject Query* (tanpa WAF): pada skenario kombinasi ini, nilai rata-rata *Load Average* dan antrian instruksi masukan lebih terkendali karena *Nginx Caching* memotong jalur pembacaan *disk I/O* secara drastis, sehingga mencegah terjadinya penumpukan proses (*process starvation*) pada kernel. Dengan demikian, penggabungan WAF ModSecurity dan *Nginx Caching* terbukti sebagai solusi

arsitektur terbaik untuk mempertahankan ketersediaan (*availability*) layanan web pada lingkungan klaster Kubernetes, meskipun dijalankan di atas infrastruktur perangkat keras dengan spesifikasi yang sangat terbatas.

4.7 Hasil Pengujian Stress Testing Sistem

Pengujian tahap akhir ini dilakukan dengan meningkatkan beban secara agresif mulai dari 600 hingga 1.000 pengguna virtual (*concurrent users*) secara simultan. Pengujian *Stress Testing* ini dirancang untuk menguji batas atas arsitektur pertahanan kombinasi (*Web Application Firewall* dan *Nginx Caching*) dalam mempertahankan ketersediaan layanan (*availability*) ketika infrastruktur fisik mengalami saturasi penuh.

4.7.1 Performa Stress Testing WordPress

Karakteristik performa aplikasi, konsumsi sumber daya *Pod*, dan metrik kernel *host* Proxmox untuk lingkungan WordPress dirangkum dalam Tabel 4.49, Tabel 4.50, dan Tabel 4.51.

Tabel 4. 49 Performa Stress Testing WordPress

Users	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	WAF Active (%)	APDEX
600	0.00	100.00	1204.00	449.48	99.97	0.52
700	0.00	100.00	1368.36	460.78	99.74	0.47
800	0.00	100.00	1539.43	469.59	99.99	0.43
900	0.00	100.00	1743.13	467.27	99.99	0.38
1000	0.00	100.00	1935.92	467.05	99.96	0.34
Average	0.00	100.00	1558.17	462.83	99.93	0.43

Tabel 4. 50 Resource Pod pada Skenario Stress Testing WordPress

Users	Replicas	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
600	1	2.20	267.50	1303.87	92.64	84.80
700	1	2.20	267.50	1272.46	96.60	100.00
800	1	2.25	267.50	1209.75	95.16	100.00

900	1	2.23	267.50	1141.91	96.84	100.00
1000	1	2.34	267.50	1102.06	98.08	100.00
Average	1	2.24	267.50	1206.01	95.86	96.96

Tabel 4. 51 Utilisasi Kernel Host Proxmox Stress Testing WordPress

Users	Avg CPU (%)	CPU IOWait	Load Avg	Avg Memory (MB)	Avg I/O (S)	Context Switches
600	91.67	0.18	9.88	4325.14	869.38	26872
700	88.47	0.38	12.41	4320.85	378.68	26397
800	84.56	0.40	10.59	4326.94	620.05	26401
900	81.98	0.30	11.36	4323.78	1040.32	24487
1000	79.82	0.39	9.90	4310.55	1105.93	25438
Average	85.30	0.33	10.83	4321.45	802.87	25919

4.7.2 Performa Stress Testing Ghost

Karakteristik performa, alokasi pod kontainer, dan parameter operasional kernel *host* Proxmox pada lingkungan Ghost dirangkum dalam Tabel 4.52, Tabel 4.53, dan Tabel 4.54.

Tabel 4. 52 Performa Stress Testing Ghost

Users	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	WAF Active (%)	APDEX
600	0.00	100.00	1027.57	527.51	99.98	0.56
700	0.00	100.00	1233.58	513.40	99.99	0.51
800	0.00	100.00	1348.91	536.73	99.99	0.47
900	0.00	100.00	1583.28	514.46	99.99	0.41
1000	0.00	100.00	1757.00	514.62	99.99	0.37
Average	0.00	100.00	1390.07	521.34	99.99	0.46

Tabel 4. 53 Resource Pod pada Skenario Stress Testing Ghost

Users	Replicas	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
600	1	3.86	277.30	1387.97	92.02	100.00
700	1	3.92	277.50	1297.63	93.44	99.20
800	1	3.83	277.75	1292.65	95.22	97.60
900	1	3.68	277.98	1188.10	95.97	99.20
1000	1	4.64	276.40	1133.94	96.80	99.20
Average	1	3.99	277.39	1260.06	94.69	99.04

Tabel 4. 54 Utilisasi Kernel Host Proxmox Stress Testing Ghost

Users	Avg CPU (%)	CPU IOWait	Load Avg	Avg Memory (MB)	Avg I/O (S)	Context Switches
600	93.59	0.18	9.44	4315.04	733.27	27600
700	93.39	0.19	9.92	4291.85	904.33	28248
800	91.46	0.18	9.15	4280.82	943.18	27881
900	87.49	0.22	11.28	4271.98	605.43	26723
1000	83.21	0.43	12.40	4281.88	907.26	26134
Average	89.83	0.24	10.44	4288.31	818.69	27317

4.7.3 Performa Stress Testing Drupal

Karakteristik performa, alokasi pod kontainer, dan metrik kernel operasi *host* Proxmox pada lingkungan Drupal dirangkum dalam Tabel 4.55, Tabel 4.56, dan Tabel 4.57.

Tabel 4. 55 Performa Stress Testing Drupal

Users	Success Rate (%)	Error Rate (%)	Avg Latency (ms)	Throughput (req/s)	WAF Active (%)	APDEX
600	0.00	100.00	1251.06	431.54	99.95	0.53
700	0.00	100.00	1216.99	520.08	99.98	0.52
800	0.00	100.00	1481.44	488.29	99.74	0.45
900	0.00	100.00	1773.69	458.16	99.95	0.38

1000	0.00	100.00	1917.51	468.64	99.91	0.34
Average	0.00	100.00	1528.14	473.34	99.91	0.44

Tabel 4. 56 Resource Pod pada Skenario Stress Testing Drupal

Users	Replicas	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
600	1	22.77	370.96	1083.01	87.90	66.40
700	1	22.37	371.34	1024.47	89.13	84.00
800	1	22.49	371.12	992.07	92.02	88.80
900	1	21.23	373.02	935.86	90.66	93.60
1000	1	20.58	373.20	872.27	92.34	92.00
Average	1	21.89	371.93	981.54	90.41	84.96

Tabel 4. 57 Utilisasi Kernel Host Proxmox Stress Testing Drupal

Users	Avg CPU (%)	CPU IOWait	Load Avg	Avg Memory (MB)	Avg I/O (S)	Context Switches
600	79.74	0.74	10.06	4180.91	1129.12	25076
700	76.11	0.67	10.50	4172.79	1411.27	24898
800	72.52	0.96	10.80	4164.88	1342.79	24871
900	69.40	0.94	9.28	4128.78	1386.12	22507
1000	70.51	2.20	9.85	4120.52	1378.68	24426
Average	73.66	1.10	10.10	4153.58	1329.60	24355

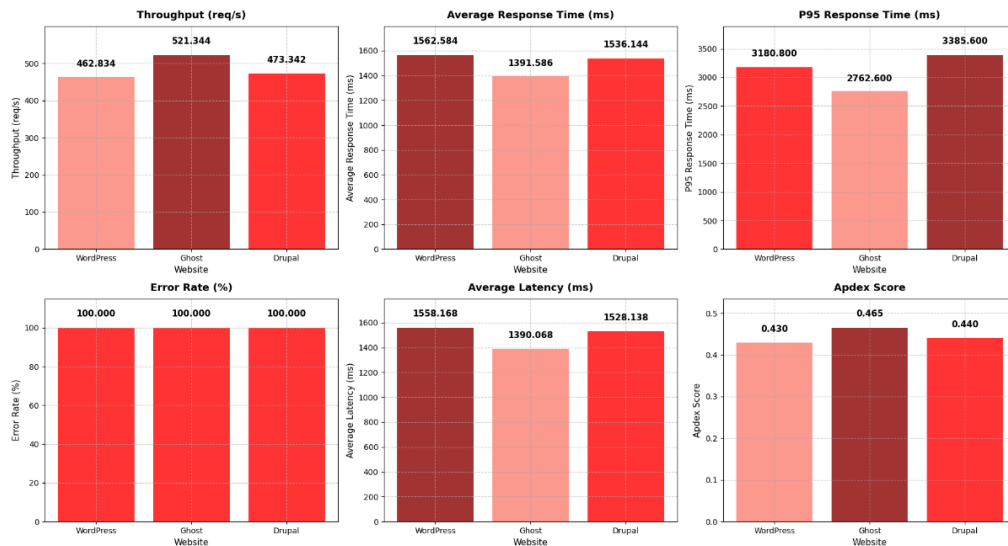
4.7.4 Hasil Komparasi Stress Testing Sistem

Untuk mengevaluasi ketahanan batas atas antar website di bawah tekanan beban banjir trafik ekstrem, Tabel 4.58 menyajikan rekapitulasi performa rata-rata pada lapisan aplikasi.

Tabel 4. 58 Hasil Komparasi Performa Stress Testing Sistem

Website	Avg Response (ms)	Avg Latency (ms)	Error Rate (%)	Throughput (req/s)	WAF Active (%)	APDEX
WordPress	1562.584	1558.168	100.000	462.834	99.930	0.430

Ghost	1391.586	1390.068	100.000	521.344	99.988	0.465
Drupal	1536.144	1528.138	100.000	473.342	99.906	0.440



Gambar 4. 47 Grafik Komparasi Stress Testing Sistem

Berdasarkan komparasi data pada Tabel 4.58, peningkatan beban kerja ekstrim hingga 1.000 *users* memicu terjadinya penurunan kualitas pengalaman pengguna secara menyeluruh. Meskipun angka *Error Rate* bertahan mutlak di angka 100,00% sebagai tanda keberhasilan intersepsi aturan keamanan WAF (WAF Active rata-rata >99,90% pada kode respons HTTP 403), nilai indeks kepuasan pengguna (APDEX) merosot ke kisaran 0,43–0,46 yang mengindikasikan status tidak memuaskan (*Frustrated*). Kenaikan intensitas beban dari 600 ke 1.000 *users* secara linier mendorong rata-rata latensi respons meningkat di atas 1,3 detik untuk ketiga website akibat terjadinya antrean pemrosesan TCP yang padat pada Ingress pintu gerbang klaster.

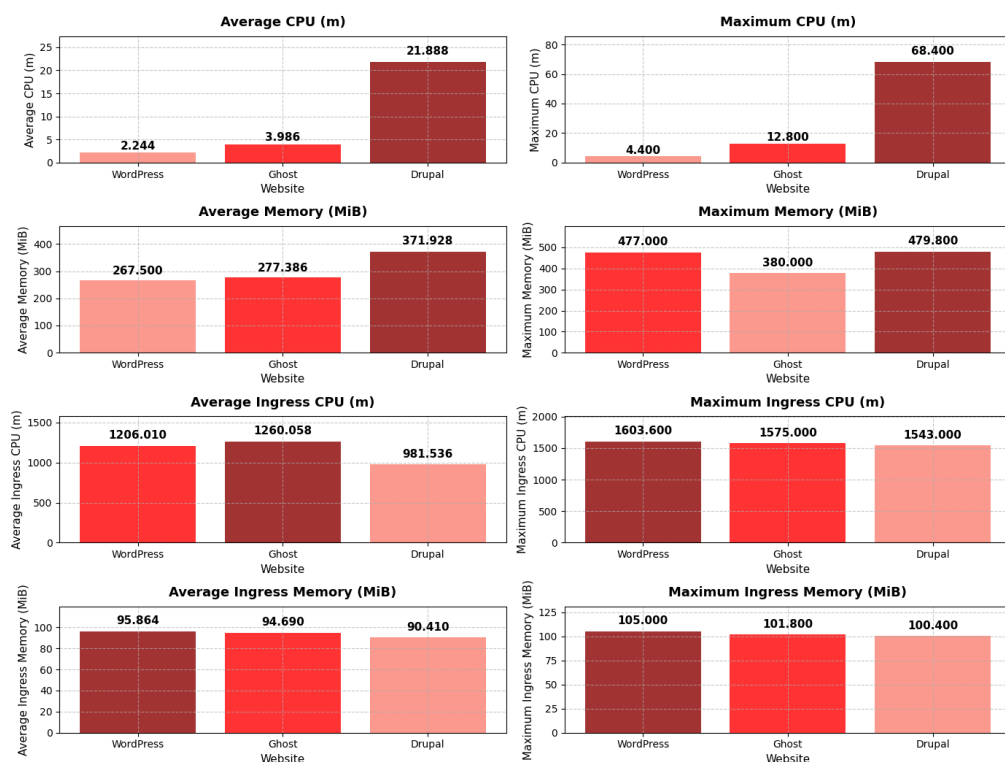
Meskipun demikian, Ghost tetap mendominasi efisiensi kecepatan dengan latensi terendah sebesar 1.391,59 ms dan pasokan *throughput* tertinggi yang menembus angka 521,34 req/s. Keberhasilan mempertahankan pengiriman *throughput* tinggi ini membuktikan ketangguhan arsitektur *runtime* berbasis *event-driven* pada Node.js dalam mengelola siklus konkurensi tinggi.

4.7.5 Hasil Komparasi Alokasi Resource Stress Test Sistem

Pengukuran alokasi daya komputasi internal tingkat kontainer dirangkum dalam Tabel 4.59 untuk menganalisis tingkat efisiensi perlindungan komponen inti aplikasi.

Tabel 4. 59 Hasil Komparasi Resource Pod Stress Testing Sistem

Website	Avg CPU (m)	Avg Memory (Mi)	Avg Ingress CPU (m)	Avg Ingress Memory (Mi)	ModSecurity Blocked (%)
WordPress	2.244	267.500	1206.010	95.864	96.960
Ghost	3.986	277.386	1260.058	94.690	99.040
Drupal	21.888	371.928	981.536	90.410	84.960



Gambar 4. 48 Grafik Komparasi Resource Stress Testing Sistem

Evaluasi data pada Tabel 4.59 memperlihatkan pembuktian krusial mengenai keberhasilan fungsi isolasi beban kerja. Walaupun volume trafik eksternal dilipatgandakan hingga menyentuh 1.000 *users*, utilisasi daya CPU kontainer *backend* tetap berada pada kondisi aman di bawah 22 m untuk WordPress dan

Ghost. Hal ini membuktikan bahwa integrasi *caching* dan penyaringan paket WAF berhasil memotong jalur instruksi di level depan, sehingga *Application Pods* murni terbebas dari ancaman kelumpuhan komputasi.

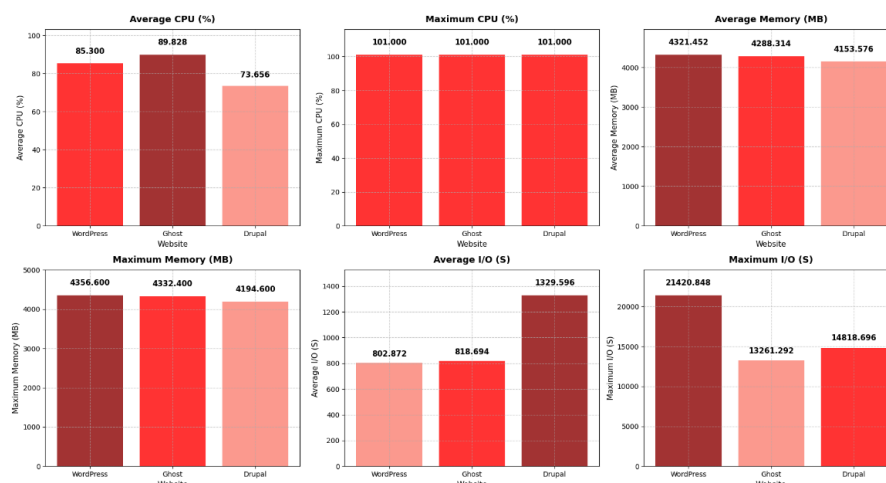
Beban kalkulasi terbesar tertahan sepenuhnya di level Ingress Controller, dengan rata-rata konsumsi CPU Ingress mencapai 1.206,01 m pada WordPress dan 1.260,06 m pada Ghost. Penurunan alokasi daya komputasi ini mengindikasikan bahwa *Ingress Controller* telah mencapai ambang batas saturasi alokasi CPU fisik yang tersedia pada *node worker* akibat limitasi perangkat keras *host* (2 Core CPU), sehingga memicu kondisi *resource starvation* yang menurunkan efisiensi pemblokiran murni ModSecurity hingga ke angka rata-rata 84,960% pada lingkungan Drupal.

4.7.6 Hasil Komparasi Utilisasi Resource Stress Test Sistem

Analisis mendalam pada lapisan *hypervisor* fisik Proxmox disajikan pada Tabel 4.60 untuk mengevaluasi batas atas daya tahan infrastruktur fisik.

Tabel 4. 60 Hasil Komparasi Kernel Host Stress Testing Sistem

Website	Avg CPU (%)	Max CPU (%)	Avg Memory (MB)	Max Memory (MB)	Avg I/O (S)	Max I/O (S)	Context Switches
WordPress	85.300	101.000	4321.452	4356.600	802.872	21420.848	25919.026
Ghost	89.828	101.000	4288.314	4332.400	818.694	13261.292	27317.156
Drupal	73.656	101.000	4153.576	4194.600	1329.596	14818.696	24355.430



Gambar 4. 49 Grafik Komparasi Kernel Stress Testing Sistem

Analisis komprehensif pada level kernel *host* melalui data Tabel 4.60 menyingkap rahasia ketahanan infrastruktur fisik *hypervisor*. Di bawah gempuran 1.000 *users*, utilisasi rata-rata CPU *host* Proxmox bertahan konstan di kisaran 73,66%–89,83% dengan nilai puncaknya menyentuh batasan kritis 101,00% (*Max CPU*). Tingginya nilai *Context Switches* yang menembus kisaran 24.000 hingga 27.000 pengalihan proses mempertegas bahwa kernel Linux sedang beroperasi dalam kondisi saturasi penjadwalan (*scheduler saturation*) yang intensif untuk memproses paket data WAF.

Meskipun CPU mengalami saturasi penuh, kestabilan konsumsi RAM rata-rata tetap terjaga aman di kisaran 4,1 GB hingga 4,3 GB dari total kapasitas fisik sebesar 8 GB. Hal ini dikarenakan efisiensi *Nginx Caching* berhasil meredam lonjakan penggunaan alokasi RAM baru pada tingkat kontainer.

Korelasi linier juga terlihat pada nilai aktivitas pemrosesan penyimpanan (*Avg I/O*) Drupal yang mencatatkan angka tertinggi sebesar 1.329,60 S dengan *Max I/O* menembus 14.818,70 S. Kondisi ini menunjukkan tingginya aktivitas transaksi pembacaan data berkas aturan WAF pada disk. Melalui temuan pengujian *Stress Testing* ini, dapat disimpulkan bahwa kombinasi ModSecurity WAF dan *Nginx Caching* terbukti kokoh menjaga ketersediaan layanan dari ancaman kelumpuhan total (*downtime/server crash*), meskipun infrastruktur komputasi fisik dipaksa beroperasi di atas titik jenuh (*saturation point*) perangkat keras.

4.8 Perbandingan Skenario Pengujian

Berdasarkan keseluruhan skenario pengujian yang telah dieksekusi, dokumentasi teknis pendukung yang mencakup keseluruhan *source code* skrip otomatisasi dan rancangan *test plan* sistem telah dihimpun dan disertakan pada bagian Lampiran. Oleh karena itu, pembahasan pada subbab ini dikhususkan untuk membedah rekapitulasi nilai rata-rata (*average*) dari kelima kondisi utama sistem guna menentukan arsitektur mitigasi yang paling efektif.

Untuk mengevaluasi performa dan efektivitas masing-masing mekanisme pertahanan, dilakukan analisis komparatif secara makro terhadap lima kondisi

sistem utama. Tabel perbandingan ini merangkum metrik performa aplikasi, utilitas *pod* kontainer, hingga beban *kernel host* pada masing-masing Website.

Tabel 4. 61 Perbandingan Pengujian WordPress

Parameter	Baseline	Base Cache	Payload ModSecurity	Combination	Stress Test
Avg Response (ms)	2.352,40	4.698,41	549,73	590,51	1.562,58
Throughput (req/s)	22,67	57,20	475,74	453,86	462,83
Error Rate (%)	0,05	0,02	100,00	100,00	100,00
WAF Blocked (%)	-	-	97,27	97,28	96,96
CPU Pod Backend (m)	266,69	2,16	2,39	2,21	2,24
CPU Ingress (m)	-	34,07	1.270,45	1.386,52	1.206,01
CPU Host Proxmox (%)	80,70	14,46	82,06	93,66	85,30
Context Switches	23.254	16.277	25.359	26.726	25.919

Tabel 4. 62 Perbandingan Pengujian Ghost

Parameter	Baseline	Base Cache	Payload ModSecurity	Combination	Stress Test
Avg Response (ms)	701,33	1.681,33	520,58	527,61	1.391,59
Throughput (req/s)	77,54	159,93	515,64	510,89	521,34
Error Rate (%)	0,02	0,00	100,00	100,00	100,00
WAF Blocked (%)	-	-	99,52	96,80	99,04
CPU Pod Backend (m)	282,56	3,98	3,96	4,14	3,99
CPU Ingress (m)	-	49,55	1.405,06	1.407,57	1.260,06
CPU Host Proxmox (%)	77,16	16,21	91,41	93,27	89,83
Context Switches	19.821	18.157	27.554	27.961	27.317

Tabel 4. 63 Perbandingan Pengujian Drupal

Parameter	Baseline	Base Cache	Payload ModSecurity	Combination	Stress Test
Avg Response (ms)	349,51	1.600,69	631,53	620,71	1.536,14
Throughput (req/s)	153,95	167,75	428,33	410,97	473,34
Error Rate (%)	4,61	0,00	100,00	100,00	100,00
WAF Blocked (%)	-	-	94,96	95,73	84,96
CPU Pod Backend (m)	223,45	16,19	20,73	21,31	21,89
CPU Ingress (m)	-	54,08	1.268,46	1.289,27	981,54
CPU Host Proxmox (%)	56,33	18,29	82,50	89,54	73,66
Context Switches	24.683	18.897	24.904	25.727	24.355

Berdasarkan sajian data komparatif pada Tabel 4.61 hingga Tabel 4.63, terlihat secara jelas pola korelasi sebab-akibat lintas arsitektur pada setiap tahapan pengujian. Pada kondisi standar (*Baseline*) tanpa mekanisme mitigasi, lonjakan beban secara langsung menyebabkan aplikasi kehilangan kemampuan pemrosesan. Hal ini paling terlihat pada lingkungan WordPress, di mana pasokan *throughput* tertahan di angka 22,67 req/s dan waktu respons melonjak tajam menembus 2.352,40 ms. Beban kalkulasi mentah ini berdampak langsung pada kelelahan CPU Pod *Backend* yang dipaksa bekerja hingga 266,69 m, sekaligus mendorong utilitas CPU *Host* fisik mencapai angka 80,70%.

Penerapan mekanisme mitigasi tunggal terbukti belum cukup memberikan perlindungan yang komprehensif. Penggunaan *Nginx Caching* secara mandiri (*Base Cache*) memang sukses besar menurunkan beban CPU *Host* hingga di bawah 20% untuk seluruh Content Manajement System serta menekan kerja Pod *Backend* mendekati nol (< 17 m). Namun, arsitektur ini memiliki celah karena tidak memberikan penyaringan keamanan (*filtering*) terhadap lalu lintas data berbahaya. Di sisi lain, penggunaan WAF ModSecurity murni (*Payload ModSecurity*) menjamin pemblokiran paket serangan dengan tingkat keberhasilan secara konsisten mencapai 95,00%. Akan tetapi, biaya inspeksi paket mendalam (*deep*

packet inspection) ini sangat membebani Ingress Controller, memaksa CPU *Ingress* melonjak tajam hingga melampaui 1.200 m dan mendorong utilisasi CPU *Host* kembali ke atas batas 82%.

Optimalisasi infrastruktur virtual pada akhirnya tercapai secara penuh melalui skenario Kombinasi (WAF dan *Caching*). Penggabungan kedua modul ini menghasilkan arsitektur pertahanan yang komplementer dan tangguh. Pada beban 500 pengguna virtual, waktu respons berhasil distabilkan ke angka 590,51 ms untuk WordPress dan 527,61 ms untuk Ghost, diiringi oleh pasokan *throughput* yang tinggi. Pada skenario ini, *Application Pod Backend* terisolasi secara mutlak dari tekanan komputasi kompromistis (selalu di bawah 22 m) karena trafik serangan langsung diputus di level Ingress dengan kode HTTP 403, yang direpresentasikan oleh *Error Rate* sebesar 100%.

Kendati demikian, ketahanan arsitektur perangkat lunak ini tetap memiliki batasan ketika dihadapkan pada limitasi perangkat keras. Pada tahap pengujian ekstrem (*Stress Test*) dengan pelipatgandaan beban hingga 1.000 pengguna virtual, arsitektur keamanan tetap berdiri kokoh melindungi *backend*, namun kualitas kecepatan respons mulai terdegradasi. Waktu respons secara merata melambat ke kisaran 1.300 ms hingga 1.500 ms. Tingginya nilai *Context Switches* yang menembus angka 25.000 hingga 27.000 pengalihan proses mengonfirmasi bahwa CPU *Host Proxmox* telah memasuki masa jenuh penjadwalan (*scheduler saturation*) demi mengimbangi beratnya kinerja mesin ModSecurity di pintu gerbang klaster.

4.9 Analisis Hasil Pengujian Secara Keseluruhan

Secara keseluruhan, rangkaian eksperimen simulasi beban *Idle* hingga *Stress Testing* serangan *HTTP(S) Flood* pada klaster Kubernetes menyingkap analisis fundamental mengenai ketahanan infrastruktur dan karakteristik teknologi aplikasi. Hasil uji membuktikan bahwa arsitektur *backend* sangat menentukan ketahanan dasar sebuah aplikasi web. Platform Ghost yang dibangun di atas ekosistem Node.js mendominasi efisiensi konkurensi melalui mekanisme *non-blocking I/O*, sehingga mampu memasok *throughput* tertinggi saat menerima gempuran serangan. Berbeda

dengan Ghost, lingkungan Drupal menonjolkan manajemen prosesor yang paling hemat daya berkat struktur *caching* internal yang agresif, meskipun pendekatan ini harus dikompensasi dengan alokasi memori dasar (RAM) yang jauh lebih tinggi. Sementara itu, WordPress yang mewakili tumpukan teknologi PHP tradisional terbukti menjadi aplikasi yang paling menguras komputasi CPU dan paling rentan mengalami kelumpuhan latensi apabila dipaksa beroperasi tanpa lapisan perlindungan eksternal.

Ketahanan perangkat lunak tersebut pada akhirnya dihadapkan pada realitas keterbatasan infrastruktur fisik. Pemantauan metrik pada *hypervisor* Proxmox mengungkap bahwa spesifikasi perangkat keras, khususnya alokasi prosesor 2 *Core* CPU, bertindak sebagai *bottleneck* utama dalam pengimplementasian sistem keamanan terpusat. Evaluasi membuktikan bahwa keberhasilan ModSecurity dalam menginspeksi lalu lintas data berbanding lurus dengan peningkatan tajam nilai *Context Switches* dan *Load Average* pada kernel *host*.

Ketika *Ingress Controller* dipaksa melakukan penguraian paket secara mendalam (*deep packet parsing*) terhadap ribuan permintaan simultan, CPU *host* mengalami saturasi maksimal hingga menyentuh batas 101,00%. Kondisi saturasi inilah yang memicu terjadinya fenomena *CPU throttling*, yang secara otomatis memperpanjang antrean waktu tunggu (*latency*) di sisi pengguna akhir. Meskipun penggunaan RAM fisik Proxmox masih menyisakan ruang aman dari total kapasitas 8 GB, ketidakmampuan prosesor fisik dalam mengimbangi kecepatan inspeksi paket *Deep Packet Inspection* (DPI) tetap menjadi faktor penentu utama degradasi performa.

Meskipun dihadapkan pada limitasi perangkat keras, penelitian ini secara meyakinkan memvalidasi bahwa penerapan strategi mitigasi tunggal memiliki kelemahan struktural yang fatal di mana mekanisme *caching* murni sangat rentan terhadap teknik *cache bypass*, sedangkan *Web Application Firewall* (WAF) mandiri akan membebani prosesor hingga melumpuhkan lalu lintas. Oleh karena itu, integrasi antara *Nginx Caching* dan WAF ModSecurity terbukti menjadi arsitektur pertahanan yang paling komplementer dan efektif. Dalam kolaborasi dua lapis (*dual-layer defense*) ini, ModSecurity bertugas sebagai penjaga gerbang yang

memastikan paket berbahaya tidak menembus klaster, sementara *Nginx Caching* memotong beban baca-tulis *disk* dan meredam beban kalkulasi pada interpreter aplikasi. Sinergi absolut ini terbukti sukses menjamin ketersediaan layanan (*availability*) dan menyelamatkan *Pod backend* klaster Kubernetes dari bahaya kegagalan sistem (*downtime*), sekalipun infrastruktur komputasi fisik dipaksa beroperasi jauh melampaui kapasitas idealnya.

BAB 5. KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan seluruh rangkaian perancangan, implementasi, dan pengujian beban ekstrem pada infrastruktur kluster Kubernetes, dapat disimpulkan bahwa penerapan arsitektur pertahanan ganda (*dual-layer defense*) merupakan solusi untuk menjaga ketersediaan layanan (*availability*). Penggabungan antara *Web Application Firewall* (WAF) ModSecurity dan *Nginx Caching* terbukti menciptakan efek sinergis yang sangat kuat. ModSecurity secara efektif menjadi penjaga gerbang yang memutus lalu lintas data berbahaya dengan tingkat pemblokiran di atas 95,00%, sementara *Nginx Caching* berperan krusial dalam memotong beban baca-tulis *disk* dan meredam antrean kalkulasi pada *backend*. Sinergi ini sukses mengisolasi *Application Pods* dari ancaman kelumpuhan sistem (*downtime*), sehingga konsumsi daya komputasi internal tetap stabil di bawah 25 *millicores* (m) meskipun sistem dihantam oleh 1.000 pengguna virtual secara simultan.

Dari hasil evaluasi mendalam, terdapat tiga temuan krusial yang menjadi titik kritis sekaligus landasan untuk melanjutkan pengembangan penelitian ini:

1. Pengujian ini membuktikan bahwa implementasi *caching* murni terbukti sangat rentan terhadap manipulasi *query string* yang memaksa sistem melakukan *cache bypass*, sehingga menyebabkan *Error Rate* aplikasi WordPress meroket hingga 50,89%. Sebaliknya, penerapan ModSecurity tanpa bantuan *cache* justru menciptakan efek *bottleneck* pada pintu gerbang Ingress akibat tingginya beban komputasi untuk menginspeksi ribuan paket data secara mendalam (*deep packet inspection*).
2. Ketahanan infrastruktur ini ternyata juga sangat dipengaruhi oleh teknologi *runtime* yang digunakan oleh masing-masing aplikasi. Platform Ghost yang dibangun di atas arsitektur *non-blocking I/O* (Node.js) menunjukkan tingkat efisiensi konkurensi tertinggi. Sementara itu, aplikasi berbasis PHP seperti Drupal dan WordPress terbukti lebih membebani system dimana Drupal mengurus alokasi memori RAM untuk *cache* internalnya, dan WordPress

menjadi aplikasi yang paling rentan mengalami kelumpuhan akibat tingginya kebergantungan pada komputasi prosesor aktif.

3. Di luar efektivitas perangkat lunak mitigasi, stabilitas kluster secara keseluruhan pada akhirnya sangat dibatasi oleh spesifikasi fisik perangkat keras. Pemantauan pada level *hypervisor* Proxmox mengonfirmasi bahwa alokasi 2 *Core* CPU menjadi titik jenuh (*saturation point*) yang membatasi kecepatan infrastruktur. Beban kalkulasi aturan ModSecurity yang masif memaksa utilisasi CPU menyentuh angka maksimal 101,00% dan memicu lonjakan *Context Switches* pada kernel. Fenomena *CPU throttling* inilah yang pada akhirnya menjadi penyebab utama terjadinya degradasi waktu respons dan penurunan indeks kepuasan pengguna (APDEX) di bawah tekanan beban yang ekstrem.

5.2 Saran

Penelitian ini telah berhasil memvalidasi efisiensi mitigasi keamanan berlapis pada infrastruktur virtual dengan sumber daya terbatas. Untuk menyempurnakan arsitektur dan melanjutkan pengembangan riset ini di masa mendatang, beberapa saran fundamental yang diajukan adalah:

1. Disarankan untuk meningkatkan alokasi *server* fisik dengan spesifikasi prosesor yang lebih tinggi (minimal 4 hingga 8 *cores*) guna mencegah terjadinya fenomena *CPU throttling* pada Ingress Controller, sehingga kapasitas ModSecurity dalam melakukan *parsing* aturan dapat dievaluasi secara maksimal.
2. Seiring dengan peningkatan perangkat keras, beban komputasi ModSecurity juga dapat direduksi melalui *rule tuning*, yaitu dengan menonaktifkan aturan pada *Core Rule Set* (CRS) yang tidak relevan dengan tumpukan teknologi (*tech stack*) aplikasi.
3. Selain itu, arsitektur berskala besar membutuhkan manajemen memori yang mandiri, sehingga disarankan untuk mengintegrasikan mekanisme *caching* eksternal terdistribusi seperti Redis atau Memcached guna memisahkan beban kerja penyimpanan dari Ingress.

4. Dari sisi metodologi pengujian, penelitian selanjutnya disarankan untuk memperluas skenario simulasi beban melampaui batas *Stress Testing*. Pengujian ketahanan jangka panjang (*Soak Testing*) perlu dilakukan untuk mengidentifikasi potensi kebocoran memori (*memory leak*) pada kontainer aplikasi saat menerima beban stabil dalam durasi yang sangat lama. Selain itu, pengujian lonjakan tiba-tiba (*Spike Testing*) sangat krusial untuk mengevaluasi seberapa cepat sistem *autoscaling* Kubernetes mampu merespons trafik ekstrem yang datang seketika.
5. Untuk mendukung kompleksitas pengujian, disarankan untuk mengimplementasikan *tools* otomatisasi pengujian modern (seperti K6 atau integrasi *pipeline CI/CD* berbasis skrip) guna menyederhanakan peluncuran skenario serangan, mereduksi bias manusia, serta mempercepat proses ekstraksi matriks log secara *real-time*.
6. Terakhir, sejalan dengan evolusi ancaman siber, pengujian lanjutan perlu mempertimbangkan variasi vektor serangan asimetris yang menargetkan kerentanan logika, seperti *Slowloris* atau *Application Logic Attacks*. Pendekatan mitigasi WAF ini akan menjadi jauh lebih tangguh dan komprehensif jika dikombinasikan dengan sistem deteksi anomali berbasis kecerdasan buatan (*Machine Learning*). Integrasi ini diharapkan mampu menekan angka *False Positive* secara signifikan, sekaligus meningkatkan akurasi sistem keamanan klaster dalam membedakan antara serangan peretasan yang sebenarnya dan lonjakan trafik pengguna sah yang terjadi.

DAFTAR PUSTAKA

- Aalam, Z., Kumar, V., & Gour, S. (2021). A review paper on hypervisor and virtual machine security. *Journal of Physics: Conference Series*, 1950(1). <https://doi.org/10.1088/1742-6596/1950/1/012027>
- Adam, G. K., Petrellis, N., & Doulos, L. T. (2021). Performance assessment of linux kernels with preempt_rt on arm-based embedded devices. *Electronics (Switzerland)*, 10(11). <https://doi.org/10.3390/electronics10111331>
- Aldiwidianto, W., & Prisma, G. L. E. (2023). Analisis Perbandingan High Availability Pada Web Server Menggunakan Orchestration Tool Kubernetes Dan Docker Swarm. *Journal of Informatics and Computer Science*, 05, 138–148.
- Aqasizade, H., Ataie, E., & Bastam, M. (2024). *Kubernetes in Action: Exploring the Performance of Kubernetes Distributions in the Cloud*. <http://arxiv.org/abs/2403.01429>
- Ardiansyah, Y., Sunandar, M. A., & Muhyidin, Y. (2023). IMPLEMENTASI KEAMANAN WEBSITE DENGAN METODE FIREWALL APLIKASI WEB (WAF) Studi kasus: WEB DESA WANTILAN. *Jurnal Mahasiswa Teknik Informatika*, 7(3), 2018–2025.
- Arman, M. (2020). Metode Pertahanan Web Server Terhadap Distributed Slow HTTP DoS Attack. *Jurnal Teknik Informatika dan Sistem Informasi*, 7(1), 56–70. <http://jurnal.mdp.ac.id>
- Ashari, I. F., Anugrah P, L. R., Andintya W, N., Denira, S. T., Sumatera, T., & Selatan, L. (t.t.). ANALISIS CELAH KEAMANAN DAN MITIGASI WEBSITE E-LEARNING ITERA MENGGUNAKAN OWASP ZED ATTACK PROXY (ZAP) VULNERABILITY AND MITIGATION ANALYSIS OF THE ITERA E-LEARNING WEBSITE USING OWASP ZED ATTACK PROXY (ZAP). *DINAMIKA REKAYASA*, 19, 29–35. Diambil <http://dinarek.unsoed.ac.id>

- Asosiasi Penyelenggara Jasa Internet Indonesia. (2026). *Survei Internet APJII 2026*. APJII. <https://survei.apjii.or.id/>
- Azi, M. N. A., Arifwidodo, B., & Wahyudi, E. (2023). Analisis Performansi Web Server Saat Menangani Permintaan Client Menggunakan Metode Reserve Proxy Caching dan Varnish. *Journal of Telecommunication, Electronics, and Control Engineering (JTECE)*, 5(1), 14–21. <https://doi.org/10.20895/jtece.v5i1.843>
- Badan Pusat Statistik. (2026). *Jumlah Penduduk Pertengahan Tahun (Ribuan Jiwa), 2025*. Badan Pusat Statistik. <https://www.bps.go.id/id/statistics-table/2/MTk3NSMy/mid-year-population--thousand-people-.html>
- Badan Siber dan Sandi Negara. (2025). *LANSKAP KEAMANAN SIBER INDONESIA 2024*.
- Chandra, J. C. (2022). Analisis Keamanan Layanan E-Learning Terhadap Serangan Dos Dan Implementasi Mitigasi Pada Universitas Budi Luhur. *Jurnal TICOM: Technology of Information and Communication*, 10(3), 197–203. <https://elearning.budiluhur.ac.id>,
- Chen, Z., Zhang, W., Tang, Y., Shu, R., Ren, F., Xu, T., & Liu, J. (2025). *Principled Performance Tunability in Operating System Kernels*. <http://arxiv.org/abs/2512.12530>
- Cloudflare. (2026). *What is the OSI Model?* Cloudflare. <https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/>
- Di Meglio, S., Starace, L. L. L., & Di Martino, S. (2026). Web app performance testing in industrial contexts: Supporting workload generation with E2E-Loader++. *Journal of Systems and Software*, 232. <https://doi.org/10.1016/j.jss.2025.112684>
- Fachrudin, M. R., & Affandi, A. S. (2025). Implementation and Analysis of Container Image Optimization Using Alpine Linux and Multi-Stage Builds. *Teknika*, 14(1), 9–18. <https://doi.org/10.34148/teknika.v14i1.1118>
- Fadhilah, F., Wahyudi, E., & Cahyadi, E. F. (2023). ANALISIS MODSECURITY DAN MODANTILORIS PADA SERANGAN DDOS SLOWHTTP

- TERHADAP WEB SERVER. *JURNAL LITEK: Jurnal Listrik Telekomunikasi Elektronika*, 20(2), 49–53. <http://192.168.121.221/>
- Fathony, I. A. N. (2022). *KONTAINERISASI SHIBBOLETH IDP SEBAGAI AKSES MANAJEMEN SINGLE SIGN ON PADA ARSITEKTUR MICROSERVICE SISTEM ENTERPRISE DENGAN METODE LOAD BALANCING*.
- filatof. (2025). *Устанавливаем кластер Kubernetes в LXC-контейнерах Proxmox*. Habr. <https://habr.com/ru/articles/901260/>
- Flauzac, O., Mauhourat, F., & Nolot, F. (2020). A review of native container security for running applications. *Procedia Computer Science*, 175, 157–164. <https://doi.org/10.1016/j.procs.2020.07.025>
- Gautham S, Abhijit, M., & Prof. Sahana .B. (2021). Deployment of Applications Using Nginx Ingress Controller. *Journal of University of Shanghai for Science and Technology*, 23(7), 352–357.
- Geges, S., & Wibisono, W. (2015). PENGEMBANGAN PENCEGAHAN SERANGAN DISTRIBUTED DENIAL OF SERVICE (DDOS) PADA SUMBER DAYA JARINGAN DENGAN INTEGRASI NETWORK BEHAVIOR ANALYSIS DAN CLIENT PUZZLE. *JUTI*, 13, 53–67.
- Godinho, A., Rosado, J., Sá, F., & Cardoso, F. (2025). Evaluating Linux Distributions and Kernel Versions Through a Standardized Performance Test Battery. *Procedia Computer Science*, 263, 377–384. <https://doi.org/10.1016/j.procs.2025.07.046>
- Gunawan, R., Aulia, S., Supeno, H., Wijanarko, A., Uwiringiyimana, J. P., & Mahayana, D. (2020). ADIKSI MEDIA SOSIAL DAN GADGET BAGI PENGGUNA INTERNET DI INDONESIA. *Jurnal Techno-Socio Ekonomika*, 14(1).
- Hanipah, R., & Dhika, H. (2020). ANALISA PENCEGAHAN AKTIVITAS ILEGAL DIDALAM JARINGAN DENGAN WIRESHARK. *Journal of Computer and Information Technology*, 4(1), 11–23. <http://e-journal.unipma.ac.id/index.php/doubleclickTelepon:>

- Harijanto, B., & Ariyanto, Y. (2015). DESAIN DAN ANALISIS KINERJA VIRTUALISASI SERVER MENGGUNAKAN PROXMOX VIRTUAL ENVIRONMENT. *Jurnal SimanteC*, 5(1), 17–24.
- Harry Setiawan, & Depandi Enda. (2024). Pengujian Load Testing Website Jurusan Teknik Informatika Politeknik Negeri Bengkalis. *JEKIN - Jurnal Teknik Informatika*, 5(1), 1–12. <https://doi.org/10.58794/jekin.v5i1.820>
- Hidayat, N., & Nugroho, M. A. (2025). ANALISIS CELAH KEAMANAN PADA WEBSITE SMA NEGERI 3 BERAU DENGAN METODE PENETRATION TESTING. *Journal of Information System Management (JOISM) e-ISSN*, 6(2), 102–108.
- Ilham, M., Atthariq, & Mursyidah. (2024). Implementasi Sistem Load Balancing untuk Optimasi Kinerja pada Web Server Nginx Menggunakan Algoritma Ip Hash. *Jurnal Infomedia : Teknik Informatika, Multimedia, dan Jaringan*, 9, 60–68.
- Indrajid, F., Andika, K. F., Putra, G. K. S. A., Bramanda, K. K., Saskara, G. A. J., & Listartha, I. M. E. (2023). *Analisis Hasil DoS SYN Flood Attack Pada Web Server*. 12.
- Ismail, A., Ananta, A. Y., Arief, S. N., & Hamdana, E. N. (2023). PERFORMANCE TESTING SISTEM UJIAN ONLINE MENGGUNAKAN JMETER PADA LINGKUNGAN VIRTUAL. *JIP (Jurnal Informatika Polinema)*, 9(2), 159–164.
- Iwana, A. A., Huwae, R. B., & Jatmika, A. H. (2025). THREAT MODELING MENGGUNAKAN PENDEKATAN STRIDE DAN DREAD UNTUK MENGETAHUI RISIKO DAN MITIGASI KEAMANAN PADA SISTEM LAYANAN PENDIDIKAN. *Jurnal Teknologi Informasi, Komputer dan Aplikasinya*, 7, 142–153. <http://jtika.if.unram.ac.id/index.php/JTIKA/>
- Jupriyadi, J., Hijriyanto, B., & Ulum, F. (2021). Komparasi Mod Evasive dan DDoS Deflate Untuk Mitigasi Serangan Slow Post. *Techno.Com*, 20(1), 59–68. <https://doi.org/10.33633/tc.v20i1.4116>

- Kiperberg, M., & Zaidenberg, N. J. (2023). HyperIO: A Hypervisor-Based Framework for Secure IO. *Applied Sciences (Switzerland)*, 13(9). <https://doi.org/10.3390/app13095232>
- Kusuma, G. H. A. (2022). IMPLEMENTASI OWASP ZAP UNTUK PENGUJIAN KEAMANAN SISTEM INFORMASI AKADEMIK. *JURNAL TEKNOLOGI INFORMASI*, 16, 178–186. <https://doi.org/10.47111/JTI>
- Mhd.Fakhmi, & Gultom, L. M. (2021). Peningkatan Keamanan Router Mikrotik Terhadap Serangan Syn Flood dengan Menggunakan Firewall Raw (Studi kasus : Sekolah Menengah Kejuruan Negeri 3 Bengkalis). *Seminar Nasional Industri dan Teknologi (SNIT)*, 260–277.
- Muhammad, H. H., Hadiana, A. I., & Ashaury, H. (2023). PENGAMANAN APLIKASI WEB DARI SERANGAN SQL INJECTION DAN CROSS SITE SCRIPTING MENGGUNAKAN WEB APPLICATION FIREWALL. *Jurnal Mahasiswa Teknik Informatika*, 7(5), 3265–3273.
- Muttaqin, Moh., Pusvita, V., Pangaribuan, T., & Simatupang, O. (2024). Partisipasi Pengguna Internet Indonesia dalam Pemanfaatan LAPOR! sebagai Layanan Pengaduan Publik. *Jurnal Pekommas*, 9(2), 189–200. <https://doi.org/10.56873/jpkm.v9i2.5314>
- Nasution, E. Y., Hariani, P., Hasibuan, L. S., & Pradita, W. (2020). Perkembangan Transaksi Bisnis E-Commerce terhadap Pertumbuhan Ekonomi di Indonesia. *Jesya*, 3(2), 506–519. <https://doi.org/10.36778/jesya.v3i2.227>
- NGINX. (2024). *Kubernetes Ingress Controller Deployment and Security with NGINX*. F5, Inc.
- Nguyen, T. T., Yeom, Y. J., Kim, T., Park, D. H., & Kim, S. (2020). Horizontal pod autoscaling in kubernetes for elastic container orchestration. *Sensors (Switzerland)*, 20(16), 1–18. <https://doi.org/10.3390/s20164621>
- Permana, E. R., Wahyu, F. N., Taufik, H., & Thoyyibah. T. (2024). The OSI and TCP/ IP Reference Models in the Era of Industry 4.0. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, 4(3), 936–942. <https://doi.org/10.57152/malcom.v4i3.1324>

- Permatasari, D. I., Santoso, B., Ningtias, N., M. Halim Y.R, Atika, R., Widad, N., Maulana, I., & Abdurrahman R, A. (2019). PENGUKURAN THROUGHPUT LOAD TESTING MENGGUNAKAN TEST CASE SAMPLING GORILLA TESTING. *Seminar Nasional Sistem Informasi*, (4), 2008–2014.
- Pratama, R. K. A., & Supatman. (2024). Implementasi HAProxy sebagai External Traffic Load Balancer Cluster Kubernetes yang Berjalan di atas Openstack. *Jurnal Restikom : Riset Teknik Informatika dan Komputer*, 6(3), 462–473. <https://restikom.nusaputra.ac.id>
- Annisa, A. R., Ardiana, M., Rahayu, P. N., Brian, T., Jami'in, M. A., Surabaya, N., & Kapal, B. (2026). Performance Evaluation of Web Applications Using JMeter Load Testing for Server Capacity and Response Efficiency. *Journal of Applied Informatics and Computing (JAIC)*, 10(1), 585–590. <http://jurnal.polibatam.ac.id/index.php/JAIC>
- Ramadhani, W., & Ridha, M. A. F. (2022). PERBANDINGAN KINERJA INGRESS CONTROLLER PADA KUBERNETES MENGGUNAKAN TRAEFIK DAN NGINX. *Jurnal Komputer Terapan*, 8(2), 289–295. <https://jurnal.pcr.ac.id/index.php/jkt/>
- Robinson, Akbar, M., & Ridha, M. A. F. (2018). SQL Injection and Cross Site Scripting Prevention Using OWASP Web Application Firewall. *INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION*, 2, 286–292.
- Sahren, Dalimunthe, R. A., Saputra, H., & Sirni, D. Y. K. (2023). IDPS PERFORMANCE ANALYSIS FOR MITIGATING SQL INJECTIONS AND SYN FLOOD ATTACKS. *JURTEKSI (Jurnal Teknologi dan Sistem Informasi)*, 10(1), 171–178. <https://doi.org/10.33330/jurteks.v10i1.2880>
- Sahren, & Lubis, A. P. (2024). INTRUSION DETECTION SYSTEM BERBASIS DEEP LEARNING UNTUK PENINGKATAN MITIGASI SQL INJECTION DAN SYN FLOOD ATTACK. *Journal of Science and Social Research*, 4(4), 1866–1874. <http://jurnal.goretanpena.com/index.php/JSSR>
- Saputra, P. S., Pratama, P. A., & Tjahyanti, L. P. A. S. (2023). PERANCANGAN DAN KOMPARASI WEB SERVER NGINX DENGAN WEB SERVER

- APACHE SERTA PEMANFAATAN REVERSE PROXY SERVER PADA NGINX. *Jurnal Komputer dan Teknologi Sains (KOMTEKS)*, 2(1), 16–21.
- Saputro, A. R., Nurchim, & Atina, V. (2025). IDENTIFIKASI ANOMALI KEAMANAN SERVER NGINX MENGGUNAKAN ALGORITMA ISOLATION FOREST. *Jurnal Mahasiswa Teknik Informatika*, 9(2), 2465–2471. <https://example.com>
- Sari, W. T. W., Purwantoro E.S.G.S, S., & Ridha, M. A. F. (2023). Pengukuran Kinerja Kubernetes Cluster pada Nested Virtualization Berbasis KVM. *Jurnal Komputer Terapan*, 9(1), 11–18. <https://doi.org/10.35143/jkt.v9i1.5606>
- Sarmah, U., Borah, P., & Bhattacharyya, D. K. (2025). *Streamlining HTTP Flooding Attack Detection through Incremental Feature Selection*. <http://arxiv.org/abs/2505.17077>
- Satria, C. G., & Yudistira, B. G. K. (2025). Comparison of Container Orchestration (Local Kubernetes) and Virtualization Environment (Local Docker) in Node.js Application Management Informasi Artikel ABSTRAK. *Journal of Artificial Intelligence and Software Engineering*, 5(3), 1265–1277. <https://doi.org/10.30811/jaise.v5i3.7266>
- Shankar, V. (2025). Machine Learning for Linux Kernel Optimization Current Trends and Future Directions. *International Journal of Computer Sciences and Engineering*, 13(3), 56–64. <https://doi.org/10.26438/ijcse/v13i3.5664>
- Sheverando, W., & Wijaya, A. F. (2025). Implementasi Layer-7 Load Balancing Dengan Sistem Geo-Aware Failover Untuk Meningkatkan Reliabilitas Layanan Berbasis Web. *Jurnal Teknologi dan Manajemen Industri Terapan (JTMIT)*, 4(4), 2154–2165.
- Silva, D., Rafael, J., & Fonte, A. (2024). Toward Optimal Virtualization: An Updated Comparative Analysis of Docker and LXD Container Technologies. *Computers*, 13(4). <https://doi.org/10.3390/computers13040094>
- Šimon, M., Huraj, L., & Búčik, N. (2023). A Comparative Analysis of High Availability for Linux Container Infrastructures. *Future Internet*, 15(8). <https://doi.org/10.3390/fi15080253>

- Sugiyatno, & Noviar, H. (2025). Perancangan Topologi Sistem Single Sign On Berbasis CAS dan OpenLDAP. *INSECT*, 11(1), 13–24.
- Suliman, Achmad, A., & Adnan. (2021). IMPLEMENTASI HONEYPOT DAN PORT KNOCKING DALAM MENDETEKSI SERANGAN DDoS ATTACK PADA SERVER JARINGAN. *semanTIK*, 7(1), 17–26. <https://doi.org/10.5281/zenodo.5034918>
- Sumayyah, Z. I., Permana, S. D. S., Tsabit, M., & Setiawan, A. (2024). Penerapan dan Mitigasi Teknik Slowloris dalam Serangan Distributed Denial-of-Service (DDos) terhadap Website Ilegal dengan Kali Linux. *Journal of Internet and Software Engineering*, 1(2), 1–14. <https://doi.org/10.47134/pjise.v1i2.2694>
- Thomas-Reynolds, & Butakov, S. (2020). Factors Affecting the Performance of Web Application Firewall Factors. *Association for Information Systems AIS Electronic Library (AISeL)*. <https://aisel.aisnet.org/wisp2020>
- Tiwari, C. S., & Jha, V. K. (2025). OSI network layers based secured data sharins in cloud computing using STXORSK-QC and dhdecct-MAC. *Discover Computing*, 28(1). <https://doi.org/10.1007/s10791-025-09510-8>
- Tran, N. T., Nguyen, V. H., Nguyen-Le, T., & Nguyen-An, K. (2020). Improving ModSecurity WAF with Machine Learning Methods. *Communications in Computer and Information Science*, 1306, 93–107. https://doi.org/10.1007/978-981-33-4370-2_7
- Wahyu Saputra, D., Siwi Pradini, R., Anshori, M., Teknologi dan Sains, F., Teknologi, I., & Kesehatan dr Soepraoen, dan R. (2025). SECURITY ANALYSIS OF COLLEGE WEBSITES USING VULNERABILITY ASSESSMENT METHOD. *International Journal of Computer Science and Information Technology*, 2(1), 13–20.
- Wiedner, F., Helm, M., Daichendt, A., Andre, J., & Carle, G. (2024). Performance evaluation of containers for low-latency packet processing in virtualized network environments. *Performance Evaluation*, 166. <https://doi.org/10.1016/j.peva.2024.102442>
- Yakubov, D., & Hästbacka, D. (2025). Comparative Analysis of Lightweight Kubernetes Distributions for Edge Computing: Security, Resilience

- and Maintainability. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* , 15547 LNCS, 96–104. https://doi.org/10.1007/978-3-031-84617-5_8
- Yang, J., & Abraham, A. (2024). Analyzing the Features, Usability, and Performance of Deploying a Containerized Mobile Web Application on Serverless Cloud Platforms. *Future Internet*, 16(12). <https://doi.org/10.3390/fi16120475>
- Yenugula, M., Kodam, R., & He, D. (2019). Performance and load testing: Tools and challenges. *International Journal of Engineering in Computer Science*, 1(1), 57–62. <https://doi.org/10.33545/26633582.2019.v1.i1a.102>
- Zain, A. R., Matin, I. M. M., & Kautsar, K. (2023). Analisis Implementasi Modsecurity dan Reverse Proxy Untuk Pencegahan Serangan Keamanan DDoS pada Web Server. *SEMINAR NASIONAL INOVASI VOKASI*, 2(1), 188–127.

LAMPIRAN

Lampiran 1 Set Up Proxmox

```
# Ensure that the system's package index is fully
updated and all existing packages are upgraded to
their latest versions before proceeding with the
following configuration to prevent any dependency
conflicts.
```

- nano /etc/hosts

```
"
127.0.0.1      localhost
::1          localhost      ip6-localhost      ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters

# add IP address, and your hostname, then your
/etc/hosts file could look like:
172.31.8.47    kiraryuuki.systems      kiraryuuki
"
```

- hostname --ip-address

```
172.31.8.47 # should return at least one non-
loopback IP address here
```

- tee /etc/apt/sources.list.d/pve-install-
repo.sources > /dev/null << 'EOF'
Types: deb
URIs: http://download.proxmox.com/debian/pve
Suites: trixie
Components: pve-no-subscription
Signed-By: /usr/share/keyrings/proxmox-archive-
keyring.gpg
EOF

- `wget https://enterprise.proxmox.com/debian/proxmox-archive-keyring-trixie.gpg -O /usr/share/keyrings/proxmox-archive-keyring.gpg`
- `sha256sum /usr/share/keyrings/proxmox-archive-keyring.gpg`
- `md5sum /usr/share/keyrings/proxmox-archive-keyring.gpg`

- `apt modernize-sources`
- `apt update`
- `apt full-upgrade`
- `apt install proxmox-default-kernel`
- `systemctl reboot`

- `apt install proxmox-ve postfix open-iscsi chrony`
- `apt remove linux-image-amd64 'linux-image-6.12*'`
- `update-grub`
- `systemctl reboot`

- `pveversion`
- `rm /etc/apt/sources.list.d/pve-install-repo.list`

Lampiran 2 Manage Network Interface

- `nano /etc/network/interfaces`

```
"
auto lo
iface lo inet loopback

iface enp39s0 inet manual

# create vmbr0
auto vmbr0
iface vmbr0 inet static
    address 192.168.45.1/24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
"
```

- `tee /etc/sysctl.d/k8s.conf > /dev/null << 'EOF'`
`net.ipv4.ip_forward = 1`
`net.bridge.bridge-nf-call-iptables = 1`
`net.bridge.bridge-nf-call-ip6tables = 1`
`EOF`

- `sysctl -f /etc/sysctl.d/k8s.conf`
- `sysctl net.ipv4.ip_forward net.bridge.bridge-nf-call-iptables net.bridge.bridge-nf-call-ip6tables`
- `sysctl --system`

```
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

maybe need to restart

- `apt install iptables-persistent nginx libnginx-mod-stream certbot python3-certbot-nginx`

- `iptables -A INPUT -i lo -j ACCEPT`
- `iptables -t nat -A POSTROUTING -s 192.168.45.0/24 -o enp39s0 -j MASQUERADE`
- `iptables -A FORWARD -i vmbr0 -o enp39s0 -j ACCEPT`
- `iptables -A FORWARD -i enp39s0 -o vmbr0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT`
- `iptables -A FORWARD -p vrrp -j ACCEPT`

- `iptables -t nat -L -n -v`
- `iptables-save > /etc/iptables/rules.v4`

verify file contents

- `cat /etc/iptables/rules.v4`

- `netfilter-persistent save`

if the component is not present in the kernel

- `modprobe overlay`
- `modprobe br_netfilter`

- `lsmod | grep overlay`
- `lsmod | grep br_netfilter`

make sure the kernel module is enabled

- `nano /etc/modules`

```
"
overlay
br_netfilter
ip_vs
nf_nat
xt_conntrack
"
```

Lampiran 3 Configuration Lxc (Host Proxmox)

- pveam update **# update the repository**
- pveam available **# list all available images**
- pveam download local debian-13-standard_13.1-2_amd64.tar.zst

check if it has been downloaded

- ls /var/lib/vz/template/cache/

debian-13-standard_13.1-2_amd64.tar.zst

- pct create 100 \
 /var/lib/vz/template/cache/debian-13-standard_13.1-2_amd64.tar.zst \
 --hostname kubernetes \
 --password '*****' \
 --arch amd64 \
 --cores 2 \
 --memory 2048 \
 --swap 0 \
 --rootfs local:10 \
 --storage local \
 --net0
 name=eth0,bridge=vmbr0,ip=192.168.45.10/24,gw=192.168.45.1 \
 --features nesting=1,keyctl=1 \
 --unprivileged 0

- nano /etc/pve/lxc/100.conf

```
"
arch: amd64
cores: 2
features: nesting=1,keyctl=1
hostname: kubernetes
memory: 2048
net0:
    name=eth0,bridge=vbr0,gw=192.168.45.1,hwaddr=BC:24
    :11:D6:3D:92,ip=192.168.45.10/24,type=veth
ostype: debian
rootfs: local:100/vm-100-disk-0.raw,size=10G
swap: 0
unprivileged: 0

# add at the end of the file:
lxc.apparmor.profile: unconfined
lxc.cap.drop:
lxc.cgroup2.devices.allow: a
lxc.mount.auto: proc:rw sys:rw
"
```


Lampiran 4 Preparing Containers For Kubernetes

- `pct start 100`
- `pct enter 100`

- `ln -s /dev/console /dev/kmsg`
- `echo 'L /dev/kmsg - - - - /dev/console' > /etc/tmpfiles.d/kmsg.conf`
- `ls -l /dev/kmsg`

```
lrwxrwxrwx 1 root root 12 Apr  9 10:25 /dev/kmsg -> /dev/console
```

- `swapoff -a`
- `sed -i '/ swap / s/^/#/' /etc/fstab`
- `swapon -s`

Lampiran 5 Runtime Container

containerd

- wget
<https://github.com/containerd/containerd/releases/download/v1.6.2/containerd-1.6.2-linux-amd64.tar.gz>
- tar Cxzvf /usr/local containerd-1.6.2-linux-amd64.tar.gz
- wget
<https://raw.githubusercontent.com/containerd/containerd/main/containerd.service>

runc

- wget
<https://github.com/opencontainers/runc/releases/download/v1.2.6/runc.amd64>
- install -m 755 runc.amd64 /usr/local/sbin/runc

cni

- wget
<https://github.com/container Networking/plugins/releases/download/v1.6.2/cni-plugins-linux-amd64-v1.6.2.tgz>
- mkdir -p /opt/cni/bin
- tar Cxzvf /opt/cni/bin cni-plugins-linux-amd64-v1.6.2.tgz

- `mv containerd.service /usr/lib/systemd/system/containerd.service`
- `systemctl daemon-reload`
- `systemctl enable --now containerd`
- `ln -s /usr/local/bin/containerd /usr/bin/containerd`
- `mkdir -p /etc/containerd`
- `containerd config default | tee /etc/containerd/config.toml`
- `nano /etc/containerd/config.toml`

```
"  
# tune containerd config  
SystemdCgroup = true  
"
```

- `systemctl restart containerd`

Lampiran 6 Installing Kubeadm, Kubectl, And Kubelet

- `apt update`
- `apt install apt-transport-https ca-certificates
curl gpg`
- `curl -fsSL
https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release
.key | gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg`
- `echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-
apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /' |
tee /etc/apt/sources.list.d/kubernetes.list`

- `apt update`
- `apt install kubelet kubeadm kubectl`
- `apt-mark hold kubelet kubeadm kubectl`
- `systemctl enable --now kubelet`

Lampiran 7 Creating An LXC Template

- `pct stop 100`
- `pct template 100`

master node

- `pct clone 100 200 --hostname master --full 1`
- `pct set 200 --net0
name=eth0,bridge=vbr0,ip=192.168.45.20/24,gw=192.168.45.1`
- `pct set 200 --cpuunits 5000`

worker node

- `pct clone 100 300 --hostname worker --full 1`
- `pct set 300 --net0
name=eth0,bridge=vbr0,ip=192.168.45.30/24,gw=192.168.45.1`
- `pct set 300 --cpuunits 500`

Lampiran 8 Load Balancing On Master Node

- `apt install keepalived haproxy cri-tools`
- `mkdir -p /etc/keepalived`
- ```
tee /etc/keepalived/keepalived.conf > /dev/null <<
'EOF'
global_defs {
 enable_script_security
 script_user nobody
}
vrrp_script check_apiserver {
 script "/etc/keepalived/apiserver.sh"
 interval 3
}
vrrp_instance VI_1 {
 state BACKUP
 interface eth0
 virtual_router_id 5
 priority 500
 advert_int 1
 nopreempt
 authentication {
 auth_type PASS
 auth_pass *****
 }
 virtual_ipaddress {
 192.168.45.5/24
 }
 track_script {
 check_apiserver
 }
}
EOF
```

- ```
tee /etc/keepalived/apiserver.sh > /dev/null <<
'EOF'
#!/bin/sh
APISERVER_VIP=192.168.45.5
APISERVER_DEST_PORT=6443
PROTO=https

errorExit() {
    echo "*** $" 1>&2
    exit 1
}

curl --silent --max-time 2 --insecure
${PROTO}://localhost:${APISERVER_DEST_PORT}/hea
lthz -o /dev/null || errorExit "Error GET
${PROTO}://localhost:${APISERVER_DEST_PORT}/hea
lthz"

if ip addr | grep -q ${APISERVER_VIP}; then
    curl --silent --max-time 2 --insecure
    ${PROTO}://${APISERVER_VIP}:${APISERVER_DES
T_PORT}/healthz -o /dev/null || errorExit
    "Error GET
    ${PROTO}://${APISERVER_VIP}:${APISERVER_DES
T_PORT}/healthz"
fi
EOF
```

- `chmod +x /etc/keepalived/apiserver.sh`
- `systemctl enable --now keepalived`
- `systemctl enable --now haproxy`

Lampiran 9 Join And Configure The Network Plugin

launch the first control node

- `kubeadm init --pod-network-cidr=10.244.0.0/16 --control-plane-endpoint "192.168.45.5:6443" --upload-certs --ignore-preflight-errors=SystemVerification`

add --control-plane and --certificat-key to join as Master

- `kubeadm join 192.168.45.5-6443 --token [token] \ --discovery-token-ca-cert-hash sha256:[ca-cert] \ --control-plane --certificate-key [key] \ --ignore-preflight-errors=SystemVerification`

- `kubectl label node worker-rex worker-roxy worker-ki node-role.kubernetes.io/field-area=worker`

- `mkdir -p $HOME/.kube`
- `cp -i /etc/kubernetes/admin.conf $HOME/.kube/config`
- `chown $(id -u):$(id -g) $HOME/.kube/config`

- `kubectl get nodes`
- `export KUBE_EDITOR="nano"`
- `kubectl edit configmap -n kube-system kube-proxy`

```
"
conntrack:
  maxPerCore: 0
  min: 0
  tcpBeLiberal: false
  tcpCloseWaitTimeout: 0s
  tcpEstablishedTimeout: 0s
  udpStreamTimeout: 0s
  udpTimeout: 0s
"
```


clean up error pods

- `kubectl get pods -A`
- `kubectl get pods -A | grep -E 'Error|Pending|CrashLoopBackOff|ContainerCreating' | awk '{print "kubectl delete pod " $2 " -n " $1}' | bash`
- `kubectl get pods -A -w`

install network plugin

- `kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml`

install metrics-server

- `kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml`

install ingress

- `kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.0/deploy/static/provider/baremetal/deploy.yaml`

install certificate

- `kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.yaml`

- `kubectl edit configmap ingress-nginx-controller -n ingress-nginx`

```
"
data:
  allow-snippet-annotations: "true"
  use-forwarded-headers: "true"
  compute-full-forwarded-for: "true"
  http-snippet: |
    proxy_cache_path /tmp/nginx-cache levels=1:2
    keys_zone=static-cache:10m max_size=1g
    inactive=60m use_temp_path=off;
"
```

- `kubectl edit deployment metrics-server -n kube-system`

```
- args:
  - --cert-dir=/tmp
  - --secure-port=10250
  - --kubelet-use-node-status-port
  - --metric-resolution=15s
  - --kubelet-insecure-tls
```

Lampiran 10 NFS Server and Postfix Configuration (Host Proxmox)

Install NFS Server and Postfix Services

- apt install nfs-kernel-server postfix mailutils

Create separate directories for each application

- mkdir -p /mnt/data/wordpress-data
- mkdir -p /mnt/data/mysql-data
- mkdir -p /mnt/data/ghost-data
- mkdir -p /mnt/data/postgres-data

Set Owner & Permissions

- chown -R nobody:nogroup /mnt/data/
- chmod -R 777 /mnt/data/

Register for Export

- nano /etc/exports

```
"
/mnt/data/wordpress-data
    192.168.45.0/24(rw,sync,no_subtree_check,no_root_sq
    uash)
/mnt/data/mysql-data
    192.168.45.0/24(rw,sync,no_subtree_check,no_root_sq
    uash)
/mnt/data/ghost-data
    192.168.45.0/24(rw,sync,no_subtree_check,no_root_sq
    uash)
/mnt/data/postgres-data
    192.168.45.0/24(rw,sync,no_subtree_check,no_root_sq
    uash)
"
```

Refresh Configuration and Restart Service

- exportfs -ra
- systemctl restart nfs-kernel-server

NFS Client Configuration to LXC

- apt install nfs-common

Lampiran 11 Nginx Ingress Deployment

WordPress Ingress Configuration

```

metadata:
  name: wordpress-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/proxy-buffering: "on"
    nginx.ingress.kubernetes.io/configuration-snippet: |
      proxy_set_header X-Forwarded-Proto https;
      proxy_set_header X-Forwarded-Ssl on;

      set $skip_cache 0;

      if ($request_method = POST) {
        set $skip_cache 1;
      }

      if ($query_string != "") {
        set $skip_cache 1;
      }

      if ($http_cookie ~*
"wordpress_logged_in|comment_author|wp-postpass") {
        set $skip_cache 1;
      }

      proxy_cache static-cache;
      proxy_cache_methods GET HEAD;

      proxy_cache_bypass $skip_cache;
      proxy_no_cache $skip_cache;

      proxy_cache_valid 200 302 60m;
      proxy_cache_valid 404 1m;

      proxy_cache_key
"$scheme$request_method$host$request_uri";

      add_header X-Cache-Status
$upstream_cache_status always;
      add_header X-Skip-Cache $skip_cache always;

```

Ghost Ingress Configuration

```

metadata:
  name: ghost-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-
prod"
    nginx.ingress.kubernetes.io/proxy-buffering: "on"
    nginx.ingress.kubernetes.io/configuration-
snippet: |
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Ssl on;

    set $skip_cache 0;

    if ($request_method = POST) {
      set $skip_cache 1;
    }

    if ($query_string != "") {
      set $skip_cache 1;
    }

    if ($request_uri ~* "^/ghost/") {
      set $skip_cache 1;
    }

    if ($request_uri ~* "^/members/") {
      set $skip_cache 1;
    }

    if ($request_uri ~* "^/api/") {
      set $skip_cache 1;
    }

    if ($http_cookie ~* "ghost-admin-api-
session|ghost-members-ssr") {
      set $skip_cache 1;
    }

    proxy_cache static-cache;
    proxy_cache_methods GET HEAD;

    proxy_cache_bypass $skip_cache;
    proxy_no_cache $skip_cache;

```

```

    proxy_cache_valid 200 302 60m;
    proxy_cache_valid 404 1m;

    proxy_ignore_headers Cache-Control Expires;

    proxy_cache_key
"$scheme$request_method$host$request_uri";

    add_header X-Cache-Status
$upstream_cache_status always;
    add_header X-Skip-Cache $skip_cache always;

```

Drupal Ingress Configuration

```

metadata:
  name: drupal-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-
prod"
    nginx.ingress.kubernetes.io/proxy-buffering: "on"
    nginx.ingress.kubernetes.io/configuration-
snippet: |
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Ssl on;

    set $skip_cache 0;

    if ($request_method = POST) {
      set $skip_cache 1;
    }

    if ($query_string != "") {
      set $skip_cache 1;
    }

    if ($request_uri ~* "^/admin/") {
      set $skip_cache 1;
    }

    if ($request_uri ~* "^/user/") {
      set $skip_cache 1;
    }

    if ($request_uri ~* "^/cart/") {

```

```
        set $skip_cache 1;
    }

    if ($http_cookie ~* "SESS|SSESS") {
        set $skip_cache 1;
    }

    proxy_cache static-cache;
    proxy_cache_methods GET HEAD;

    proxy_cache_bypass $skip_cache;
    proxy_no_cache $skip_cache;

    proxy_cache_valid 200 302 60m;
    proxy_cache_valid 404 1m;

    proxy_ignore_headers Cache-Control Expires Set-
Cookie;

    proxy_cache_key
"$scheme$request_method$host$request_uri";

    add_header X-Cache-Status
$upstream_cache_status always;
    add_header X-Skip-Cache $skip_cache always;
```

Konfigurasi Ingress ModSecurity Only

```

annotations:
  cert-manager.io/cluster-issuer: "letsencrypt-
prod"
  nginx.ingress.kubernetes.io/enable-modsecurity:
"true"
  nginx.ingress.kubernetes.io/enable-owasp-core-
rules: "true"
  nginx.ingress.kubernetes.io/modsecurity-snippet:
|
    SecRuleEngine On
    SecRequestBodyAccess On
    SecResponseBodyAccess Off
    SecAuditEngine RelevantOnly
    SecAuditLogParts ABIJDEFHZ
    SecRule REQUEST_HEADERS:Content-Type
"^application/json"
"id:100001,phase:1,nolog,pass,ctl:requestBodyProces
sor=JSON"
    SecRule REQUEST_URI "@beginsWith /ghost/api/"
"id:100002,phase:1,pass,nolog,ctl:ruleEngine=Off"
  nginx.ingress.kubernetes.io/configuration-
snippet: |
    add_header X-WAF-Status "ACTIVE" always;

```


Lampiran 12 Load Testing Automation

```
# This script is designed to run sequential tests for
100 to 1,000 virtual users based on a pre-created
Test Plan
(Test Report - [...] - Combination.jmx)
, and provides a 5-minute cooldown per cycle to
normalize the Kernel/Pod resource load.
```

```
@echo off
setlocal enabledelayedexpansion

for /L %U in (100, 100, 1000) do (
    echo MENJALANKAN PENGUJIAN DENGAN BEBAN %U USER

    jmeter -n -t "Test Report - [...] -
Combination.jmx" -Juser_threads=%U

    echo Pengujian %U User Selesai. Menunggu jeda
pengujian selama 5 menit...
    timeout /t 300 /nobreak
)
echo SEMUA RANGKAIAN PENGUJIAN TELAH SELESAI!
pause
```

```
# This script is executed on the Proxmox Host
terminal to record CPU utilization, memory usage,
process queue (load average), context switches, and
I/O activity every 5 seconds.
```

```
# kernel-monitoring.sh
```

```
#!/bin/bash

USER_THREADS=${1:-100}

SAVE_PATH="/home/kira/result/wordpress"
FILE_NAME="log-kernel-monitoring_wordpress-
incremental_combination-${USER_THREADS}-user.csv"

mkdir -p $SAVE_PATH
if [ ! -f "$SAVE_PATH/$FILE_NAME" ]; then
```

```

        echo
        "Timestamp,Epoch,CPU_User,CPU_System,CPU_IOWait,Mem
        _Free_MB,Mem_Cache_MB,Load_Avg,Interrupts,Context_S
        witches,IO_Read_S,IO_Write_S" >
        $SAVE_PATH/$FILE_NAME
    fi

    echo "Kernel Monitoring untuk ${USER_THREADS} User
        dimulai... Tekan [Ctrl+C] untuk berhenti."

    DURASI=625
    INTERVAL=5
    TOTAL_LOOP=$((DURASI / INTERVAL))

    for ((i=1; i<=TOTAL_LOOP; i++)); do
        START_LOOP=$(date +%s.%N)

        VMSTAT=$(vmstat 1 2 | tail -1)
        EPOCH=$(date +%s)

        CPU_US=$(echo $VMSTAT | awk '{print $13}')
        CPU_SY=$(echo $VMSTAT | awk '{print $14}')
        CPU_WA=$(echo $VMSTAT | awk '{print $16}')

        MEM_FREE=$(free -m | awk '/Mem:/ {print $4}')
        MEM_CACHE=$(free -m | awk '/Mem:/ {print $6}')

        INTERRUPTS=$(echo $VMSTAT | awk '{print $11}')
        CS=$(echo $VMSTAT | awk '{print $12}')

        IO_DATA=$(iostat -d 1 2 | grep "nvme0n1" | tail -
1)
        IO_READ=$(echo $IO_DATA | awk '{print $3}')
        IO_WRITE=$(echo $IO_DATA | awk '{print $4}')

        LOAD_AVG=$(uptime | awk -F'load average:' '{
print $2 }' | cut -d, -f1 | xargs)

        echo "$(date
        +%H:%M:%S), $EPOCH, $CPU_US, $CPU_SY, $CPU_WA, $MEM_FREE
        , $MEM_CACHE, $LOAD_AVG, $INTERRUPTS, $CS, $IO_READ, $IO_
        WRITE" >> $SAVE_PATH/$FILE_NAME

        END_LOOP=$(date +%s.%N)
        ELAPSED=$(echo "$END_LOOP - $START_LOOP" | bc)

        SLEEP_TIME=$(echo "$INTERVAL - $ELAPSED" | bc)
    done

```

```
        if (( $(echo "$SLEEP_TIME > 0" | bc -l) )); then
            sleep $SLEEP_TIME
        fi
    done

    echo "Kernel Monitoring untuk ${USER_THREADS} User
        Selesai!"
```

kernel-runner.sh

```
#!/bin/bash

for user in {100..1000..100}
do
    echo "MENJALANKAN MONITORING KERNEL..."
    bash /home/kira/kernel-monitoring.sh $user

    echo "MENUNGGU JEDA..."
    sleep 300
done

echo "SEMUA MONITORING KERNEL SELESAI!"
```

kernel-monitoring.service

```
[Unit]
Description=Kernel Monitoring
After=network.target

[Service]
Type=simple
User=root
ExecStart=/bin/bash /home/kira/kernel-runner.sh
Restart=no

[Install]
WantedBy=multi-user.target
```

```
# This script is executed on the Master Node using
the kubectl command to extract pure memory and CPU
resource allocation data from each application pod,
the Nginx Ingress Controller, as well as extracting
the Web Application Firewall (ModSecurity)
inspection status through real-time log reading.
```

```
# monitoring.sh
```

```
#!/bin/bash

export
  PATH="/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin"
export KUBECONFIG="/root/.kube/config"

USER_THREADS=${1:-100}

SAVE_PATH="/home/result/wordpress"
FILE_NAME="log_wordpress-incremental_combination-
  ${USER_THREADS}-user.csv"

mkdir -p $SAVE_PATH

if [ ! -f "$SAVE_PATH/$FILE_NAME" ]; then
  echo
  "Timestamp,Pod_Name,App_CPU_m,App_Memory_Mi,Ingress
  _CPU_m,Ingress_Memory_Mi,ModSecurity_Status" >
  $SAVE_PATH/$FILE_NAME
fi

echo "Monitoring untuk ${USER_THREADS} User
  dimulai... Tekan [Ctrl+C] untuk berhenti."

DURASI=625
INTERVAL=5
TOTAL_LOOP=$((DURASI / INTERVAL))

for ((i=1; i<=TOTAL_LOOP; i++)); do

  APP=$(kubectl top pods -l app=wordpress --no-
  headers | awk '{print $1,"$2","$3}')
```

```

MODSECURITY=$(kubectl logs -n ingress-nginx
deploy/ingress-nginx-controller --tail=20
2>/dev/null | grep -i "ModSecurity" | tail -1)

if [ -z "$MODSECURITY" ]; then
    MODSECURITY="NONE"
else
    MODSECURITY="BLOCKED"
fi

if [ ! -z "$APP" ]; then
    while read -r LINE; do
        echo "$(date
+%H:%M:%S), $LINE, $INGRESS, $MODSECURITY" >>
$SAVE_PATH/$FILE_NAME
    done <<< "$APP"
fi

sleep $INTERVAL
done

echo "Monitoring untuk ${USER_THREADS} User Selesai!"

```

monitoring-runner.sh

```

#!/bin/bash

for user in {100..1000..100}
do
    echo "MENJALANKAN MONITORING KUBERNETES..."
    bash /home/monitoring.sh $user

    echo "MENUNGGU JEDA..."
    sleep 300
done

echo "SEMUA MONITORING KUBERNETES SELESAI!"

```

monitoring.service

```
[Unit]
Description=Kubernetes Monitoring
After=network.target

[Service]
Type=simple
User=root
Environment=KUBECONFIG=/root/.kube/config
ExecStart=/bin/bash /home/monitoring-runner.sh
Restart=no

[Install]
WantedBy=multi-user.target
```